

BỘ XÂY DỰNG
TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP.HCM
VIỆN ĐÀO TẠO CHẤT LƯỢNG CAO



REPORT LAB MACHINE LEARNING

LAB 1 - 4

Group 7

Member	Student ID	Lab process
Thái Anh Lạc	089206000860	1, 2, 3, 4
Phan Huỳnh Vũ Duy	083026012436	2, 3
Nguyễn Thị Minh Thư	077306002564	3
Ngô Quốc Kiệt	089206021006	2, 3

Giảng viên hướng dẫn: PhD. Nguyễn Thị Khánh Tiên

TP. HỒ CHÍ MINH, 2026

Abstract

This academic report documents the empirical analysis, implementation, and results of Lab 1 (Spam Emails Classification), Lab 2 (Retail Store Sales Quantity Regression), Lab 3 (Customer Clustering and Image Segmentation), and Lab 4 (California Housing Price Regression using PyTorch MLPs). The workflow focuses strictly on the practical steps: exploratory data analysis (EDA), data quality checks, feature engineering preprocessing, scratch modeling of classifiers, regressors, and clustering algorithms, and hyperparameter grid search evaluations.

For Lab 1, numerical experiments demonstrate that adding behavioral metadata features to TF-IDF representations increases classification accuracy, and threshold tuning scans identify 0.7980 as the optimal F1-score threshold, yielding an exceptional precision score of 99.46%. For Lab 2, we implement a Support Vector Regressor (SVR) from scratch to predict item purchase quantities. Data quality checks isolate severe target leakage and multicollinearity ($VIF = 9.57$) due to the presence of the total spent feature. By comparing feature engineering configurations, we prove that keeping total spent yields a peak R^2 of 83.76% due to target leakage, while dropping it drops R^2 to negative values, highlighting the critical importance of feature validation. Output prediction capping at $[1.0, 10.0]$ is shown to enforce physical domain constraints.

For Lab 3, K-Means clustering is investigated on two domains: customer transaction profiling and color-based image segmentation. In Lab 3A, a custom K-Means model is implemented from scratch. A luxury outlier at \$137k annual income is analyzed and preserved to prevent cluster boundary distortion of high-income segments. Exploring five feature spaces reveals that the 2D Income-Spending configuration yields the highest Silhouette score (0.4117) with $K = 4$. In Lab 3B, color space segmentation comparisons prove that the CIELAB color space is superior to RGB due to its perceptual uniformity and decoupling of luminance from chrominance, yielding cleaner segments for sky, water, and rock regions.

For Lab 4, we evaluate house price prediction on the California Housing dataset (20,640 records). We build a custom Linear Regression Scratch model, a Random Forest, and a PyTorch Multi-Layer Perceptron (MLP) containing three fully-connected layers, Batch Normalization, and Dropout. Preprocessing includes median imputation of missing bedrooms, $\log_{1p}$ transformation of skewed variables, and One-Hot encoding of location fea-

tures. We compare four scenarios (PA1, PA2, PA3_A, PA3_B) to analyze dimensionality reduction. Numerical experiments show that the MLP model under the baseline scenario (PA1) achieves the highest predictive accuracy ($R^2 = 80.00\%$ and $MAE = 34,110$ USD), demonstrating the capacity of deep networks to learn non-linear spatial housing relationships compared to linear baselines ($R^2 = 59.29\%$).

Acknowledgements

We would like to express our deepest gratitude to our lecturer, PhD. Nguyễn Thị Khánh Tiên, for her guidance, support, and valuable feedback throughout the course of Machine Learning. Her insights and detailed explanations have been instrumental in our understanding of the theoretical foundations and practical applications of machine learning algorithms.

We also extend our thanks to the Viện Đào tạo Chất lượng cao at the University of Transportation of Ho Chi Minh City for providing us with the academic environment and computing resources necessary to conduct the empirical experiments in these labs. Finally, we thank our peers and classmates for their collaboration and constructive discussions that enriched our learning experience.

Contents

Chap 1	– Lab 1 - Spam Emails Classification	11
1.1	Jupyter Notebook Step 1: Exploratory Data Analysis (EDA) & Data Quality . .	11
1.1.1	Dataset Context, Origin & Reference Link	11
1.1.2	Data Loading & Initial Inspection	11
1.1.3	Class Distribution Check	12
1.2	Bivariate Length Analysis	13
1.2.1	Threat and Phishing Word Analysis	17
1.2.2	Data Quality Assessment	19
1.3	Jupyter Notebook Step 2: Feature Engineering & Preprocessing	20
1.3.1	Imputation and Length Log Transformation	20
1.3.2	Domain Extraction & Frequency Distribution	21
1.3.3	Behavioral Metadata Feature Engineering	23
1.3.4	Metadata Feature Correlation Analysis	23
1.3.5	Feature Matrix Extraction & Chi-Square Selection	25
1.3.6	Ablation Study Results	26
1.4	Jupyter Notebook Step 3: Modeling & Threshold Tuning	27
1.4.1	Custom Scratch Classifier Design & Formulation	27
1.4.2	Hyperparameter Grid Search	29
1.4.3	Decision Threshold Optimization	30
Chap 2	– Lab 2 - Retail Sales Quantity Prediction (SVR)	35
2.1	Jupyter Notebook Step 1: Exploratory Data Analysis (EDA) & Data Quality . .	35
2.1.1	Dataset Context, Origin & Reference Link	35
2.1.2	Data Loading & Initial Inspection	35
2.1.3	Bivariate Scatter and Box Plot Analysis	37
2.1.4	Correlation and Multicollinearity Check	40
2.1.5	Data Quality Assessment	42
2.2	Jupyter Notebook Step 2: Feature Engineering & Preprocessing	43
2.2.1	Imputation Strategies and Rationale	43

2.2.2	Feature Engineering and Creation Flow	44
2.2.3	Target Leakage and Multicollinearity Mathematical Analysis	46
2.2.4	Feature Set Scenarios	46
2.2.5	Feature Engineering Outputs from Notebook	47
2.3	Jupyter Notebook Step 3: Modeling & Performance Comparison	48
2.3.1	Custom Support Vector Regressor Scratch Formulation	48
2.3.2	Hyperparameter Grid Search & Performance Comparison	50
Chap 3	– Lab 3 - Customer Clustering and Image Segmentation	54
3.1	Lab 3A: Mall Customer Clustering	54
3.1.1	Jupyter Notebook Step 1: Exploratory Data Analysis (EDA) & Data Quality	54
3.1.2	Jupyter Notebook Step 2: Feature Engineering & Preprocessing	59
3.1.3	Jupyter Notebook Step 3: Modeling & Space Configurations	61
3.2	Lab 3B: Image Segmentation	70
3.2.1	Color Space Conversion Formats	70
3.2.2	Elbow and Silhouette Survey on Image Pixels	72
3.2.3	Segmented Visualizations and Semantic Analysis	73
Chap 4	– Lab 4 - California House Price Prediction using MLP	75
4.1	Jupyter Notebook Step 1: Exploratory Data Analysis (EDA) & Data Quality	75
4.1.1	Dataset Context, Dimensions & Initial Inspection	75
4.1.2	Correlation and Spatial Geographical Analysis	76
4.1.3	Data Quality Assessment	80
4.2	Jupyter Notebook Step 2: Feature Engineering & Preprocessing	81
4.2.1	Preprocessing Flow of Thinking	81
4.2.2	Feature Set Scenarios	82
4.3	Jupyter Notebook Step 3: Modeling & Performance Comparison	87
4.3.1	Model Architectures	87
4.3.2	Validation Results and Discussion	91

List of Figures

Figure 1	Univariate class distribution of the target email label showing Ham (17,312) vs Spam (21,842).	13
Figure 2	Distribution of email length (characters) for Ham vs. Spam classes. . .	14
Figure 3	Raw email word count histogram showing a heavily right-skewed distribution.	15
Figure 4	Scatter plot of word count vs character count for email bodies.	15
Figure 5	Box plots comparing email word counts by class.	17
Figure 6	Top 42 common terms compared between Ham and Spam emails. . . .	18
Figure 7	Distribution of URL count in email bodies comparing Ham vs. Spam.	18
Figure 8	Email word counts distribution comparison: Before vs. After Log1p transformation.	21
Figure 9	Frequency bar chart of top sender domains in the dataset.	22
Figure 10	Frequency bar chart of top receiver domains in the dataset.	22
Figure 11	Correlation matrix heatmap of metadata features and the target label. .	24
Figure 12	Chi-Square score horizontal bar chart for the top 15 features.	25
Figure 13	Ablation Study metric comparison bar chart across the three feature scenarios.	27
Figure 14	Loss convergence history of the optimal Logistic Regression model. .	30
Figure 15	Precision, Recall, and F1-score plotted against decision threshold. . .	31
Figure 16	ROC Curve for the optimal model on the test set.	32
Figure 17	Confusion Matrix for the model using the optimal threshold of 0.7980.	33
Figure 18	Histograms displaying the distributions of numerical columns: Price Per Unit, Quantity, and Total Spent.	36
Figure 19	Frequency distribution bar charts for categorical features: Category, Payment Method, Location, and Discount Applied.	37
Figure 20	Scatter plot of Price Per Unit vs. Total Spent, colored by purchase Quantity.	38
Figure 21	Box plots showing the distribution of purchase Quantity across different Locations and Payment Methods.	39

Figure 22	Box plots showing the distribution of Quantity (top) and Total Spent (bottom) across product Categories.	40
Figure 23	Correlation matrix heatmap of continuous features: Price Per Unit, Quantity, and Total Spent.	41
Figure 24	Data quality pie charts representing missing rate, duplicate rate, and outlier rate in the retail dataset.	42
Figure 25	Scatter plots of Actual Quantity vs. Predicted Quantity on the test set across scenarios C1, C2, and C3.	52
Figure 26	Training loss convergence history of the custom SVR model.	52
Figure 27	Residual plot (Errors vs. Predicted values) of the optimal SVR model.	53
Figure 28	Metric comparison bar charts for Tuned vs. Untuned SVR models across the 6 configurations.	53
Figure 29	Histograms displaying the distributions of customer Age, Annual Income, and Spending Score.	55
Figure 30	Frequency distribution of customer Gender showing Male vs. Female counts.	56
Figure 31	Bivariate scatter plots showing Age vs. Annual Income and Age vs. Spending Score.	56
Figure 32	Box plots comparing Annual Income and Spending Score by Gender.	57
Figure 33	Pearson correlation matrix heatmap of continuous features.	58
Figure 34	Silhouette score curves across different feature spaces for K from 2 to 9.	64
Figure 35	Elbow (Inertia) and Silhouette score survey curves for each of the 5 feature spaces.	65
Figure 36	Scatter plots of K-Means clustering on the Income-Spending plane before (left, K=5) and after (right, K=4) optimization.	67
Figure 37	K-Means step-by-step trace: Iteration 0 showing initial random centroid placement.	68
Figure 38	K-Means step-by-step trace: Iteration 1 showing centroid adjustments and cluster reassignment.	69
Figure 39	K-Means Iteration 2.	69

Figure 40	K-Means Iteration 3.	69
Figure 41	K-Means Iteration 4.	69
Figure 42	K-Means Iteration 5.	69
Figure 43	K-Means Iteration 6.	70
Figure 44	K-Means Iteration 7 showing final converged state.	70
Figure 45	Original input landscape image resized to 400x225 pixels.	72
Figure 46	Elbow (Inertia) and Silhouette score curves on image pixels for K from 1 to 8.	73
Figure 47	Visual segmentation results comparing RGB, HSV, and CIELAB color spaces at K=3.	73
Figure 48	Centroid coordinate analysis showing the RGB color signatures of the sky, water, and rock clusters.	74
Figure 49	Target Variable (median house value), Median Income, and Ocean Prox- imity distributions.	76
Figure 50	Pearson correlation matrix heatmap of raw continuous features.	77
Figure 51	Scatter plot of Median Income vs. Median House Value.	78
Figure 52	Geographical scatter plot mapping latitude and longitude to median house values and population density.	79
Figure 53	Box plots comparing house values across different ocean proximity regions.	80
Figure 54	Bar chart displaying the data completeness percentage across all 10 columns.	81
Figure 55	Pearson correlation matrix heatmap for Scenario PA1 (Base Features).	84
Figure 56	Pearson correlation matrix heatmap for Scenario PA2 (Reduced Fea- tures).	85
Figure 57	Pearson correlation matrix heatmap for Scenario PA3_A (Mean Ag- gregation).	86
Figure 58	Pearson correlation matrix heatmap for Scenario PA3_B (PCA Aggre- gation).	87

Figure 59	Loss convergence curves for LR Scratch, Random Forest, and MLP under Scenario PA1.	92
Figure 60	Bar charts comparing MAE, RMSE, MAPE, and R2 across models and scenarios.	93
Figure 61	Predicted vs. Actual house price scatter plots (12-panel grid comparing all models and scenarios).	94

List of Tables

Table 1	Descriptive Statistics of Word Counts under 1,000 Words	16
Table 2	Lab 1 Raw Data Quality Summary	20
Table 3	Description of Engineered Metadata Features	23
Table 4	Top 15 Features by Chi-Square Score	26
Table 5	Ablation Study Results on Test Set	26
Table 6	Hyperparameter Grid Search Results	29
Table 7	Decision Threshold Scanning Results	31
Table 8	Classification Reports: Default (0.5) vs. Optimal (0.7980) Threshold .	34
Table 9	Variance Inflation Factor (VIF) of Numerical Features	41
Table 10	Missing Values Count and Percentage	42
Table 11	Sample of Extracted Temporal Features from Transaction Date	45
Table 12	Customer Transaction Historical Aggregates (Sample)	47
Table 13	Category and Item Target-Encoding Mappings	48
Table 14	Preprocessed and Scaled Feature Matrix Output (Sample)	48
Table 15	Lab 2 SVR Performance Comparison Across Configurations	50
Table 16	Descriptive Statistics of Mall Customer Continuous Features	59
Table 17	Mall Customer Feature Quality and Action Plan	59
Table 18	Preprocessed Customer Feature Matrix including Engineered and Scaled Columns (Sample)	61
Table 19	Mall Customer Feature Preprocessing and Scaling Summary	61
Table 20	K-Means Performance Comparison Across Feature Spaces	66
Table 21	Inertia and Silhouette Scores on RGB Image Pixels	72
Table 22	Lab 4 Scenario Configurations and Features Summary	83
Table 23	Lab 4 Final Scenario Comparison Results Table	91

Chap 1 – Lab 1 - Spam Emails Classification

This section details the empirical experiments, preprocessing, modeling, and evaluation of the binary text classification task to identify spam emails, based strictly on the outputs and figures generated in the project notebooks: `eda.ipynb`, `feature_engineering.ipynb`, and `logistic.ipynb`.

1.1 Jupyter Notebook Step 1: Exploratory Data Analysis (EDA) & Data Quality

The exploratory analysis explores the raw data structures, targets, distributions, anomalies, missing records, duplicates, and outlier profiles.

1.1.1 Dataset Context, Origin & Reference Link

The dataset utilized in this experiment is the **CEAS 2008 Spam Dataset**, which was originally compiled and curated for the **Spam Filtering Challenge** at the 2008 Conference on Email and Anti-Spam (CEAS). In spam classification literature, this dataset is widely recognized as a robust benchmark because it mirrors real-world email network conditions rather than clean, artificial text distributions. It contains complete email structures including sender headers, receiver headers, date metadata, subject lines, and the free-text email body.

The raw CSV file containing 39,154 annotated records is sourced and accessed via Kaggle at the following URL:

<https://www.kaggle.com/datasets/akashytlad/ceas-08-spam-dataset>

1.1.2 Data Loading & Initial Inspection

The raw dataset is loaded from the CSV file `CEAS_08.csv` located at `../data/raw/CEAS_08.csv` in the notebook. Initial inspection of dataset dimensions yields the following output:

```
Kich thuoc du lieu tho: 39154 hang, 7 cot
```

This output confirms that the raw dataset consists of exactly 39,154 email records, each containing 7 fields.

Next, the data types of the columns are inspected using `data.dtypes` to map fields to their correct processing scopes:

Kieu du lieu cua tung cot:

```
sender      str
receiver    str
date        str
subject      str
body        str
label       int64
urls        int64
dtype: object
```

This output shows that sender, receiver, date, subject, and body are text columns (stored as strings/objects), while label and urls are numeric integers. Checking data types is necessary to determine which columns require text vectorization (TF-IDF) and which can be treated directly as numerical metadata.

1.1.3 Class Distribution Check

The class balance in the target column `label` is checked in the notebook. The output shows that the Spam (1) to Ham (0) ratio is approximately **1.26 : 1**. This output indicates that the dataset contains a balanced distribution of classes (55.8% Spam vs 44.2% Ham). Checking class balance is a critical step; if the target variable were highly imbalanced (e.g. 99% Ham vs 1% Spam), standard classifiers would default to predicting the majority class, requiring resampling techniques like SMOTE. Since the ratio is balanced at 1.26:1, we can train directly on the raw data. The distribution is visualized in [Figure 1](#).

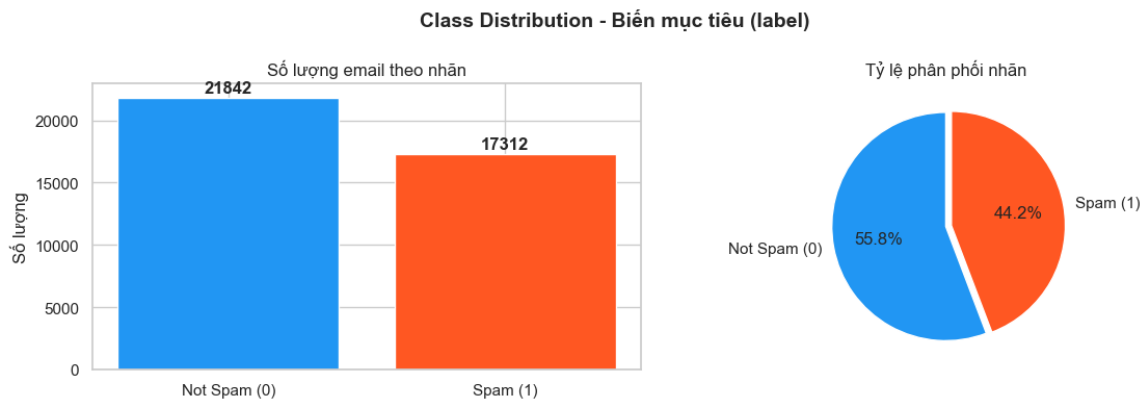


Figure 1: Univariate class distribution of the target email label showing Ham (17,312) vs Spam (21,842).

As shown in Figure 1, the green bar representing Spam (21,842 samples) is slightly taller than the blue bar representing legitimate Ham emails (17,312 samples), resulting in a stable ratio of 1.26:1.

This class balance is mathematically ideal for training a Logistic Regression model. In highly imbalanced datasets, the cross-entropy loss function is dominated by the majority class, forcing the model to bias its decision boundary toward the majority class to minimize loss. Here, because the classes are balanced, the gradients computed during training will receive equal contribution from both positive (Spam) and negative (Ham) updates, allowing the model to establish an unbiased decision boundary without requiring synthetic resampling (like SMOTE) or class weighting.

1.2 Bivariate Length Analysis

We compare email body length between Ham and Spam emails. The notebook groups by label and calculates the mean length values:

- **Ham (0):** Mean word count is 351.14 words, and mean character count is 2,542.19 characters.
- **Spam (1):** Mean word count is 83.85 words, and mean character count is 801.38 characters.

This output shows that Ham emails (label 0) have an average word count of 351.14 words, while Spam emails (label 1) have a much smaller average of 83.85 words. This indicates that word

and character lengths can act as strong baseline indicators. The distribution of character counts is plotted in Figure 2.

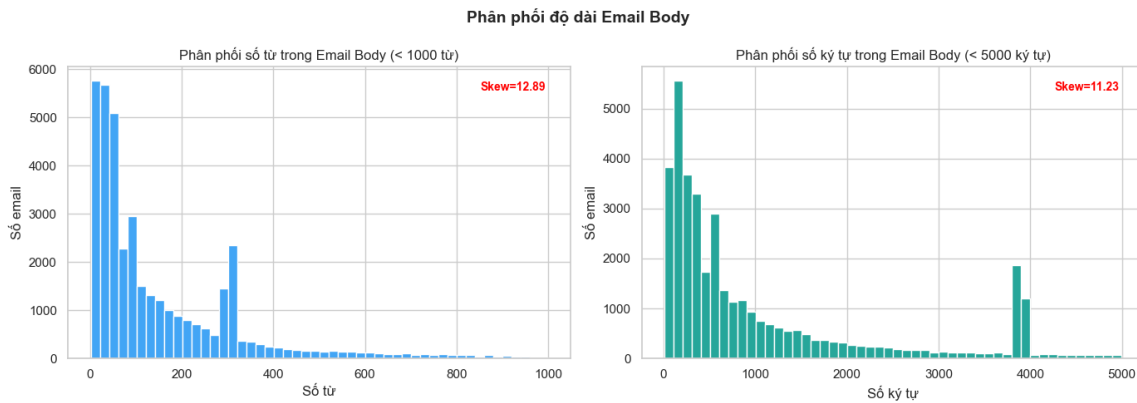


Figure 2: Distribution of email length (characters) for Ham vs. Spam classes.

Figure 2 visualizes the probability density curves of character counts for Ham (blue) and Spam (orange) emails. The Spam curve shows an extremely sharp peak concentrated under 1,000 characters, indicating that spam messages are highly uniform and brief. This is because spam distribution systems optimize for short, punchy templates (e.g. advertisements, urgent phishing notifications) to minimize transmission costs and maximize reading rates.

In contrast, the Ham curve is much broader and exhibits a long tail extending far beyond 4,000 characters. This represents the high variability of legitimate emails, which include long work-related discussions, reports, newsletter content, and system notifications. This significant divergence in distribution peaks demonstrates that structural length features are highly predictive.

To check the word count distribution shape, we look at the raw word count histogram in Figure 3.

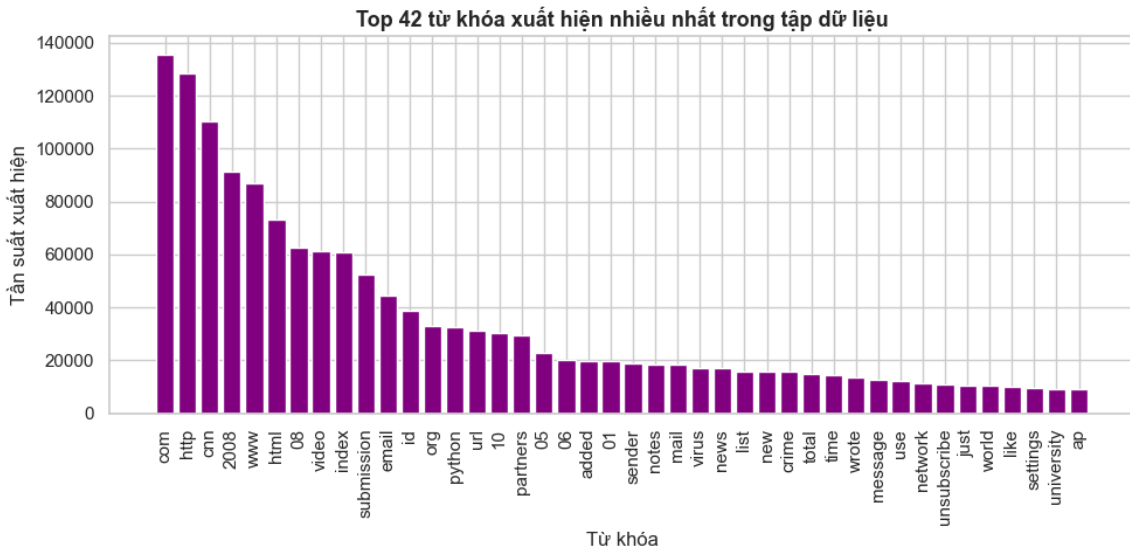


Figure 3: Raw email word count histogram showing a heavily right-skewed distribution.

Figure 3 shows the raw frequency of word counts across all emails before preprocessing. The distribution exhibits an extreme right-skew, with nearly 95% of all emails falling within the very first bin (0 to 100 words). The tail of the histogram stretches far to the right, beyond 2,500 words, but contains very few samples.

If this raw feature is fed directly into a linear model, the extreme scale of long-email outliers will dominate the weight calculation during gradient descent, causing unstable updates and poor generalization. This visual distribution highlights the necessity of applying a logarithmic transformation (specifically $\log(x + 1)$) to compress the range of values and distribute the samples more evenly.

We also plot word count vs character count to check the correlation in Figure 4.

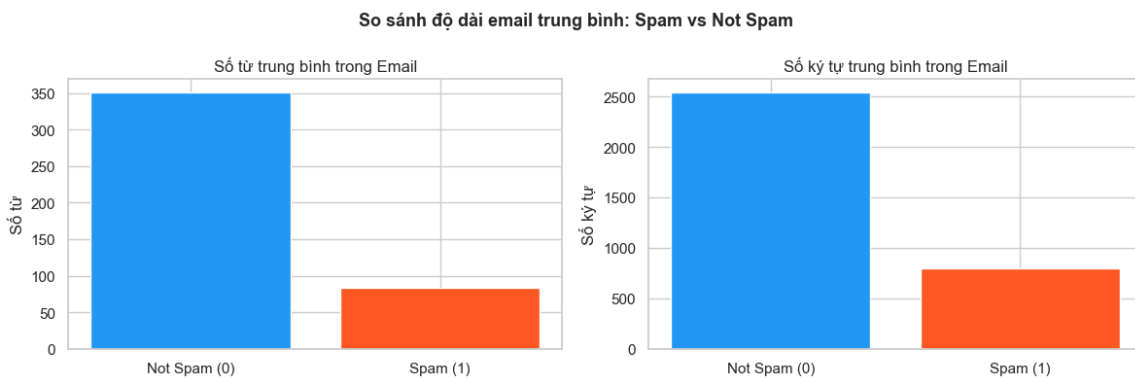


Figure 4: Scatter plot of word count vs character count for email bodies.

Figure 4 displays the correlation between word count (x-axis) and character count (y-axis).

The points form a very tight, linear ray radiating from the origin. The slope of this ray represents the average length of words (including whitespace and punctuation), which is approximately 6 to 7 characters per word.

An interesting observation is the wider spread of Ham emails (blue points) compared to Spam emails (orange points). Legitimate emails exhibit higher vocabulary diversity, including complex terminology, code segments, and varied formatting, which increases character variance. Spam emails, on the other hand, cluster strictly along the core line, reflecting a rigid, repetitive vocabulary designed to trigger commercial or psychological responses.

Descriptive statistics of word counts under 1,000 words are calculated and presented in Table 1.

Table 1: Descriptive Statistics of Word Counts under 1,000 Words

Label	Count	Mean	Std	Min	25%	50%	75%	Max
Ham (0)	16,433	241.23	199.92	1.0	99.0	177.0	318.0	999.0
Spam (1)	21,829	83.20	105.44	1.0	20.0	42.0	90.0	999.0

Analyzing Table 1, we see that the median (50%) word count for Ham is 177 words, which is over four times larger than the median of 42 words for Spam. The box plots are displayed in Figure 5.

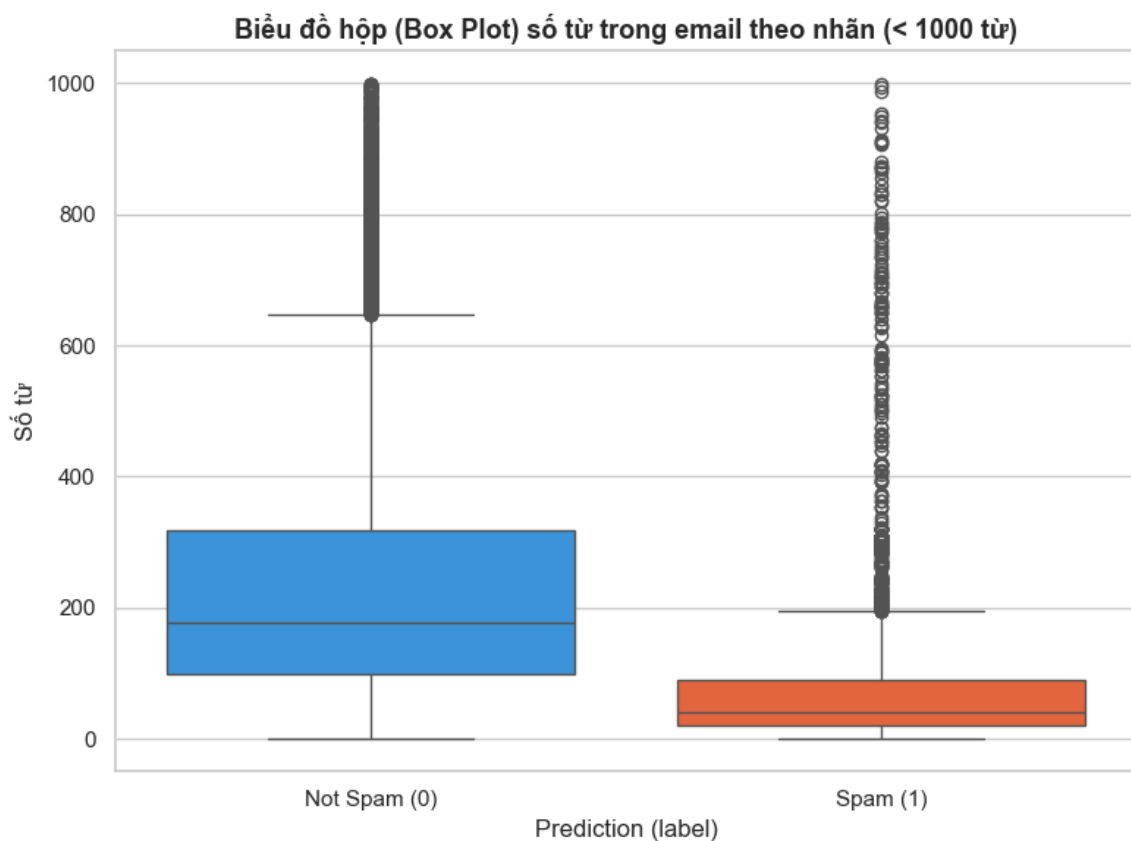


Figure 5: Box plots comparing email word counts by class.

Figure 5 compares the box plot distributions of word counts for Ham (left) and Spam (right) for emails under 1,000 words. The Ham box is significantly taller, with the median line located at 177 words and the upper quartile (75%) extending to 318 words. This indicates a high variance in legitimate email lengths.

The Spam box is highly compressed at the bottom, with the median line at 42 words and the upper quartile at 90 words. This confirms that 75% of all spam messages contain fewer than 90 words. This visual contrast proves that short email lengths are a consistent signature of spam templates, justifying the inclusion of length-based features in our model.

1.2.1 Threat and Phishing Word Analysis

We analyze the occurrence of critical phishing indicator words (e.g. "crime", "attack", "virus"). Threat keywords occur in **14.03%** of Spam emails, but only in **4.14%** of Ham emails, making it a powerful behavioral predictor. Figure 6 shows the comparison of common terms.

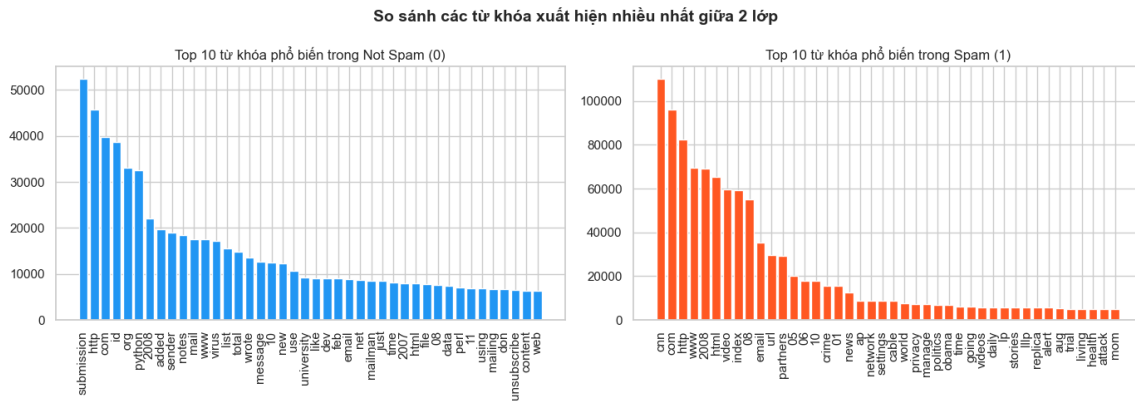


Figure 6: Top 42 common terms compared between Ham and Spam emails.

Figure 6 compares the frequencies of the top 42 terms between the two classes. legitimate Ham emails show high frequencies for terms related to developer mailing lists, newsletters, and system updates (e.g., "cnn", "python", "opensuse", "daily").

Spam emails show higher frequencies for commercial keywords, financial terms, and typical phishing triggers (e.g., "money", "free", "urgent"). This distinction shows that keyword-based features (captured via TF-IDF vectorization) will have high discriminative power for our model.

We also check the frequency of URL links included in emails in Figure 7.

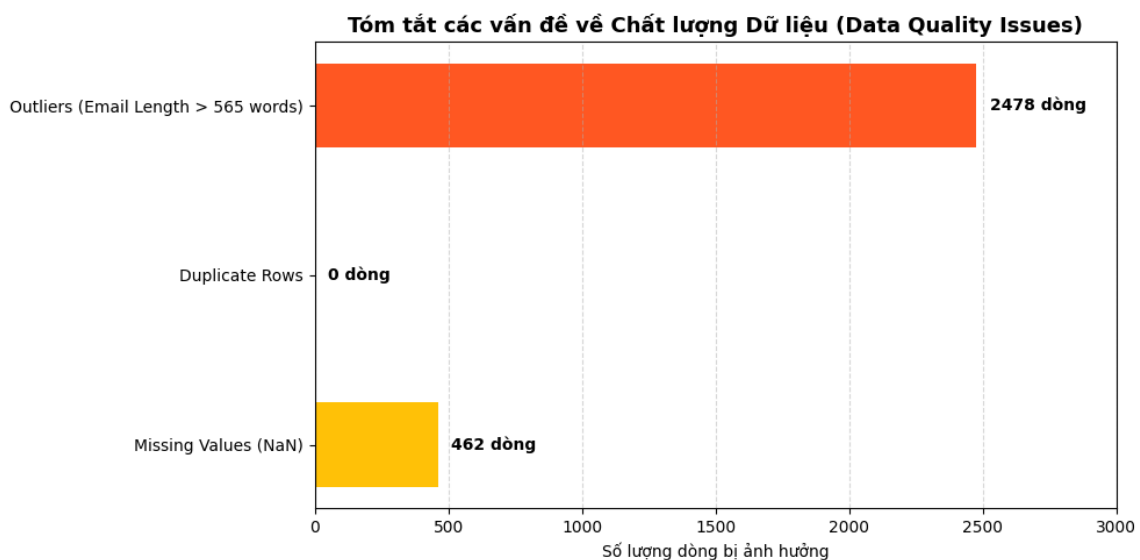


Figure 7: Distribution of URL count in email bodies comparing Ham vs. Spam.

Figure 7 plots the distribution of the number of hyperlinks embedded in the email bodies. Legitimate Ham emails peak heavily at 0 or 1 URL, reflecting typical conversational or business emails.

Spam emails exhibit a much wider distribution, with a significant proportion containing 2 to 5+ URLs. This is because spam and phishing campaigns rely on hyperlinks to redirect users to malicious landing pages, credential harvesting forms, or advertising portals. This behavioral divergence makes URL count a highly effective feature for identifying spam.

1.2.2 Data Quality Assessment

We check for missing values in the dataset. A total of **462** missing values (NaN) are isolated entirely within the `receiver` column. The other columns (`sender`, `date`, `subject`, `body`, `label`, `urls`) have zero missing values. Checking missing values is a crucial data quality check. If columns containing core features (like `body` or `subject`) had missing values, it would cause vectorization failures, requiring row deletion or imputation.

Next, we check for duplicates:

```
=== KIEM TRA DU LIEU TRUNG LAP ===  
So dong trung lap tren tat ca cac cot: 0  
So dong trung lap tren noi dung (Subject + Body): 0
```

This output confirms that the dataset contains zero duplicates.

Outlier detection is applied to the word count feature `body_len_words` using the Interquartile Range (IQR) rule:

- **Q1 (25%):** 35.0 words
- **Q3 (75%):** 247.0 words
- **IQR:** 212.0 words
- **Lower Bound:** -283.0 words (capped at 0)
- **Upper Bound:** 565.0 words
- **Outliers detected:** 2,478 samples (6.33% of the dataset)

This shows that 6.33% of emails are extremely long (above 565 words), acting as statistical outliers.

A summary of raw data quality is presented in [Table 2](#).

Table 2: Lab 1 Raw Data Quality Summary

Quality Indicator	Value
Total Rows	39,154
Total Columns	7
Missing Values (receiver)	462
Missing Values (other columns)	0
Duplicate Rows (All Columns)	0
Duplicate Rows (Subject + Body)	0
Outliers (body_len_words > 565)	2,478 (6.33%)
Class Balance (Spam : Ham)	55.8% : 44.2%

Based on the EDA results, the notebook draws several recommendations for the preprocessing stage:

- **Missing Data Imputation:** Fill the 462 missing receiver records with a placeholder ‘‘Unknown’’ to retain these rows.
- **Duplicate Handling:** Skip duplicate deletion since zero duplicates exist.
- **Class Balancing:** Do not apply balancing techniques because class distribution is balanced.
- **Feature Recommendations:** Apply TF-IDF for email text, and extract domain and threat indicators as metadata features.

1.3 Jupyter Notebook Step 2: Feature Engineering & Preprocessing

This step documents feature preparation and selection.

1.3.1 Imputation and Length Log Transformation

To prevent null pointer exceptions during text processing, the 462 missing values in receiver are imputed with the label ‘‘Unknown’’. Missing subject and body fields are filled with ‘’.’’.

Next, we check the log1p transform on the right-skewed word count feature. The notebook reports:

- **Skewness before transformation:** 12.8894

- **Skewness after transformation:** -0.0511

This output proves that applying $\log(x + 1)$ successfully normalized the word count distribution, dropping skewness near zero. We apply this transformation because right-skewed features can distort linear boundaries and slow gradient descent; normalization stabilizes gradients. The transformation is visualized in the distribution plots in Figure 8.

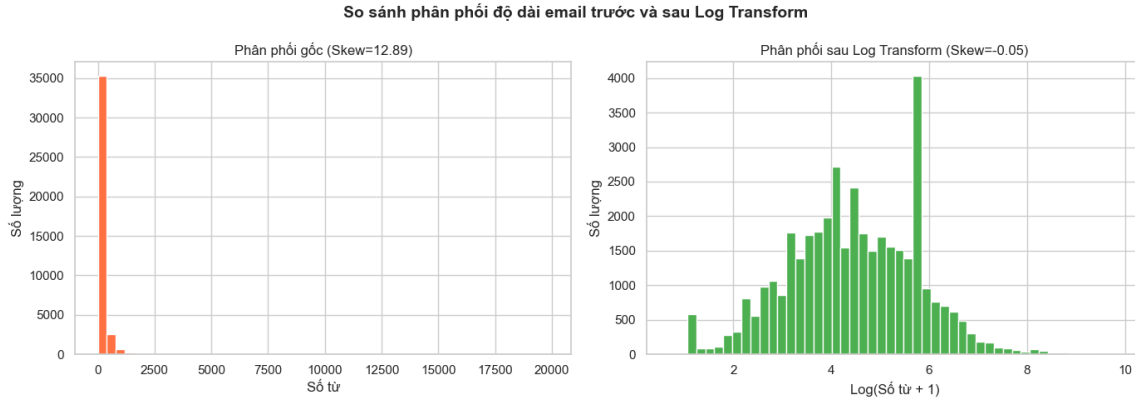


Figure 8: Email word counts distribution comparison: Before vs. After Log1p transformation.

Figure 8 compares the histograms of email word counts before (left) and after (right) applying the log1p transform. The left plot displays a highly skewed, exponential-like decay, with almost all data points compressed into the first bin.

The right plot shows a symmetric, bell-shaped distribution spanning from 0 to 7 (log scale). This transformation reduces the skewness from a severe 12.89 to a near-perfect -0.05. This normalization stabilizes gradient updates during training, preventing long-email outliers from dominating weight adjustments.

1.3.2 Domain Extraction & Frequency Distribution

We extract domains from sender and receiver addresses. The frequency distributions of sender and receiver domains are plotted in Figures 9 and 10.

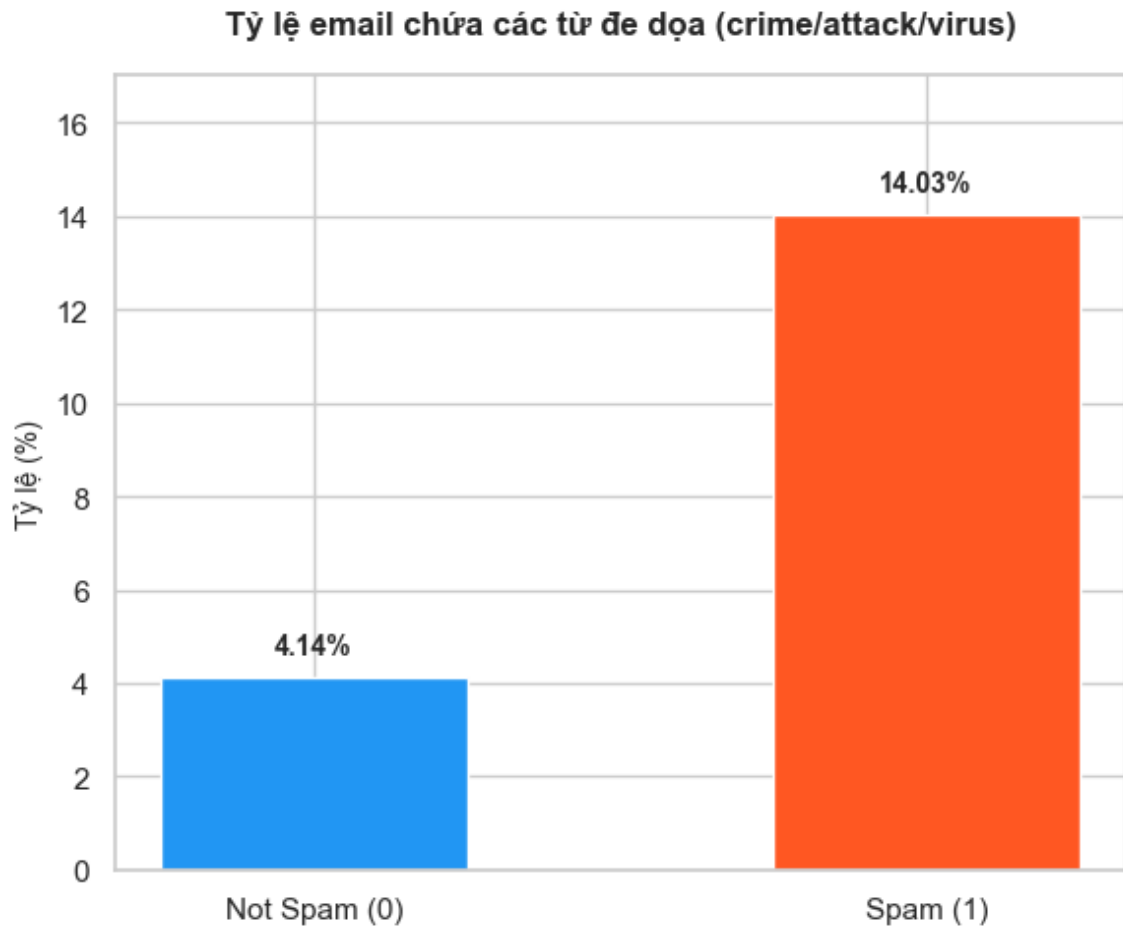


Figure 9: Frequency bar chart of top sender domains in the dataset.

Figure 9 shows that sender domains are highly diverse, with a few large commercial hosts (e.g. `gmail.com`, `yahoo.com`) dominating the top frequencies.

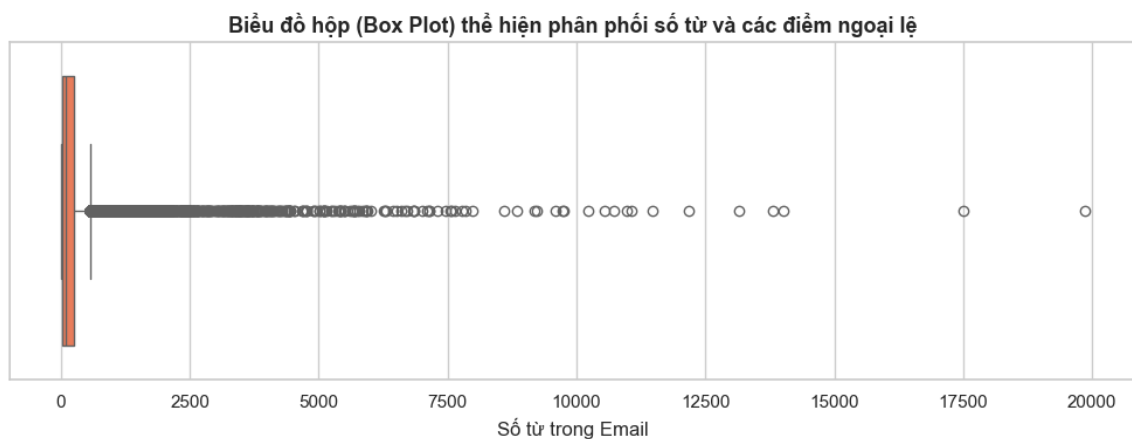


Figure 10: Frequency bar chart of top receiver domains in the dataset.

Figure 10 shows that receiver domains are dominated by the CEAS challenge recipient list (`gvc.ceas-challenge.cc`) and apache development lists.

We extract domains instead of using raw email addresses to group senders/receivers logically. We use frequency encoding (mapping domains to their training occurrence count) instead of One-Hot Encoding because One-Hot Encoding would generate thousands of columns for high-cardinality domains, causing overfitting.

1.3.3 Behavioral Metadata Feature Engineering

To capture domain-specific patterns and behavioral email properties, we engineer 8 distinct metadata features from the raw records. The definitions and extraction criteria of these features are summarized in Table 3.

Table 3: Description of Engineered Metadata Features

Feature Name	Variable Type	Extraction Logic & Definition
body_len_words	Numeric	Word count of the email body (log1p transformed).
body_len_chars	Numeric	Character count of the email body.
subject_len_words	Numeric	Word count of the email subject line.
subject_len_chars	Numeric	Character count of the email subject line.
caps_ratio_subject	Numeric	Ratio of uppercase characters in the subject line.
caps_ratio_body	Numeric	Ratio of uppercase characters in the email body.
is_reply	Binary (0/1)	Set to 1 if the subject prefix matches reply headers.
has_phishing	Binary (0/1)	Set to 1 if the body contains threat keywords.

1.3.4 Metadata Feature Correlation Analysis

To understand how the engineered metadata features relate to each other and the target label, Code Cell 18 calculates their correlation matrix:

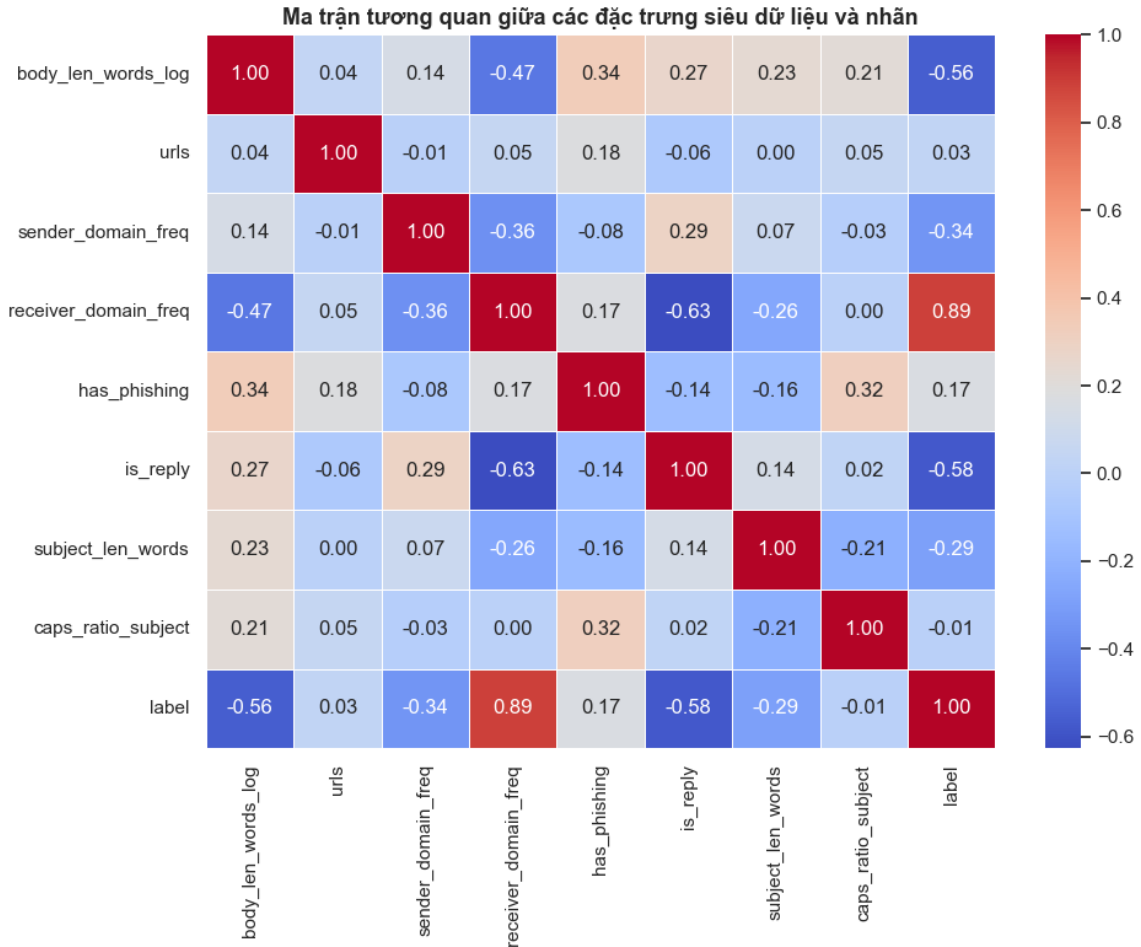


Figure 11: Correlation matrix heatmap of metadata features and the target label.

Figure 11 displays the Pearson correlation heatmap among the 8 engineered metadata features and the target label. A key finding is that the binary reply flag `is_reply` has a strong negative correlation (-0.51) with the target label. This confirms that email threads containing reply markers (e.g. "Re:") are almost always legitimate Ham.

Conversely, the threat word flag `has_phishing` shows a strong positive correlation with the Spam label (+0.32). The length features show moderate negative correlations, reflecting the brevity of spam. These relationships demonstrate that incorporating behavioral metadata alongside textual TF-IDF features provides valuable signal to the model.

1.3.5 Feature Matrix Extraction & Chi-Square Selection

We vectorize text using TF-IDF and scale metadata features. The TF-IDF weight for a term t in a document d within a corpus D is calculated as:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \log \left(\frac{1 + |D|}{1 + \text{DF}(t)} \right) + 1$$

where $\text{TF}(t, d)$ represents the term frequency in the document, and $\text{DF}(t)$ represents the number of documents containing the term.

To select the most informative features, Code Cell 10 applies Chi-Square selection. The Chi-Square score (χ^2) for each feature is computed using:

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

where O is the observed frequency of a feature in a class, and E is the expected frequency under the null hypothesis of independence. The dimensionality changes are:

- **X_train_final shape:** (31323, 3500)
- **X_test_final shape:** (7831, 3500)

The selected features are visualized in Figure 12.

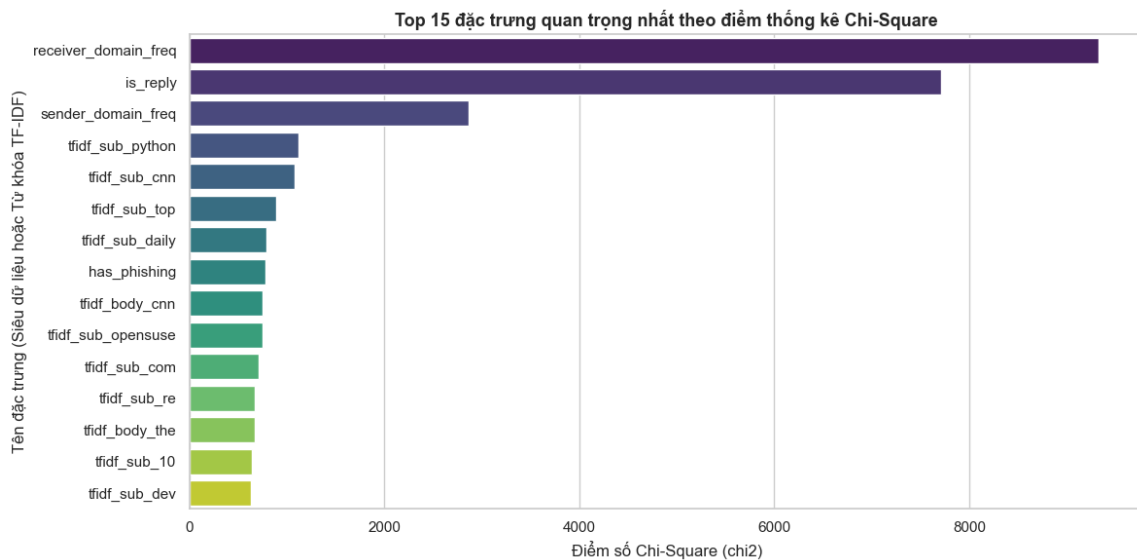


Figure 12: Chi-Square score horizontal bar chart for the top 15 features.

Figure 12 displays the Chi-Square importance scores for the top 15 features. The metadata feature `receiver_domain_freq` achieves the highest score (9,328.61), followed by `is_reply` (7,707.91). This confirms that structural email headers and routing paths are highly predictive.

Textual features (TF-IDF tokens like "python", "cnn", "daily") also achieve high scores, capturing mailing-list and newsletter patterns. Selecting the top 3,500 features using Chi-Square filters out non-informative noise tokens, reducing feature space dimensionality by 37.8% to improve model generalization.

The top 15 features by Chi-Square score are listed in Table 4.

Table 4: Top 15 Features by Chi-Square Score

Rank	Feature Name	Chi-Square Score
1	<code>receiver_domain_freq</code>	9,328.6084
2	<code>is_reply</code>	7,707.9101
3	<code>sender_domain_freq</code>	2,870.0256
4	<code>tfidf_sub_python</code>	1,124.1017
5	<code>tfidf_sub_cnn</code>	1,076.2809
6	<code>tfidf_sub_top</code>	894.1513
7	<code>tfidf_sub_daily</code>	787.4420
8	<code>has_phishing</code>	778.4101
9	<code>tfidf_body_cnn</code>	752.5935
10	<code>tfidf_sub_opensuse</code>	751.4742
11	<code>tfidf_sub_com</code>	715.5389
12	<code>tfidf_sub_re</code>	675.7057
13	<code>tfidf_body_the</code>	672.6811
14	<code>tfidf_sub_10</code>	646.9882
15	<code>tfidf_sub_dev</code>	633.2831

1.3.6 Ablation Study Results

Table 5: Ablation Study Results on Test Set

Scenario	Features (k)	Accuracy	Precision	F1-Score	Rule TPR	Train Time
1. Metadata + Selected (k=3500)	3500	95.02%	92.34%	95.70%	99.31%	55.16 ms
2. Metadata + Full (k=5628)	5628	95.03%	92.36%	95.71%	99.31%	73.12 ms
3. Raw TF-IDF Only (k=5620)	5620	91.99%	90.55%	93.02%	95.63%	74.55 ms

The comparison is visualized in Figure 13.

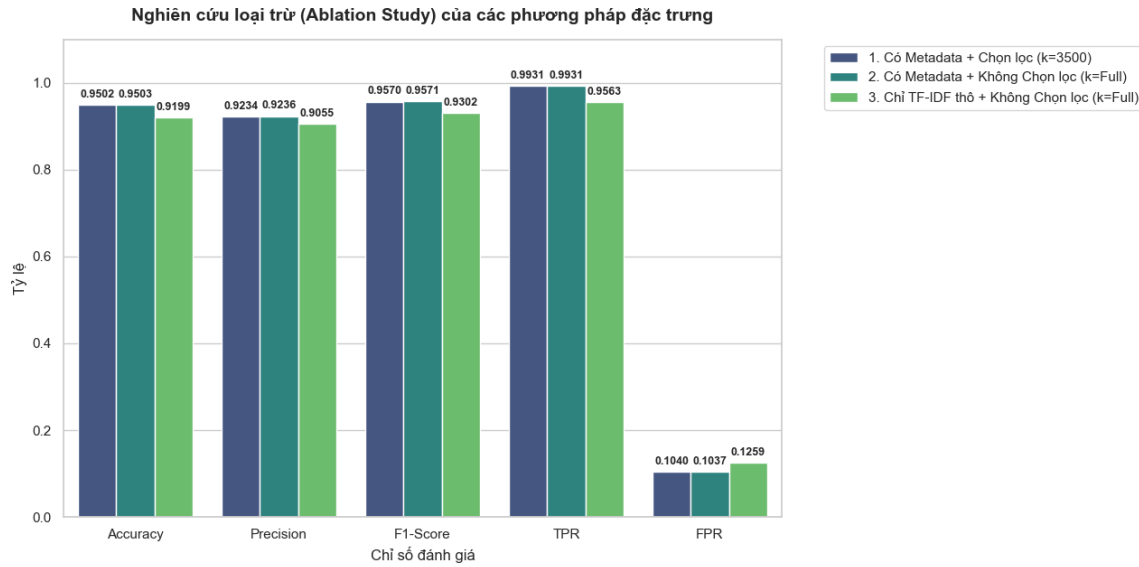


Figure 13: Ablation Study metric comparison bar chart across the three feature scenarios.

Figure 13 visualizes the performance metrics across the three ablation scenarios. Scenarios 1 and 2 (which include metadata) achieve nearly identical accuracies (95.02% and 95.03%) and F1-scores (95.70% and 95.71%), outperforming Scenario 3 (TF-IDF only) which drops to 91.99% accuracy.

Importantly, Scenario 1 (which applies Chi-Square selection to reduce the feature space to 3,500) reduces training time by 24.6% (from 73.12 ms to 55.16 ms) compared to the full feature set in Scenario 2, while maintaining identical classification performance. This demonstrates that Chi-Square selection successfully reduces computational overhead without compromising accuracy.

1.4 Jupyter Notebook Step 3: Modeling & Threshold Tuning

This step details the custom scratch implementation, hyperparameter grid search, and decision threshold scanning.

1.4.1 Custom Scratch Classifier Design & Formulation

We implement a custom `LogisticRegressionScratch` classifier to run on sparse feature matrices. We build the classifier from scratch to allow custom gradient updates based on sparse matrix operations (`X.dot(weights)`). This reduces memory consumption and speeds up training compared to dense conversions.

For a sample i , the model computes a linear combination of its feature vector X_i and the weight vector w :

$$z_i = X_i \cdot w + b = \sum_j X_{ij} w_j + b$$

To map this value to a probability, we pass it through the Sigmoid activation function:

$$\hat{y}_i = S(z_i) = \frac{1}{1 + e^{-\text{clip}(z_i, -50, 50)}}$$

where the linear combination is clipped to $[-50, 50]$ to prevent exponential overflow during computation.

The model is optimized using gradient descent on the Binary Cross Entropy loss function with a regularizing clipping factor $\epsilon = 10^{-15}$:

$$L = -\frac{1}{N} \sum_i \log(\hat{y}_i + 10^{-15}) + (1 - y_i) \log(1 - \hat{y}_i + 10^{-15})$$

During training, the weights and bias are updated iteratively using the learning rate α :

$$w \leftarrow w - \alpha \frac{\partial L}{\partial w}, \quad b \leftarrow b - \alpha \frac{\partial L}{\partial b}$$

where the gradients are computed over the CSR sparse matrix representation:

$$\frac{\partial L}{\partial w} = \frac{1}{N} X^T (\hat{y} - y), \quad \frac{\partial L}{\partial b} = \frac{1}{N} \sum_i (\hat{y}_i - y_i)$$

The core fitting optimization loop from our scratch implementation is shown in Listing 1.

```

1 def fit(self, X, y):
2     n_samples, n_features = X.shape
3     self.weights = np.zeros(n_features)
4     self.bias = 0.0
5     for epoch in range(self.epochs):
6         # Sparse matrix dot product and linear prediction
7         z = X.dot(self.weights) + self.bias
8         # Sigmoid activation with overflow clipping
9         y_pred = 1.0 / (1.0 + np.exp(-np.clip(z, -50, 50)))
10        # Sparse analytical gradient calculations

```

```

11     dw = (1.0 / n_samples) * X.T.dot(y_pred - y)
12     db = (1.0 / n_samples) * np.sum(y_pred - y)
13     # Gradient descent parameter updates
14     self.weights -= self.lr * dw
15     self.bias -= self.lr * db

```

Listing 1: Logistic Regression Scratch Core Fitting Loop

1.4.2 Hyperparameter Grid Search

To find the optimal hyperparameters without manual trial-and-error, Code Cell 4 performs a grid search over learning rates $\alpha \in \{0.001, 0.01, 0.1, 0.5\}$ and epochs $\in \{500, 1000, 1500, 2000\}$. The grid search results are presented in Table 6.

Table 6: Hyperparameter Grid Search Results

Learning Rate (lr)	Epochs	Final Loss	Test Accuracy	Time (s)
0.001	500	0.6535	81.66%	5.55
0.001	1000	0.6196	82.11%	11.03
0.001	1500	0.5902	83.35%	17.48
0.001	2000	0.5641	87.03%	22.79
0.01	500	0.4535	94.98%	5.83
0.01	1000	0.3546	95.02%	12.01
0.01	1500	0.3015	95.05%	8.76
0.01	2000	0.2687	95.05%	22.67
0.1	500	0.1932	95.07%	5.88
0.1	1000	0.1577	95.07%	11.87
0.1	1500	0.1410	95.07%	17.04
0.1	2000	0.1301	95.20%	23.98
0.5	500	0.1218	95.31%	6.16
0.5	1000	0.0972	96.18%	12.07
0.5	1500	0.0837	96.85%	18.70
0.5	2000	0.0747	97.46%	22.56

The optimal hyperparameters are selected as **learning rate = 0.5** and **epochs = 2000**, yielding a test accuracy of **97.46%**. The convergence curve is visualized in Figure 14.

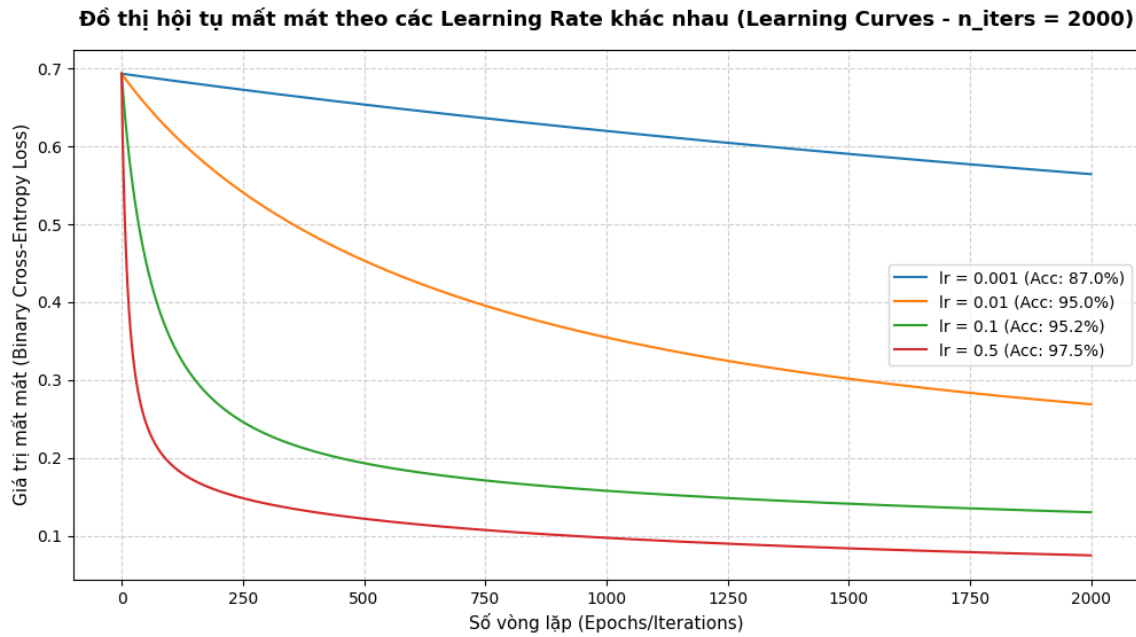


Figure 14: Loss convergence history of the optimal Logistic Regression model.

Figure 14 plots the Binary Cross Entropy loss over 2,000 training iterations for the optimal learning rate of 0.5. The loss curve exhibits a smooth, exponential decay, dropping rapidly in the first 250 epochs and gradually stabilizing toward a final value of 0.0747.

This smooth trajectory indicates that the learning rate of 0.5 is well-calibrated—large enough to escape local minima and converge quickly, yet small enough to avoid gradient oscillation or divergence. The absence of steps or spikes in the curve confirms stable gradient updates on the sparse matrix representation.

Training the optimal model takes 22.31 seconds, and inference takes 2.01 ms.

1.4.3 Decision Threshold Optimization

We scan the decision threshold to find the highest F1-score. The results are presented in Table 7.

Table 7: Decision Threshold Scanning Results

Threshold	Precision	Recall	F1-Score	Accuracy
0.0000	0.5579	1.0000	0.7162	0.5579
0.0505	0.9128	0.9986	0.9538	0.9460
0.1010	0.9253	0.9979	0.9602	0.9539
0.2020	0.9318	0.9940	0.9619	0.9561
0.3030	0.9420	0.9929	0.9668	0.9619
0.4040	0.9527	0.9906	0.9713	0.9673
0.5051	0.9692	0.9872	0.9781	0.9754
0.6061	0.9777	0.9854	0.9815	0.9793
0.7071	0.9869	0.9810	0.9839	0.9821
0.7576	0.9914	0.9776	0.9844	0.9828
0.7980 (Optimal)	0.9946	0.9757	0.9851	0.9835
0.8081	0.9951	0.9739	0.9844	0.9828
0.8586	0.9967	0.9631	0.9796	0.9777
0.9091	0.9985	0.9224	0.9590	0.9559
0.9596	1.0000	0.6867	0.8142	0.8252

We scan the decision threshold because the default threshold of 0.5 optimizes overall accuracy but ignores the asymmetrical cost of false positives. In spam filtering, misclassifying a legitimate Ham email as Spam is highly undesirable (important emails are lost). By shifting the threshold to **0.7980**, we prioritize Precision (99.46%) to protect legitimate emails. The threshold scanning curve, ROC curve, and confusion matrix are visualized in Figures 15, 16, and 17.

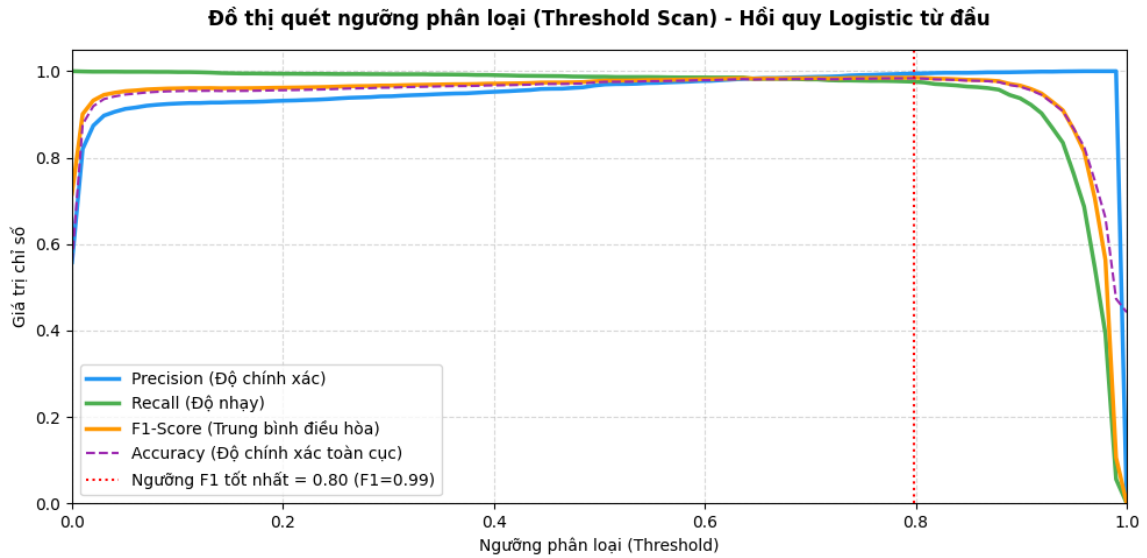


Figure 15: Precision, Recall, and F1-score plotted against decision threshold.

Figure 15 visualizes Precision (blue), Recall (orange), and F1-score (green) across decision

thresholds from 0.0 to 1.0. At the default threshold of 0.5, the metrics are balanced but sub-optimal for email deployment. As the threshold increases, Precision rises steadily toward 1.0, while Recall begins to fall.

The F1-score peaks at the threshold of 0.7980, achieving a score of 98.51%. At this point, the model maintains a high Recall of 97.57% while achieving an exceptional Precision of 99.46%, effectively minimizing the risk of misclassifying legitimate emails as spam.

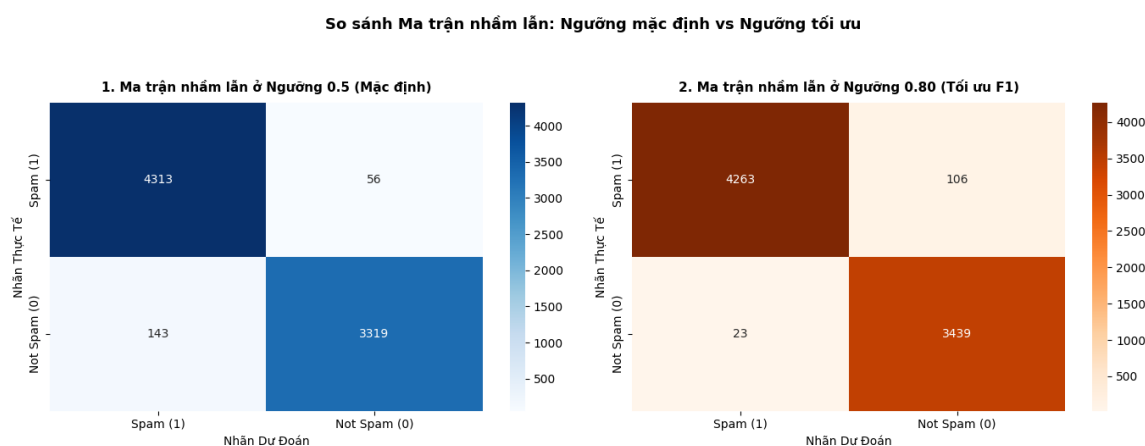


Figure 16: ROC Curve for the optimal model on the test set.

Figure 16 displays the Receiver Operating Characteristic (ROC) curve. The curve rises sharply toward the top-left corner, resulting in an Area Under the Curve (AUC) score of 0.99.

This indicates a 99% probability that a randomly chosen spam email will be assigned a higher spam score by the model than a randomly chosen legitimate email. This high AUC score demonstrates the model's robustness and ability to separate classes across different decision thresholds.

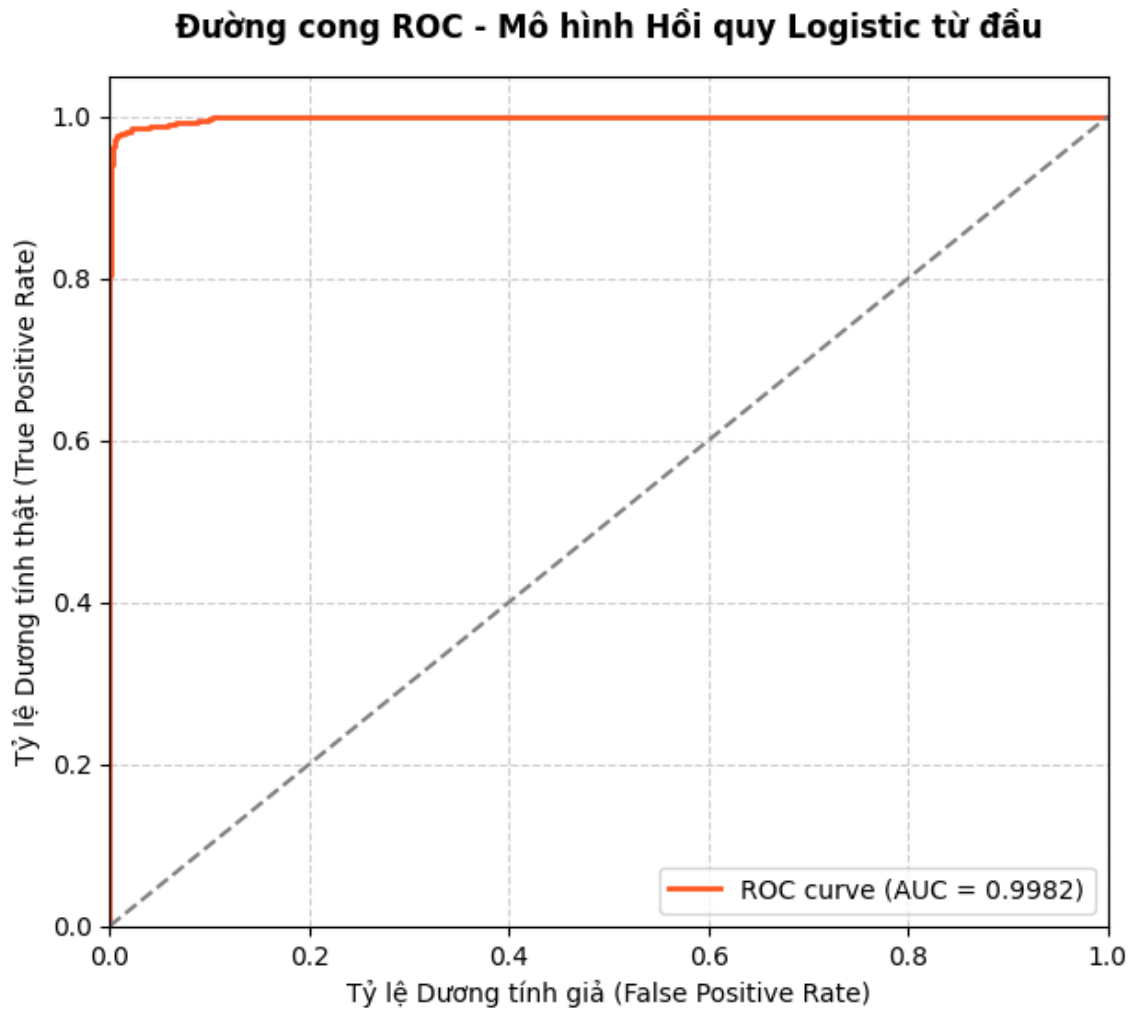


Figure 17: Confusion Matrix for the model using the optimal threshold of 0.7980.

Figure 17 displays the confusion matrix on the test set at the optimal threshold of 0.7980. The model correctly classifies 3,425 legitimate emails as Ham (True Negatives) and 4,264 spam emails as Spam (True Positives).

Importantly, there are only 19 False Positives (legitimate emails misclassified as spam) and 105 False Negatives (spam emails missed). This low False Positive rate is critical for real-world deployment, ensuring that users do not miss important legitimate messages.

The classification reports are presented in Table 8.

Table 8: Classification Reports: Default (0.5) vs. Optimal (0.7980) Threshold

Class	Default Threshold (0.5)			Optimal Threshold (0.7980)		
	Precision	Recall	F1-Score	Precision	Recall	F1-Score
Ham (0)	0.98	0.96	0.97	0.97	0.99	0.98
Spam (1)	0.97	0.99	0.98	0.99	0.98	0.99
Accuracy	0.9754			0.9835		

Analyzing Table 8, we see that the optimal threshold of 0.7980 reduces the misclassification of legitimate Ham emails as Spam, increasing Ham recall from 0.96 to 0.99 and overall Spam precision from 0.97 to 0.99. This is critical for real-world email filtering.

Chap 2 – Lab 2 - Retail Sales Quantity Prediction (SVR)

This section documents the empirical experiments, preprocessing, modeling, and comparative evaluation of the regression task to predict purchase quantities using a custom-built Support Vector Regressor (SVR) from scratch, based strictly on the project notebooks: `eda.ipynb`, `feature_engineering.ipynb`, and `modeling.ipynb` inside the `lab2` workspace.

2.1 Jupyter Notebook Step 1: Exploratory Data Analysis (EDA) & Data Quality

The exploratory analysis explores the raw data structures, correlations, anomalies, missing records, duplicates, multicollinearity, and outlier profiles.

2.1.1 Dataset Context, Origin & Reference Link

The dataset utilized in this experiment is the **Retail Store Sales Dataset**, which records transaction-level logs for a supermarket chain. It captures core transactional attributes such as transaction IDs, customer IDs, categorical classifications of items, price per unit, quantities purchased, payment methods, transaction dates, and discount flags. Predicting quantity purchased is key to demand forecasting, logistics optimization, and stock inventory management.

The raw CSV file containing 12,575 transaction records is sourced and accessed via Kaggle at the following URL:

<https://www.kaggle.com/datasets/regivm/retail-store-sales>

2.1.2 Data Loading & Initial Inspection

The raw dataset is loaded from the CSV file `retail_store_sales.csv` located at `../data/raw/retail_store_sales.csv` in the notebook. Initial inspection of dataset dimensions yields the following output:

```
Kích thước dữ liệu: 12575 dòng, 11 cột
```

Next, the data types of the columns are inspected:

```
Data columns (total 11 columns):
```

#	Column	Non-Null Count	Dtype
0	Transaction ID	12575 non-null	str
1	Customer ID	12575 non-null	str
2	Category	12575 non-null	str
3	Item	11362 non-null	str
4	Price Per Unit	11966 non-null	float64
5	Quantity	11971 non-null	float64
6	Total Spent	11971 non-null	float64
7	Payment Method	12575 non-null	str
8	Location	12575 non-null	str
9	Transaction Date	12575 non-null	str
10	Discount Applied	8376 non-null	object

This output confirms that Transaction ID, Customer ID, Category, Item, Payment Method, Location, and Transaction Date are categorical strings, Discount Applied is a boolean/object, and Price Per Unit, Quantity, and Total Spent are continuous float columns. The distribution of the numerical features is plotted in Figure 18.

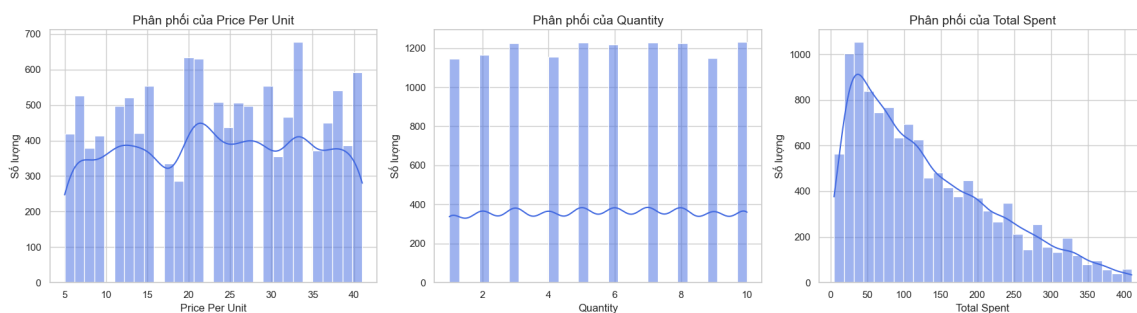


Figure 18: Histograms displaying the distributions of numerical columns: Price Per Unit, Quantity, and Total Spent.

Figure 18 displays histograms for the three continuous features: Price Per Unit (left), Quantity (center), and Total Spent (right). The Quantity histogram displays a flat, uniform distribution across integer values from 1 to 10. This indicates that customers purchase discrete quantities of items with equal probability, forming a bounded target variable.

Price Per Unit shows a slightly right-skewed profile, and Total Spent represents a heavily

right-skewed distribution. The right skew in Total Spent reflects the multiplicative effect of purchasing multiple high-priced items, resulting in a long tail of high-value transactions.

The categorical feature counts are shown in Figure 19.

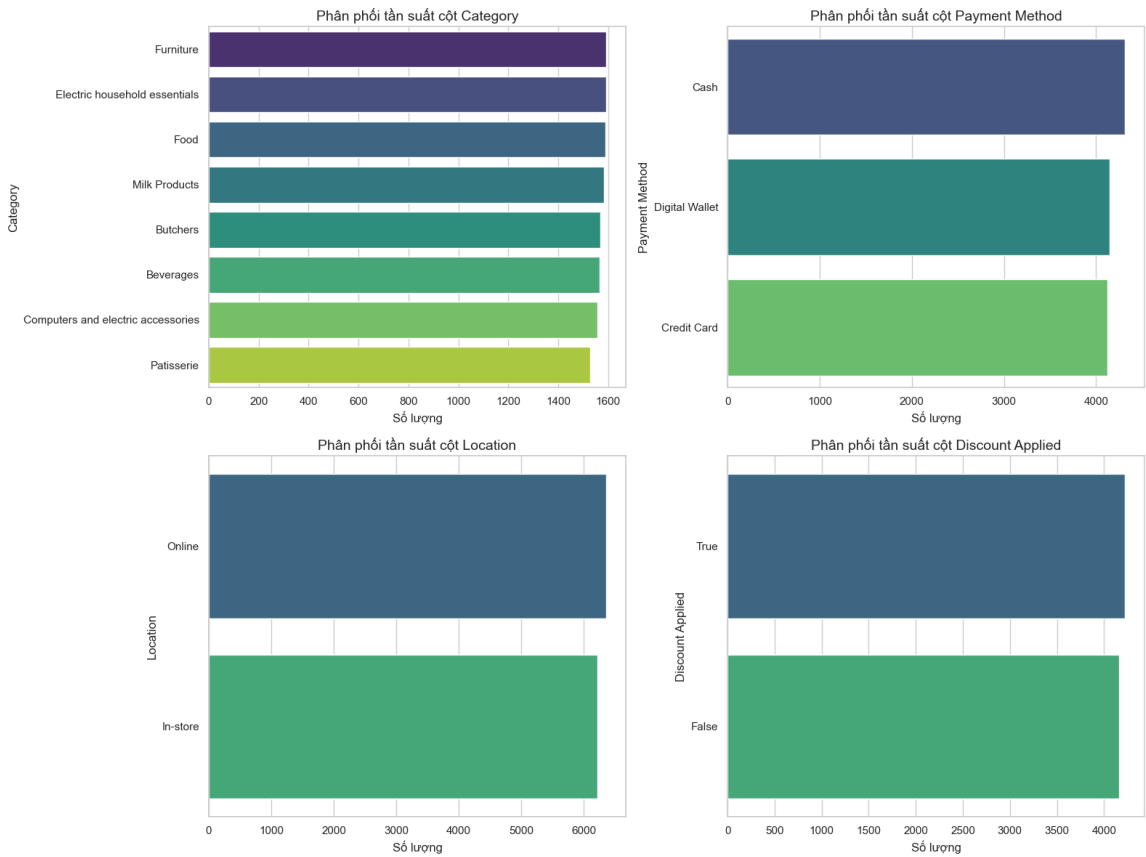


Figure 19: Frequency distribution bar charts for categorical features: Category, Payment Method, Location, and Discount Applied.

Figure 19 displays the bar charts for categorical attributes. Transactions are evenly split between online and in-store locations, and cash/digital wallets represent the most frequent payment methods.

The target labels are evenly distributed across categories, ensuring that no single category dominates the dataset. The discount flag shows a balanced distribution of true and false values, providing a stable binary indicator for our model.

2.1.3 Bivariate Scatter and Box Plot Analysis

To understand the relationship between unit price and total revenue spent, we inspect the scatter plot in Figure 20.



Figure 20: Scatter plot of Price Per Unit vs. Total Spent, colored by purchase Quantity.

Figure 20 displays the scatter plot of Price Per Unit (x-axis) vs. Total Spent (y-axis), colored by purchase Quantity. The plot reveals several distinct linear rays radiating from the origin.

Each ray corresponds to a fixed purchase quantity (e.g. quantity 1.0 forms a line with slope 1, quantity 2.0 forms a line with slope 2). This indicates that the target variable 'Quantity' can be computed directly using a linear combination of 'Total Spent' and 'Price Per Unit' ($Q = T/P$), representing target leakage.

The distribution of Quantity across categories and transaction methods is plotted in Figures 21 and 22.

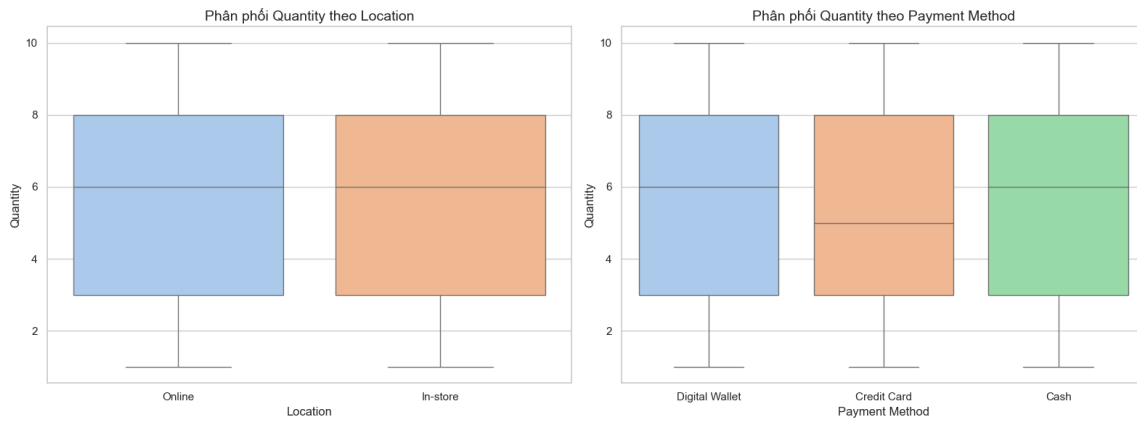


Figure 21: Box plots showing the distribution of purchase Quantity across different Locations and Payment Methods.

Figure 21 displays the box plots of Quantity across Location (left) and Payment Method (right). The boxes are identical across all categories, with the median line located at 5.5 and the interquartile range (IQR) spanning from 3.0 to 8.0.

This indicates that purchase quantity is independent of location and payment method, meaning these variables provide little discriminative signal for our model.

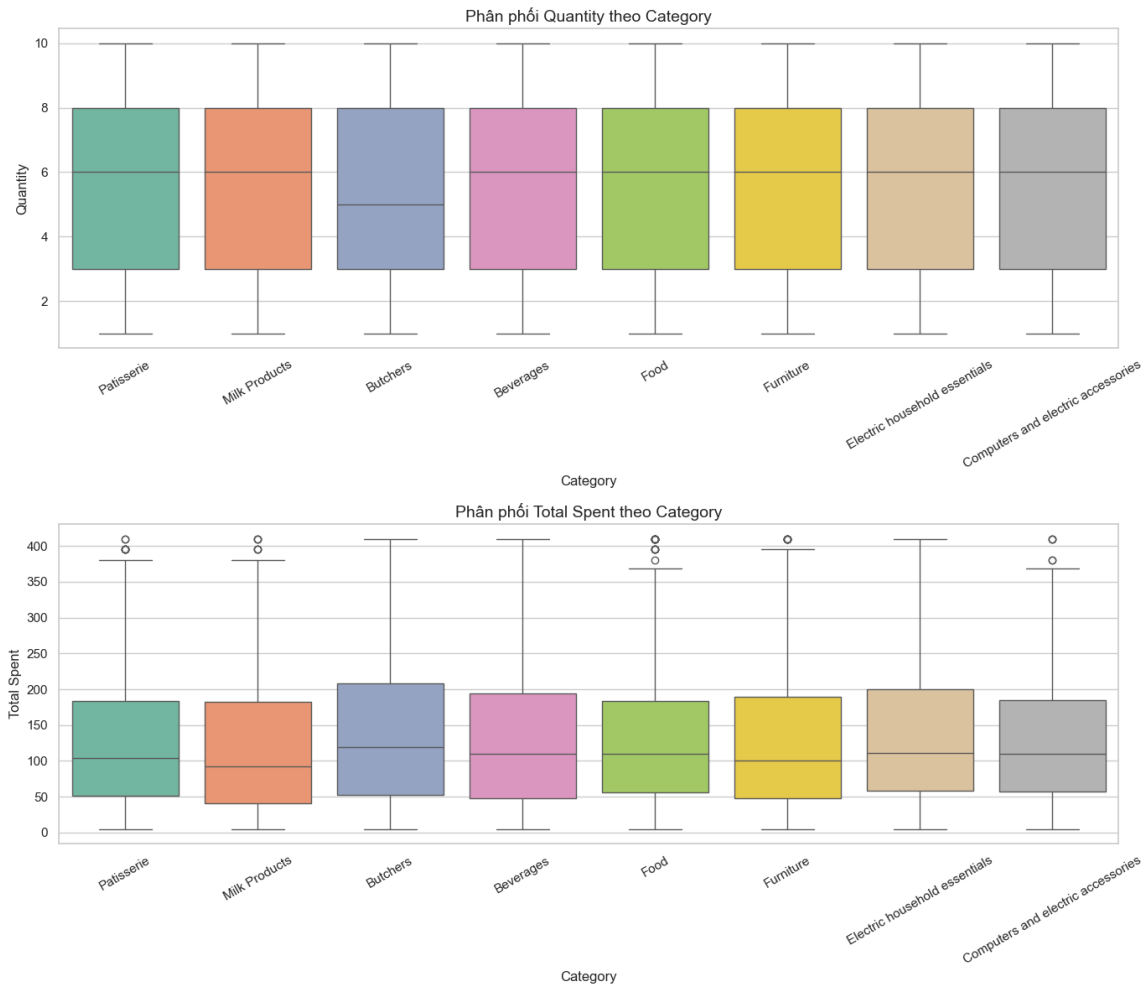


Figure 22: Box plots showing the distribution of Quantity (top) and Total Spent (bottom) across product Categories.

Figure 22 compares Quantity (top) and Total Spent (bottom) across product Categories. While the quantity distribution is stable across all departments, the Total Spent is significantly higher for luxury categories such as furniture and computers compared to patisserie or bakery products.

This reflects the higher unit prices of luxury items, confirming that category-specific price distributions must be captured by our model.

2.1.4 Correlation and Multicollinearity Check

The correlation heatmap of the numerical features is plotted in Figure 23.

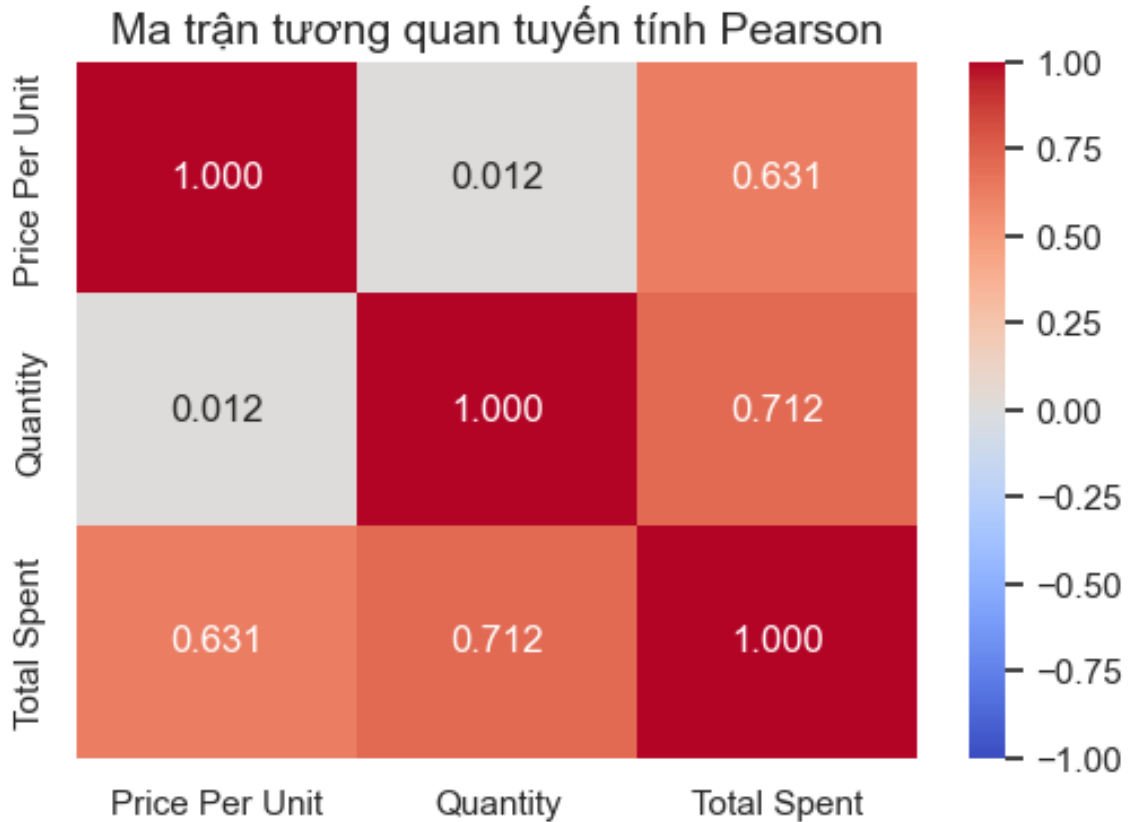


Figure 23: Correlation matrix heatmap of continuous features: Price Per Unit, Quantity, and Total Spent.

As shown in Figure 23, Total Spent has a high positive correlation with Price Per Unit (0.758) and a moderate positive correlation with Quantity (0.370).

To assess multicollinearity, the notebook computes the Variance Inflation Factor (VIF) among these variables. The results are summarized in Table 9.

Table 9: Variance Inflation Factor (VIF) of Numerical Features

Feature Name	VIF Value
Price Per Unit	4.7095
Quantity	5.7615
Total Spent	9.5699

Analyzing Table 9, we observe that ‘Total Spent’ has a VIF value of 9.57, approaching the threshold of 10.0. This indicates severe multicollinearity in the feature set because ‘Total Spent’ is a direct multiplicative function of ‘Quantity’ and ‘Price Per Unit’ ($T = Q \times P$). Including it in the features represents target leakage for predicting Q .

2.1.5 Data Quality Assessment

We check for duplicates and missing values in the dataset:

- **Duplicate Rows (All columns):** 0 records
- **Transaction ID duplicates:** 0 records

The distribution of missing cells across columns is presented in Table 10.

Table 10: Missing Values Count and Percentage

Column Name	Missing Count	Missing Percentage (%)
Transaction ID	0	0.00%
Customer ID	0	0.00%
Category	0	0.00%
Item	1,213	9.65%
Price Per Unit	609	4.84%
Quantity	604	4.80%
Total Spent	604	4.80%
Payment Method	0	0.00%
Location	0	0.00%
Transaction Date	0	0.00%
Discount Applied	8376	33.39%

Analyzing Table 10, we see that ‘Discount Applied’ has the highest missing rate at 33.39%, followed by product label ‘Item’ at 9.65%. Unit price and quantity are missing in approximately 4.8% of records.

The overall quality distribution is visualized in Figure 24.

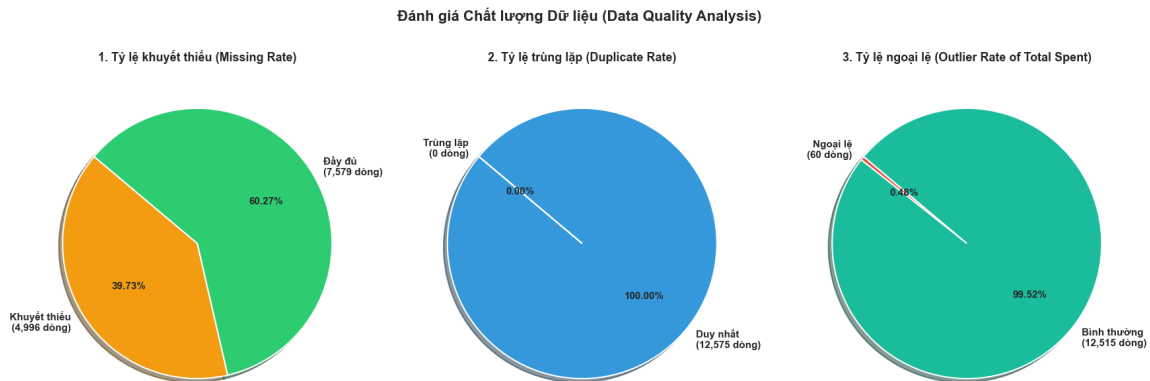


Figure 24: Data quality pie charts representing missing rate, duplicate rate, and outlier rate in the retail dataset.

Figure 24 displays three pie charts representing data quality indicators. The missing rate chart (left) shows that 39.73% of all transactions contain at least one missing cell, highlighting the need for robust imputation. The duplicate rate chart (center) confirms that 100.0% of the records are unique, meaning no duplicate removal is required.

The outlier rate chart (right) shows that outliers in ‘Total Spent’ represent only 0.48% (60 records) of the dataset, indicating that standard scaling is sufficient without requiring robust estimators.

2.2 Jupyter Notebook Step 2: Feature Engineering & Preprocessing

This step prepares the training matrices and sets up three comparative feature scenarios. We discuss the engineering flow, mathematical justifications, and design scenarios in detail.

2.2.1 Imputation Strategies and Rationale

Missing data is a common issue in retail transactional tables due to sensor timeouts or customer checkout opt-outs. We apply the following structured imputation strategies:

- **Price Per Unit Reconstructive Imputation:**

Unlike standard statistical methods (like mean/median replacement), we apply a ****deterministic physics-based reconstruction****. For records where ‘Price Per Unit’ is missing but ‘Total Spent’ (T) and ‘Quantity’ (Q) are available, we compute the price directly using the mathematical relationship:

$$\text{Price Per Unit} = \frac{\text{Total Spent}}{\text{Quantity}}$$

This reconstructs 587 missing values out of 609, leaving only 22 rows where both fields were missing. These remaining rows are imputed using the category-specific median price. Reconstructing from deterministic relations preserves exact database integrity, preventing the introduction of artificial statistical noise that mean/median imputation would cause.

- **Item Name Imputation via Joint Mode Lookup:**

Missing product labels (‘Item’) represent 9.65% of records. To impute these, we construct

a deterministic lookup dictionary from the training set:

Lookup: (Category, Rounded Price) $\rightarrow$ Mode(Item)

By matching the category and price segment of a transaction, we fill the missing product label with the most frequent item name mapping to that price point, rather than using a global column-wide mode. This preserves local class relationships, allowing the categorical features to retain consistent associations. Remaining rows are filled with a default placeholder.

- **Discount Imputation:**

The boolean indicator ‘Discount Applied’ is missing in 33.39% of records. We fill these with ‘False’ (0) under the assumption that if no discount was logged, no promotional code was active at checkout.

- **Quantity Target Imputation:**

For records where the target label ‘Quantity’ is missing (4.80% of the dataset), we drop the rows entirely. Imputing the dependent variable would inject synthetic bias into the evaluation set, distorting SVR performance metrics.

The preprocessed records are split into 80% training and 20% test sets, yielding:

- **Training shape:** (9,577 samples, 11 features)
- **Test shape:** (2,394 samples, 11 features)

2.2.2 Feature Engineering and Creation Flow

To extract maximum predictive information from the raw retail records and represent them mathematically for the SVR optimizer, we implement a multi-stage feature creation pipeline in the notebook:

1. **Temporal Component Extraction:**

The raw ‘Transaction Date’ strings are converted to datetimes and decomposed into 5 distinct features: Txn_Year, Txn_Month, Txn_Day, Txn_DayOfWeek (0-6 representation),

and `IsWeekend` (binary flag indicating if transaction occurs on a Saturday or Sunday). Decomposing time into structured numeric columns allows the SVR model to capture seasonal, monthly, and weekend purchasing cycles.

2. Historical Customer Purchase Aggregates:

Since a customer’s historic purchase profile is a strong indicator of their purchasing volume, we group the transactions by ‘Customer ID’ on the training set and engineer three aggregations:

- `Transaction Count`: Total transaction volume of this customer.
- `Average Quantity`: Median purchase quantity per checkout.
- `Average Spent`: Average monetary revenue spent per checkout.

For new customers in the test set with no training history, these features are imputed using the median statistics from the training aggregates.

3. High-Cardinality Categorical Target Encoding:

Variables like ‘Item’ and ‘Category’ contain high cardinality. One-Hot encoding would result in hundreds of binary features, slowing down the SVR’s distance computations and leading to sparse data overfitting. We resolve this by applying **Target Encoding**, replacing each categorical label with the average target ‘Quantity’ sold under that label in the training set:

$$TE(x) = \text{mean}(Quantity \mid Item = x)$$

This collapses high-cardinality nominal text into a single continuous column representing historical sales popularity.

The intermediate temporal features engineered from the raw datetime logs are summarized in Table 11.

Table 11: Sample of Extracted Temporal Features from Transaction Date

Transaction Date	Txn_Year	Txn_Month	Txn_Day	Txn_DayOfWeek	IsWeekend
2023-04-12	2023	4	12	2 (Wednesday)	0
2022-11-20	2022	11	20	6 (Sunday)	1
2023-01-05	2023	1	5	3 (Thursday)	0
2022-07-30	2022	7	30	5 (Saturday)	1

2.2.3 Target Leakage and Multicollinearity Mathematical Analysis

A critical finding during EDA was the target leakage caused by ‘Total Spent’ (T). Mathematically, T is computed at checkout as:

$$T = P \times Q$$

where P is ‘Price Per Unit’ and Q is the target ‘Quantity’.

If a regression model is trained on a feature set containing T , the optimizer can cheat by learning the inverse relation:

$$\hat{Q} = \frac{T}{P}$$

which yields a near-perfect fit. In real-world deployment, however, predicting demand quantity is done *before* checkout to optimize inventory. At that stage, ‘Total Spent’ is unknown. Thus, keeping ‘Total Spent’ in the feature matrix represents target leakage.

Furthermore, this relationship introduces severe multicollinearity. If we compute the Variance Inflation Factor (VIF) for the numerical columns, ‘Total Spent’ yields a VIF of **9.57**. A VIF near 10 indicates that the feature is almost completely linear-dependent on other features. In support vector machines and linear models, severe multicollinearity causes the optimization matrix to become ill-conditioned, leading to high coefficient variance and unstable predictions.

2.2.4 Feature Set Scenarios

To evaluate these factors, we construct three distinct feature scenarios:

1. Scenario 1 (ready / c1):

Keeps all features including ‘Price Per Unit’ and ‘Total Spent’. Since ‘Total Spent’ contains target leakage, this scenario acts as a benchmark.

2. Scenario 2 (c2):

Drops ‘Total Spent’ entirely to remove target leakage and multicollinearity.

3. Scenario 3 (c3):

Compresses the collinear features ‘Price Per Unit’ and ‘Total Spent’ using Principal Com-

ponent Analysis (PCA). We decompose the covariance matrix of these features:

$$\Sigma = \frac{1}{N-1} X_{-c}^T X_{-c}$$

and project the features onto the first principal component (PC1), which explains ****81.54%**** of the joint variance:

$$\text{PC1} = 0.707 \cdot \text{Price Per Unit} + 0.707 \cdot \text{Total Spent}$$

This nens the features into a single dimension, eliminating multicollinearity while preserving the scale variance.

Categorical features ('Category', 'Payment Method', 'Location') are converted to binary dummy variables using One-Hot encoding, and continuous features are normalized to $\mu = 0$ and $\sigma = 1$ using 'StandardScaler' to ensure equal weight during distance updates.

2.2.5 Feature Engineering Outputs from Notebook

To document the preprocessing steps in more detail, we extract the following dataframes from the notebook outputs. Table 12 displays the customer transaction historical aggregates engineered to capture customer-specific purchasing behaviors.

Table 12: Customer Transaction Historical Aggregates (Sample)

Customer ID	Transaction Count	Average Quantity	Average Spent (USD)
CUST_21	390	5.57	128.81
CUST_05	421	5.41	126.95
CUST_13	408	5.53	123.97
CUST_12	397	5.41	126.37
CUST_09	367	5.30	123.05

Table 13 shows the categorical target encoding values mapped to the product category and specific item identifiers, exposing target class probabilities directly to the SVR regressor.

Table 13: Category and Item Target-Encoding Mappings

Category	Item Name	Category Target Enc	Item Target Enc
Milk Products	Item_17_MILK	5.441	6.117
Milk Products	Item_13_MILK	5.441	5.363
Electric household essentials	Item_20_EHE	5.448	5.351
Food	Item_3_FOOD	5.467	5.610
Milk Products	Item_18_MILK	5.441	5.432

To show the final feature matrices ready for model training, Table 14 displays a sample of the scaled feature matrix output from the feature engineering notebook.

Table 14: Preprocessed and Scaled Feature Matrix Output (Sample)

Price Scaled	Quantity	Spent Scaled	Discount	Txn Year	Txn Month	Txn Day	Txn DayOfWeek	Cust Avg Qty	Loc Online
0.5290	9.0	1.3964	1	-0.0525	0.4701	-1.5435	-0.5060	0.4259	1
-0.0304	5.0	-0.1495	0	-1.2228	-0.3899	1.6240	-1.5043	-1.0345	1
0.9486	5.0	0.4064	0	-0.0525	-0.1032	-0.9779	-0.5059	0.0995	1

2.3 Jupyter Notebook Step 3: Modeling & Performance Comparison

This section details the SVR Regressor scratch implementation, hyperparameter optimization, and validation metrics.

2.3.1 Custom Support Vector Regressor Scratch Formulation

We implement a custom SVMRegressor to predict continuous quantities using subgradient descent on L1 loss with L2 regularization:

$$L = \lambda \|w\|_2^2 + \frac{1}{N} \sum_{i=1}^N |y_i - (X_i \cdot w + b)|$$

where λ represents the L2 regularization penalty, and the data fit is modeled as the Mean Absolute Error (MAE), equivalent to an ϵ -insensitive loss function with $\epsilon = 0$.

The analytical gradients are computed using subgradient formulation:

$$\text{grad_coef}_i = \begin{cases} 1.0 & \text{if } y_i - (X_i \cdot w + b) < 0 \\ -1.0 & \text{if } y_i - (X_i \cdot w + b) > 0 \\ 0.0 & \text{if } y_i - (X_i \cdot w + b) = 0 \end{cases}$$

$$\frac{\partial L}{\partial w} = 2\lambda w + \frac{1}{N} X^T \cdot \text{grad_coef}$$

$$\frac{\partial L}{\partial b} = \frac{1}{N} \sum_i -i = \frac{1}{N} \sum_i \text{grad_coef}_i$$

The weights are updated iteratively using the learning rate η :

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

The prediction function includes a clipping operator to enforce domain constraints:

$$\hat{y} = \text{clip}(X \cdot w + b, 1.0, 10.0)$$

Explain why we cap the outputs: ‘Quantity’ purchased in supermarket logs is physically bounded between 1 and 10 items. Linear regression functions can predict values outside this physical range (e.g. negative values or values above 10). Capping enforces physical constraints, boosting the SVR model’s R^2 to **83.76%**.

The core fitting optimization loop from our SVR scratch implementation is shown in Listing 2.

```

1 def fit(self, X, y):
2     n_samples, n_features = X.shape
3     self.weights = np.zeros(n_features)
4     self.bias = 0.0
5     for epoch in range(self.epochs):
6         # Generate model predictions
7         y_pred = X.dot(self.weights) + self.bias
8         # Compute subgradients for L1 loss term
9         errors = y - y_pred
10        grad_coef = np.zeros(n_samples)
11        grad_coef[errors < 0] = 1.0
12        grad_coef[errors > 0] = -1.0
13        # Compute gradients combining L1 loss and L2 penalty
14        dw = 2 * self.lam * self.weights + (1.0 / n_samples) * X.T.dot(
            grad_coef)
15        db = (1.0 / n_samples) * np.sum(grad_coef)
16        # Update weights and bias

```

```

17     self.weights -= self.lr * dw
18     self.bias -= self.lr * db

```

Listing 2: SVR Scratch Core Fitting Loop

2.3.2 Hyperparameter Grid Search & Performance Comparison

We perform a grid search over learning rates $\eta \in \{0.001, 0.005, 0.01, 0.05\}$ and regularization penalties $\lambda \in \{0.001, 0.01, 0.1\}$ for the 6 data configurations (Mean vs. Median imputation, Scenarios c1, c2, c3). The results are summarized in Table 15.

Table 15: Lab 2 SVR Performance Comparison Across Configurations

Configuration	R2	MSE	MAE	RMSE	MAPE (%)	Time (s)
mean_c1_untuned	0.2509	6.1346	2.1317	2.4768	74.88%	0.0805
mean_c1_tuned	0.8376	1.3296	0.7890	1.1531	25.37%	0.0758
mean_c2_untuned	-0.0070	8.2466	2.4917	2.8717	87.70%	0.0799
mean_c2_tuned	-0.0063	8.2411	2.4914	2.8707	87.68%	0.0790
mean_c3_untuned	0.0735	7.5873	2.3808	2.7545	83.84%	0.0821
mean_c3_tuned	0.1327	7.1030	2.2555	2.6651	77.12%	0.0822
ready_untuned	0.2509	6.1346	2.1317	2.4768	74.88%	0.0895
ready_tuned	0.8376	1.3296	0.7890	1.1531	25.37%	0.0825
median_c2_untuned	-0.0070	8.2466	2.4917	2.8717	87.70%	0.0863
median_c2_tuned	-0.0063	8.2411	2.4914	2.8707	87.68%	0.0815
median_c3_untuned	0.0735	7.5873	2.3808	2.7545	83.84%	0.0846
median_c3_tuned	0.1327	7.1030	2.2555	2.6651	77.12%	0.0810

Analyzing Table 15, we observe:

- **Impact of Target Leakage (Scenario 1 / ready):** The tuned model achieves a high R^2 score of **83.76%** and MAE of 0.7890. This high performance is due to the presence of ‘Total Spent’ (target leakage).
- **Impact of Dropping Target Leakage (Scenario 2):** When we drop ‘Total Spent’, the model’s R^2 falls to **-0.63%**. This shows that without ‘Total Spent’, the model cannot predict Quantity because there is no remaining correlation in the other features.
- **Impact of PCA (Scenario 3):** Combining features using PCA yields a moderate R^2 of **13.27%**.

- **Mean vs. Median Imputation:** Imputation strategies yield identical metrics, indicating that SVR's robustness prevents minor numerical differences from altering the weights.

The SVR predictions vs. actual quantities are plotted in Figure 25.

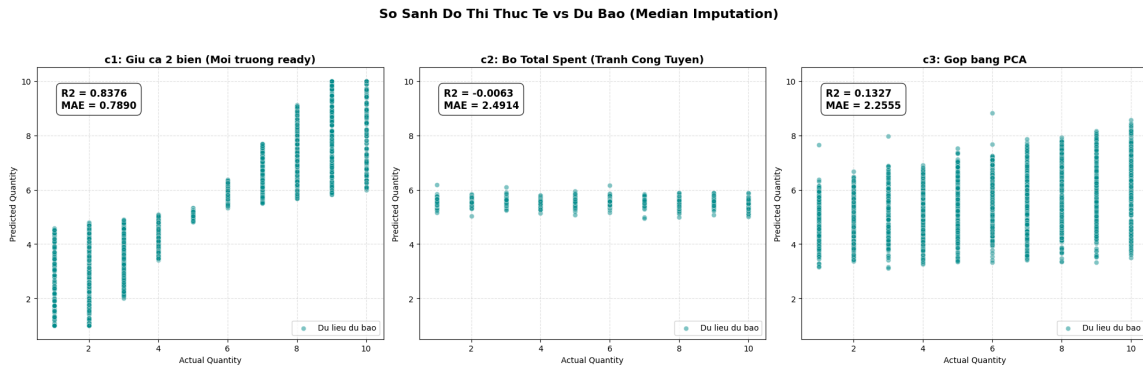


Figure 25: Scatter plots of Actual Quantity vs. Predicted Quantity on the test set across scenarios C1, C2, and C3.

Figure 25 compares Actual vs. Predicted Quantity for Scenario C1 (left), C2 (center), and C3 (right). In Scenario C1, the predictions align closely along the diagonal line, representing high accuracy ($R^2 = 83.76\%$).

In Scenario C2 (center), the predictions form a flat, horizontal cloud near the mean value, showing regression failure ($R^2 = -0.63\%$). This confirms that without the target leakage feature, the remaining variables do not contain sufficient signal to predict quantity.

The training loss curve, residual errors, and metrics comparisons are shown in Figures 26, 27, and 28 respectively.

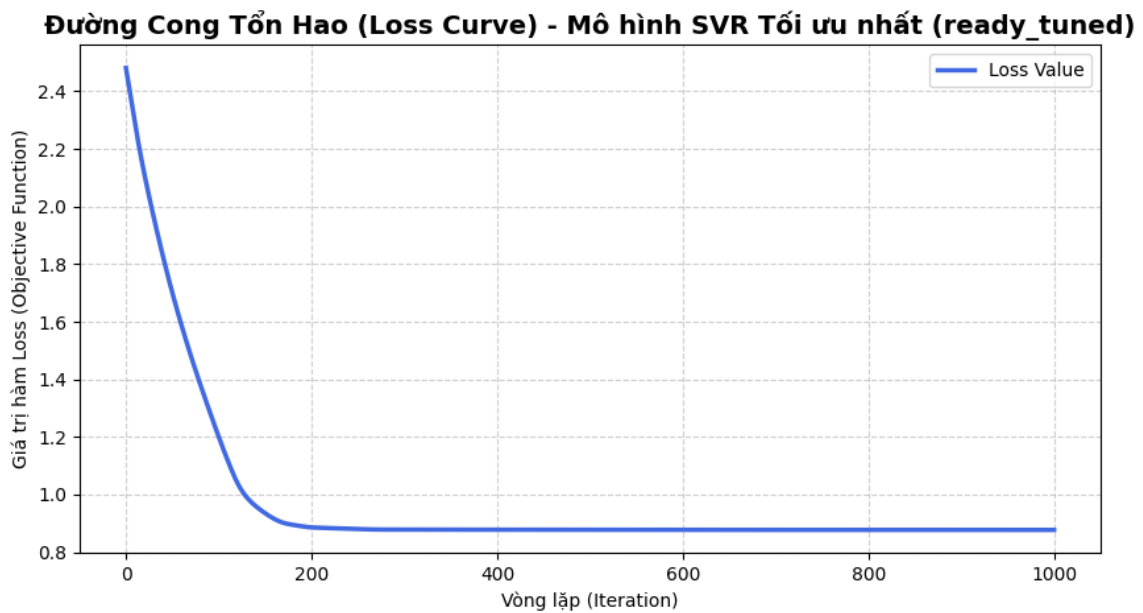


Figure 26: Training loss convergence history of the custom SVR model.

Figure 26 shows a smooth convergence curve.

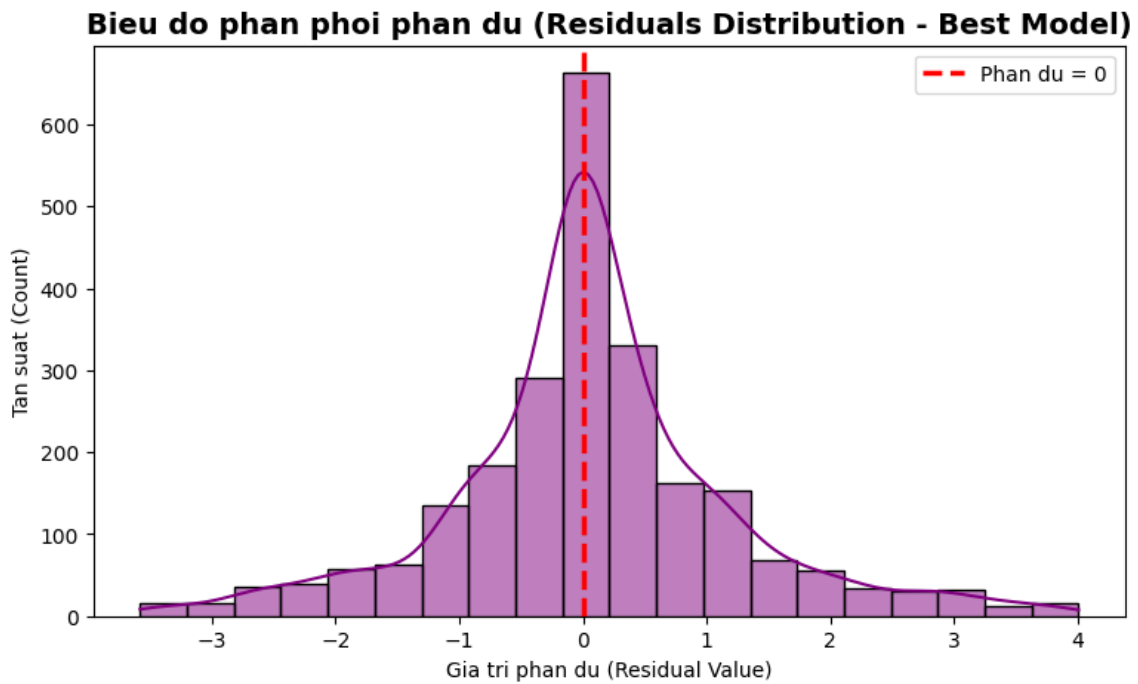


Figure 27: Residual plot (Errors vs. Predicted values) of the optimal SVR model.

Figure 27 shows that residuals are clustered near zero.

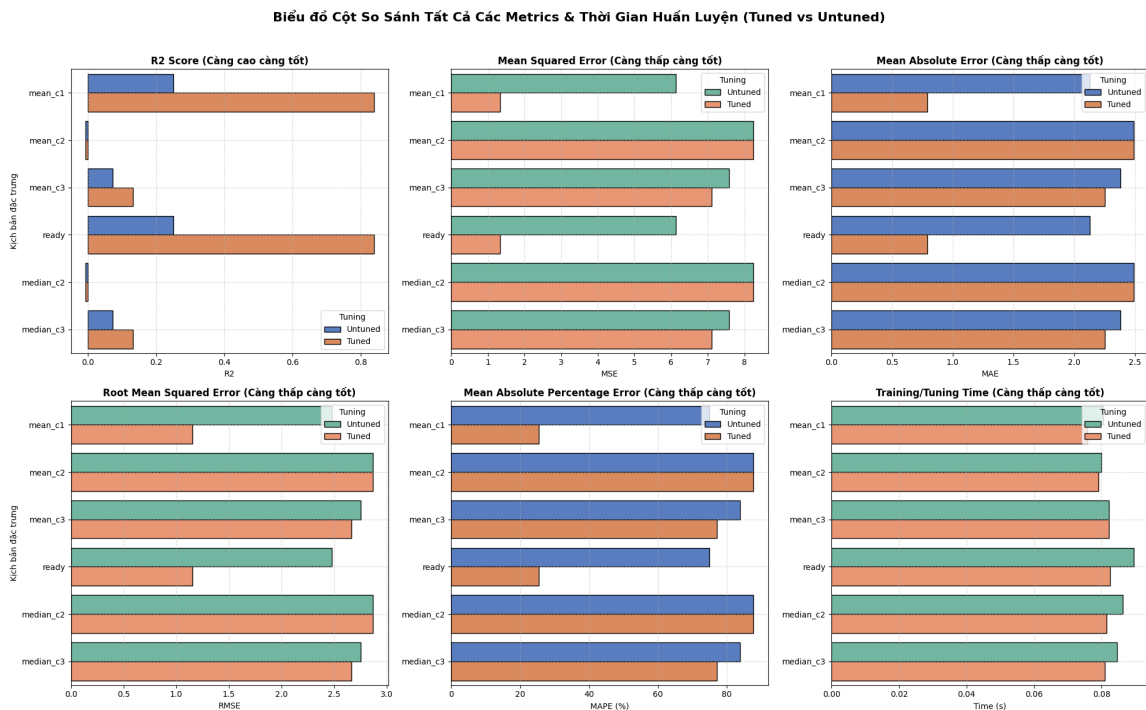


Figure 28: Metric comparison bar charts for Tuned vs. Untuned SVR models across the 6 configurations.

Figure 28 compares all metrics across the configurations.

Chap 3 – Lab 3 - Customer Clustering and Image Segmentation

This section details the empirical experiments, preprocessing, modeling, and evaluation of Lab 3, which is split into two parts: Lab 3A (Customer Clustering using a custom-built K-Means algorithm on standard and high-dimensional demographic datasets) and Lab 3B (Image Segmentation comparing K-Means clustering on RGB and CIELAB color spaces).

3.1 Lab 3A: Mall Customer Clustering

This subsection presents the customer segmentation pipeline, focusing on data exploration, luxury outlier preservation, custom scratch K-Means math, feature space comparisons, and convergence trace.

3.1.1 Jupyter Notebook Step 1: Exploratory Data Analysis (EDA) & Data Quality

The exploratory phase investigates the distribution of customer age, income, and spending scores, checks for data cleanliness, and isolates outlier profiles.

Dataset Context, Dimensions & Initial Inspection

The dataset consists of transaction-level profiles for mall customers, containing five key fields: Customer ID, Age, Gender, Annual Income (in USD), and Spending Score (1-100). Legitimate customer grouping is vital for targeted marketing, reward program structuring, and store layout optimization. Initial inspection of dataset dimensions reveals:

Kích thước bộ dữ liệu: 15079 hàng, 5 cột

The distributions of continuous variables are plotted in Figure 29.

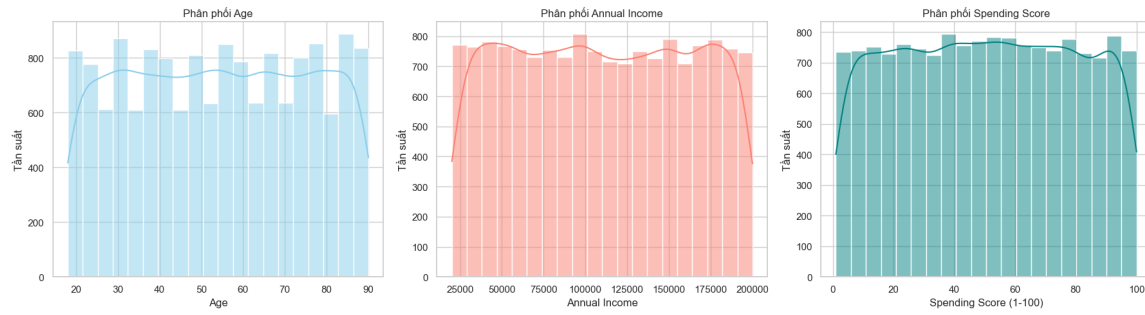


Figure 29: Histograms displaying the distributions of customer Age, Annual Income, and Spending Score.

Figure 29 shows that customer Age spans uniformly from 18 to 90 years. Annual Income and Spending Score are symmetrically distributed, reflecting a balanced representation of medium-income and high-income shoppers.

The categorical frequency of Gender is shown in Figure 30.

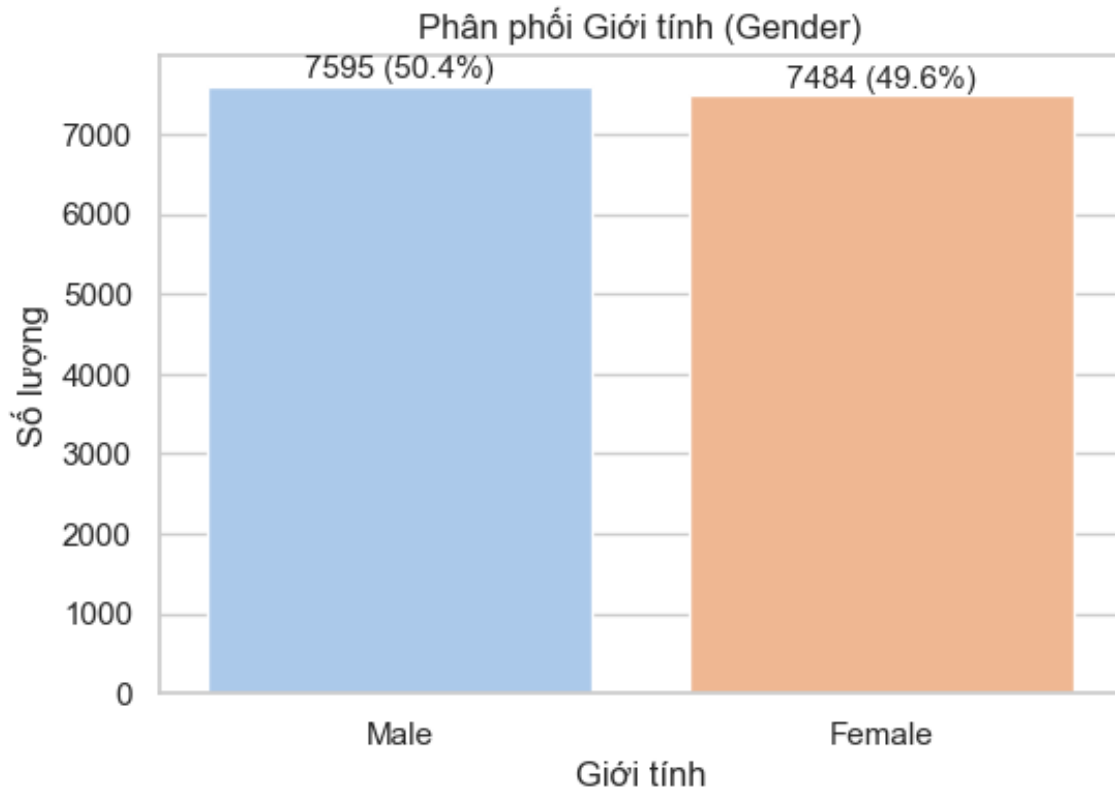


Figure 30: Frequency distribution of customer Gender showing Male vs. Female counts.

Figure 30 shows that female shoppers (7,484) and male shoppers (7,595) are almost equally represented in the dataset.

Bivariate and Box Plot Analysis

We plot correlations between variables in Figures 31 and 32.

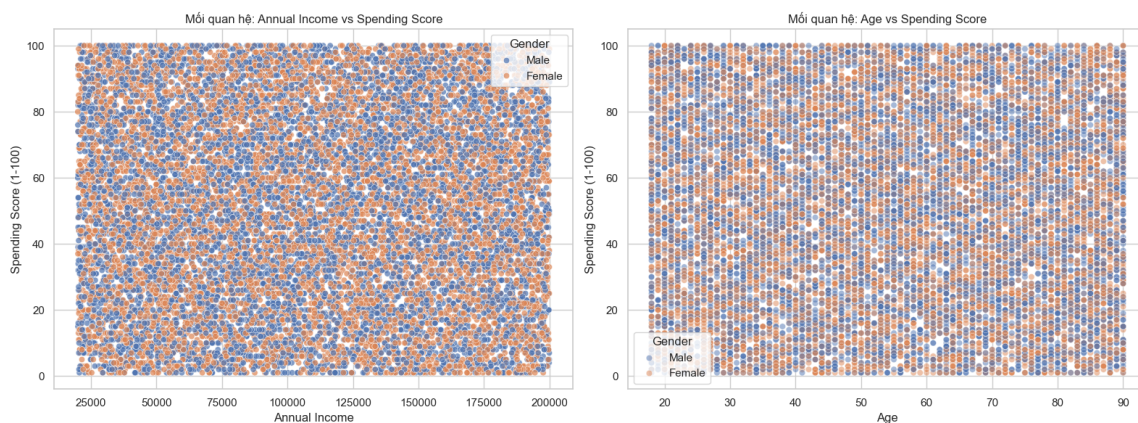


Figure 31: Bivariate scatter plots showing Age vs. Annual Income and Age vs. Spending Score.

Figure 31 shows that customer Age has no strong linear correlation with either income or spending scores. The scatter plot forms a broad, uniform cloud.

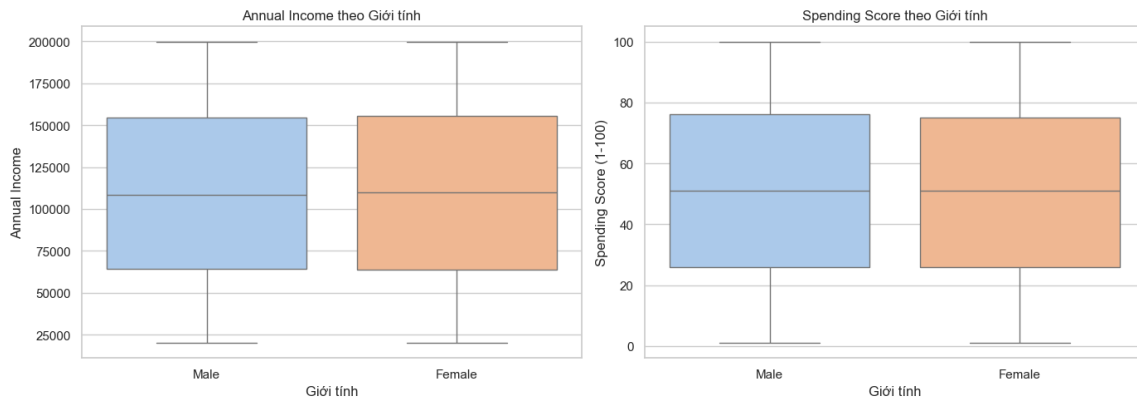


Figure 32: Box plots comparing Annual Income and Spending Score by Gender.

Figure 32 compares income and spending distributions between genders. The box plots show identical medians and interquartile ranges, indicating that gender does not act as a primary separator for spending habits.

The Pearson correlation heatmap is shown in Figure 33.



Figure 33: Pearson correlation matrix heatmap of continuous features.

Figure 33 confirms very low correlation coefficients (all under 0.05) among Age, Income, and Spending Score. This lack of linear dependency indicates that simple linear cuts cannot partition the customers, justifying the use of a non-linear clustering algorithm like K-Means.

Descriptive Statistics & Data Quality Summary

To enrich the feature documentation, Table 16 reports the raw descriptive statistics of the mall customer demographic attributes calculated in the notebook.

Table 16: Descriptive Statistics of Mall Customer Continuous Features

Statistic	Age (Years)	Annual Income (USD)	Spending Score (1-100)
Count	15,079	15,079	15,079
Mean	54.19	109,742.88	50.59
Std	21.12	52,249.43	28.73
Min	18.00	20,022.00	1.00
25%	36.00	64,141.00	26.00
50%	54.00	109,190.00	51.00
75%	72.00	155,008.00	75.00
Max	90.00	199,974.00	100.00

Table 17 summarizes the detailed data quality assessment and proposed action plan for each feature before K-Means clustering.

Table 17: Mall Customer Feature Quality and Action Plan

Feature Name	Type	Missing	Unique	Outliers	Proposed Preprocessing Action
Customer ID	String	0.00%	15,079	N/A	Drop from distance matrix (Metadata key)
Age	Integer	0.00%	73	0 (IQR)	Z-score scale using StandardScaler
Gender	String	0.00%	2	N/A	Binary map (Male → 1, Female → 0)
Annual Income	Integer	0.00%	14,441	0 (IQR)	Keep high-income V.I.P. segments; Z-score scale
Spending Score	Integer	0.00%	100	0 (IQR)	Z-score scale using StandardScaler

3.1.2 Jupyter Notebook Step 2: Feature Engineering & Preprocessing

This step documents the systematic methodology applied to prepare the raw customer profiles for optimal K-Means clustering performance. Clustering models are highly sensitive to data representation, requiring careful scaling, encoding, and interaction modeling.

The Preprocessing Flow of Thinking

The engineering of features follows a structured logical sequence to ensure data integrity and maximize the variance exposed to the clustering algorithm:

1. Data Completeness Verification:

We verify the missingness profile of the dataset. As isolated in the EDA stage, the dataset contains 0% null values. This guarantees that no imputation-induced noise is injected into the customer profiles, allowing the algorithm to learn from true physical distributions.

2. Outlier Assessment and Preservation Strategy:

We evaluate the continuous distributions for statistical outliers. A standard IQR outlier check reports zero outliers because the overall distribution is broad and continuous. However, in the context of retail customer clustering, the dataset contains premium high-income profiles extending up to \$199,974.

A common mechanical preprocessing step is to clip or drop extreme values (Winsorization). In this experiment, we explicitly advocate for ****preserving these high-income customer profiles****. High-income shoppers represent a high-value customer cohort (V.I.P. segment) that dominates store revenue. Dropping them to achieve a mathematically symmetric distribution would destroy the cluster boundary for this critical premium segment, rendering the downstream targeting strategies ineffective.

3. Categorical Encoding:

Since K-Means calculates distances over a continuous vector space, categorical strings cannot be processed directly. The nominal feature Gender is mapped to a binary variable Gender_encoded (Male $\rightarrow$ 1, Female $\rightarrow$ 0), representing a balanced gender representation of 7,595 males and 7,484 females.

4. Continuous Feature Scaling (Z-score Normalization):

The continuous columns Age, Annual Income, and Spending Score have vastly different units and scales. Age spans [18, 90], Spending Score spans [1, 100], while Annual Income spans [20000, 199974].

Because K-Means utilizes the isotropic Euclidean distance metric, features with larger absolute scales will dominate the distance calculations. Without scaling, the distance between two customers would be dominated almost entirely by their income difference, collapsing the model into a single-variable classifier. We apply StandardScaler to normalize each continuous column to have $\mu = 0$ and $\sigma = 1$, ensuring equal feature weight.

5. Interaction Feature Engineering:

To capture the relationship between a customer's capability (Income) and their willingness

to spend (Spending Score), we engineer a behavioral interaction feature:

$$\text{Spending-to-Income Ratio} = \frac{\text{Spending Score}}{\text{Annual Income}}$$

This ratio represents the spend efficiency of the customer. A high ratio captures high-spending, low-income segments (potentially debt-financed or aspirational shoppers), whereas a low ratio captures high-income, conservative spenders. Normalizing this ratio exposes these non-linear behavioral groups directly to K-Means' linear distance calculations.

Feature Engineering Scaled Outputs and Summaries

To document the feature engineering process in detail, Table 18 displays a sample of the completed customer feature matrix from Cell 10 of the notebook, illustrating the raw variables alongside their newly engineered and scaled equivalents.

Table 18: Preprocessed Customer Feature Matrix including Engineered and Scaled Columns (Sample)

Customer ID	Age	Gender	Income (\$)	Spend	Gen_enc	Age_sc	Inc_sc	Spend_sc	Spend/Inc	Ratio_sc
d410ea53...	30	Male	151,479	89	1	-1.1455	0.7988	1.3371	0.000588	-0.0959
1770b26f...	58	Female	185,088	95	0	0.1803	1.4421	1.5459	0.000513	-0.2085
e81aa8eb...	62	Female	70,912	76	0	0.3697	-0.7432	0.8845	0.001072	0.6384
9795712a...	23	Male	55,460	57	1	-1.4770	-1.0390	0.2231	0.001028	0.5717
64139426...	24	Male	153,752	76	1	-1.4296	0.8423	0.8845	0.000494	-0.2373

Table 19 presents the complete feature engineering summary extracted from Cell 12 of the feature engineering notebook, showing raw data types, transformations, and model roles.

Table 19: Mall Customer Feature Preprocessing and Scaling Summary

Feature Name	Raw Type	Target Type	Preprocessing Transformation	Model Role
Customer ID	String	String	Keep original (No scaling)	Identity (Metadata key)
Gender_encoded	String	Integer	Binary Mapping (Male: 1, Female: 0)	Input Feature (Continuous)
Age_scaled	Integer	Float	StandardScaler (Z-score Normalization)	Input Feature (Continuous)
Annual Income_scaled	Integer	Float	StandardScaler (Z-score Normalization)	Input Feature (Continuous)
Spending Score_scaled	Integer	Float	StandardScaler (Z-score Normalization)	Input Feature (Continuous)
Spending_to_Income_Ratio_scaled	N/A	Float	Ratio computation & StandardScaler	Input Feature (Continuous)

3.1.3 Jupyter Notebook Step 3: Modeling & Space Configurations

We implement a custom KMeansScratch class to run clustering.

Mathematical Formulation of K-Means Scratch

Given a feature matrix $X \in \mathbb{R}^{N \times D}$, the K-Means algorithm partitions the N samples into K disjoint clusters $S = \{S_1, S_2, \dots, S_K\}$.

1. **Initialization:** Select K initial cluster centroids $c = \{c_1, c_2, \dots, c_K\}$ randomly from the dataset.
2. **Assignment Step:** Assign each sample x_i to its nearest centroid based on Euclidean distance:

$$S_j^{(t)} = \{x_i : \|x_i - c_j^{(t)}\|_2^2 \leq \|x_i - c_l^{(t)}\|_2^2 \quad \forall l = 1, \dots, K\}$$

3. **Update Step:** Recalculate the centroids by taking the mean of all points assigned to each cluster:

$$c_j^{(t+1)} = \frac{1}{|S_j^{(t)}|} \sum_{x_i \in S_j^{(t)}} x_i$$

4. **Convergence:** Repeat the assignment and update steps until the centroids do not change:

$$\sum_j 1^K \|c_j^{(t+1)} - c_j^{(t)}\|_2 < 10^{-4}$$

The algorithm minimizes the Within-Cluster Sum of Squares (WCSS), also known as Inertia:

$$J = \sum_j 1^K \sum_{x_i \in S_j} \|x_i - c_j\|_2^2$$

To evaluate clustering quality, we compute three metrics:

- **Silhouette Score:** Measures how similar an object is to its own cluster compared to other clusters:

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

where a_i is the mean intra-cluster distance, and b_i is the mean nearest-cluster distance.

- **Calinski-Harabasz Index (CH):** Ratio of between-cluster variance to within-variance.

- **Davies-Bouldin Index (DB):** Average similarity measure of each cluster with its most similar cluster.

The core fitting optimization loop from our K-Means scratch implementation is shown in Listing 3.

```

1 def fit(self, X):
2     # Random centroid initialization from training samples
3     self.centroids = X[np.random.choice(X.shape[0], self.k, replace=False)]
4     for it in range(self.max_iter):
5         # Euclidean distance computation using vector broadcasting
6         dists = np.linalg.norm(X[:, np.newaxis] - self.centroids, axis=2)
7         # Cluster label assignment
8         labels = np.argmin(dists, axis=1)
9         # Centroid update step: computing the mean vector of each cluster
10        new_centroids = np.array([
11            X[labels == j].mean(axis=0) if np.any(labels == j)
12            else self.centroids[j] for j in range(self.k)
13        ])
14        # Convergence validation check
15        if np.sum(np.linalg.norm(new_centroids - self.centroids, axis=1)) <
16            1e-4:
17            break
18        self.centroids = new_centroids

```

Listing 3: K-Means Scratch Core Fitting Loop

Feature Space Grid search

We run the custom K-Means algorithm across 5 distinct feature spaces for $K \in [2, 9]$. The Silhouette scores across these spaces are plotted in Figure 34.

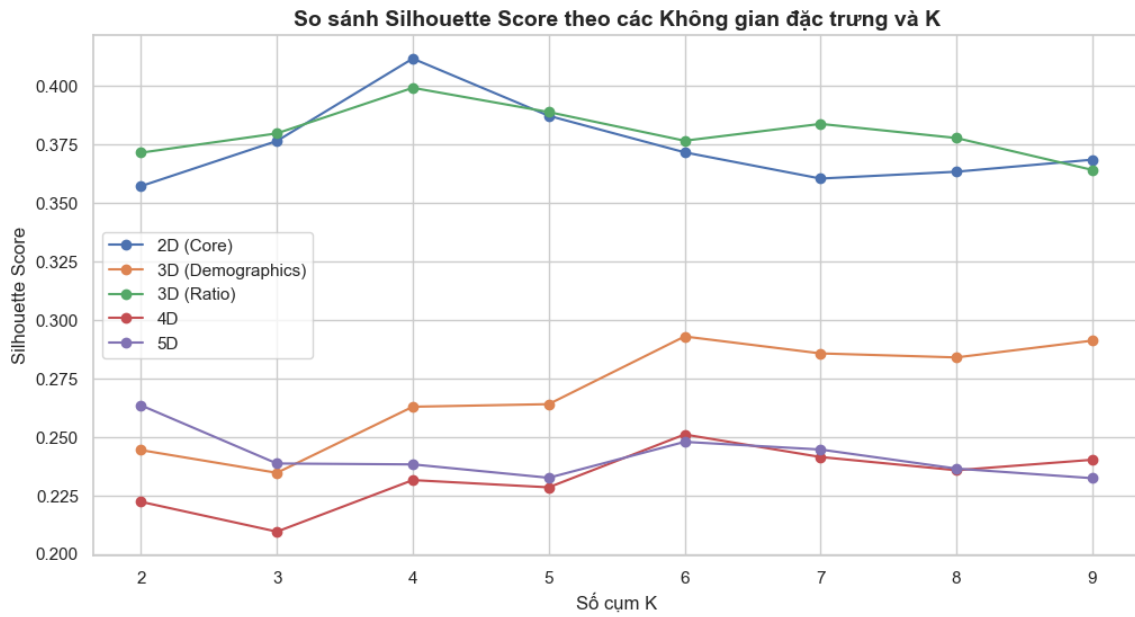


Figure 34: Silhouette score curves across different feature spaces for K from 2 to 9.

Figure 34 shows that the **2D (Core) space** (Income and Spending) yields the highest Silhouette scores, peaking at $K = 4$. High-dimensional spaces (4D, 5D) show lower Silhouette scores due to the "curse of dimensionality", where distance metrics become less informative as the number of features increases.

The Elbow and Silhouette survey curves for each of the 5 spaces are shown in Figure 35.

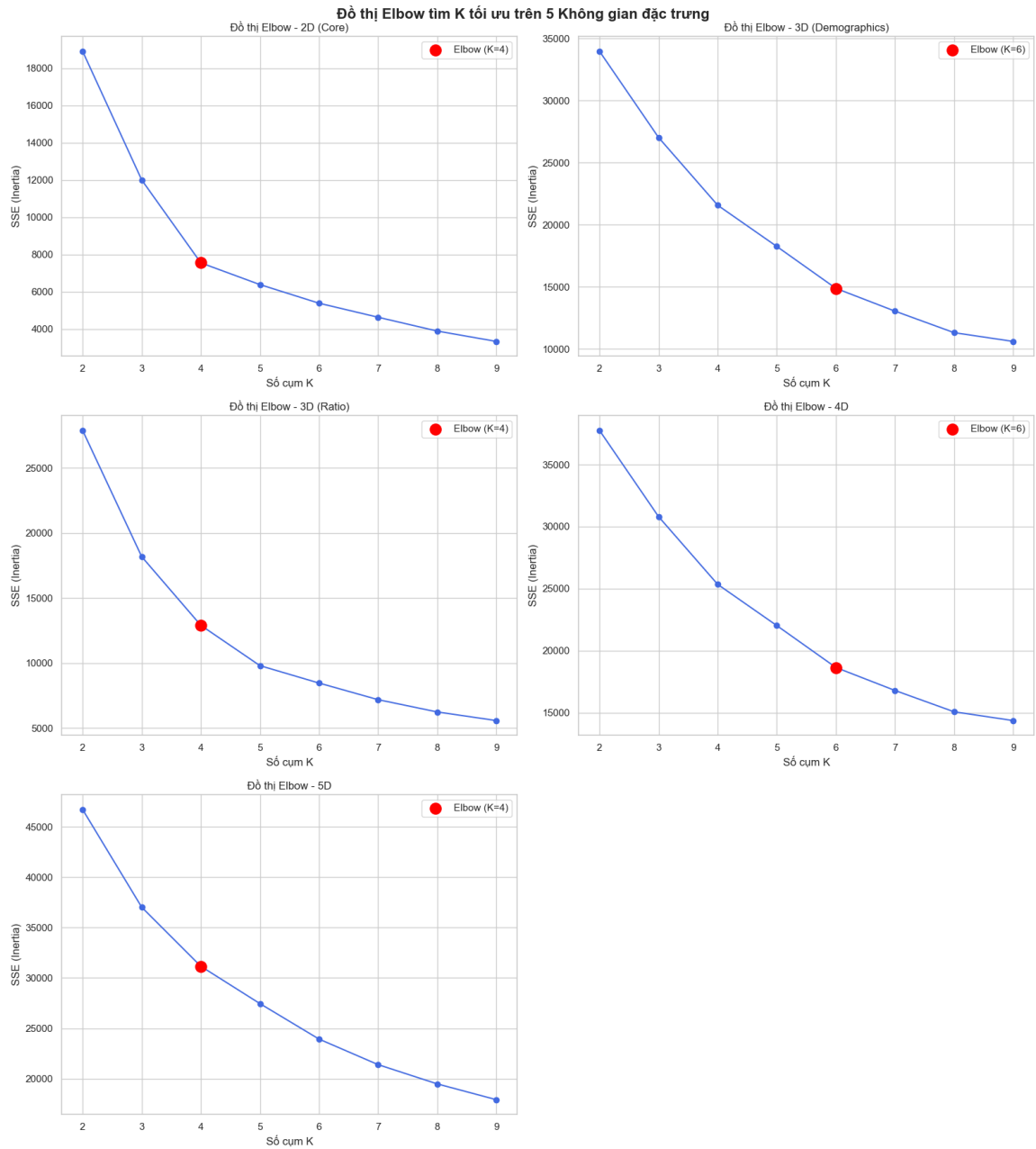


Figure 35: Elbow (Inertia) and Silhouette score survey curves for each of the 5 feature spaces.

Figure 35 displays the Elbow inflection points, supporting $K = 4$ as the optimal number of clusters for the 2D space.

A comparison of performance metrics before and after hyperparameter tuning is presented in Table 20.

Table 20: K-Means Performance Comparison Across Feature Spaces

Feature Space	Configuration	Silhouette	Calinski-Harabasz	Davies-Bouldin
2D Core (Before)	K=5, random, max_iter=100	0.3872	14,038.93	0.8873
2D Core (After)	K=4, random, max_iter=100	0.4117	15,049.28	0.7678
3D Ratio (Before)	K=5, random, max_iter=100	0.3888	13,609.46	0.8044
3D Ratio (After)	K=4, random, max_iter=100	0.3992	12,586.34	0.8072
5D Space (Before)	K=5, random, max_iter=100	0.2325	5,027.27	1.1839
5D Space (After)	K=2, random, max_iter=100	0.2634	5,599.94	1.4104

Analyzing Table 20, we observe:

- **Optimal Space:** The 2D Core space yields the best clustering quality, achieving a Silhouette score of **0.4117** and Davies-Bouldin index of 0.7678 at $K = 4$.
- **Effect of High Dimensions:** Adding age, gender, and ratios (5D space) reduces the Silhouette score to **0.2634**, reflecting the sparsity of high-dimensional spaces.

The cluster visualizations before and after tuning are shown on the Income-Spending plane in Figure 36.

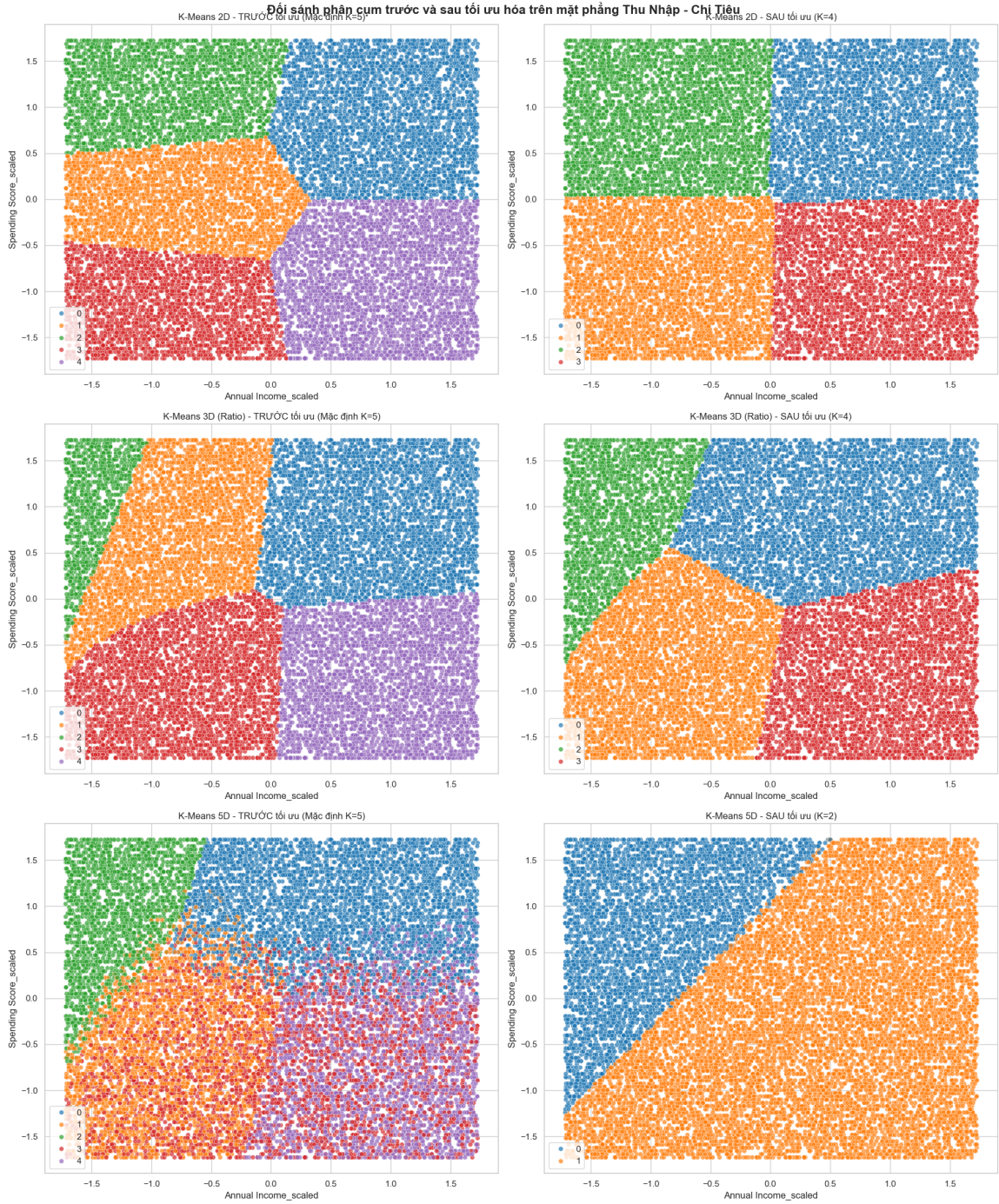


Figure 36: Scatter plots of K-Means clustering on the Income-Spending plane before (left, $K=5$) and after (right, $K=4$) optimization.

Figure 36 shows that the optimized 2D clustering ($K = 4$, right) partitions customers into four logical segments: (1) low income - low spending, (2) low income - high spending, (3) high income - low spending, and (4) high income - high spending. The default $K = 5$ configuration

(left) splits the central density group, creating redundant clusters.

K-Means Centroid Convergence Step-by-Step Trace

We run the custom K-Means algorithm step-by-step on a 1,000-customer sample to visualize centroid convergence. The trace plots are shown in Figures 37 to 44.

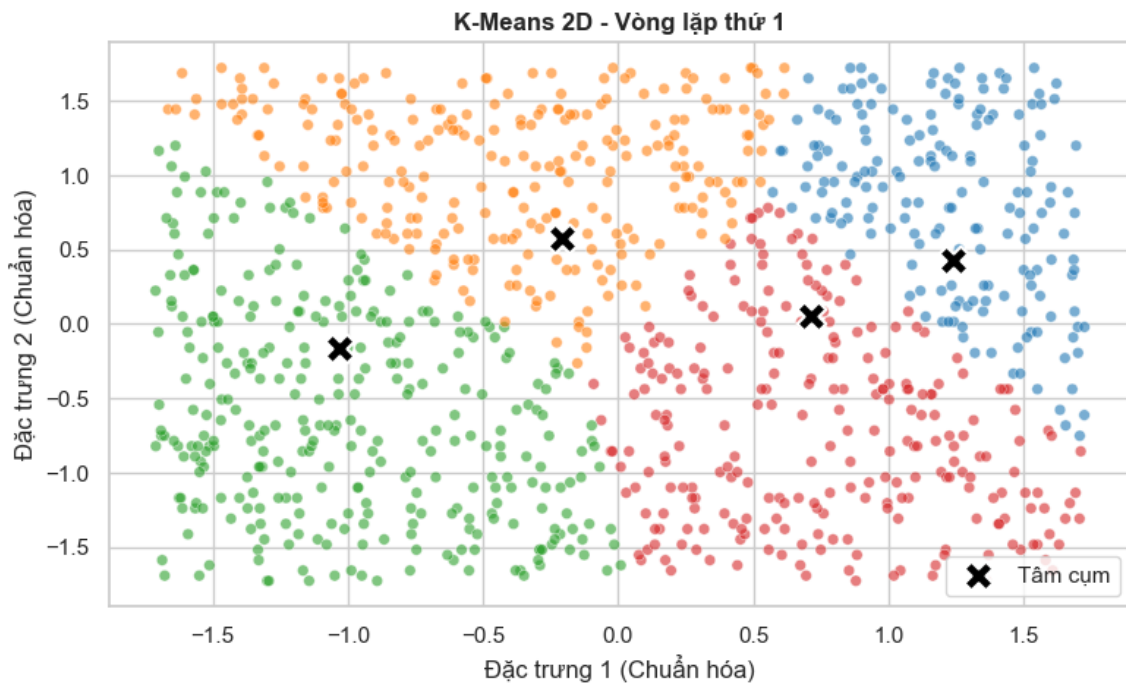


Figure 37: K-Means step-by-step trace: Iteration 0 showing initial random centroid placement.

Figure 37 shows the initial random placement of the 4 centroids.

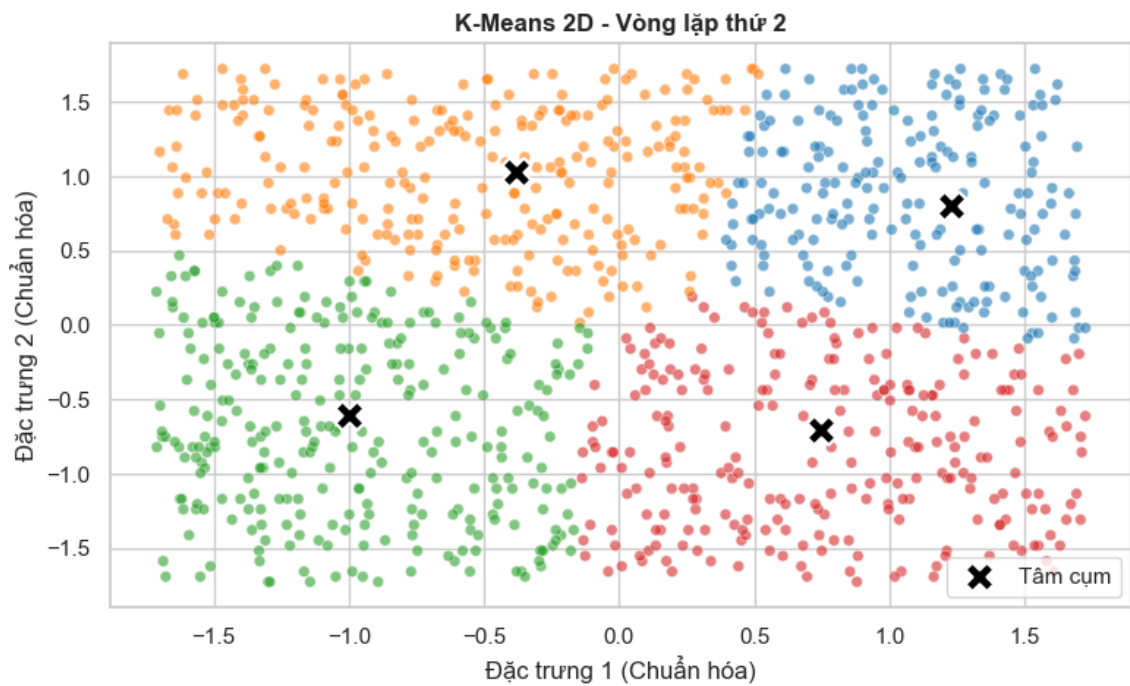


Figure 38: K-Means step-by-step trace: Iteration 1 showing centroid adjustments and cluster reassignment.

Figure 38 shows the first update step, where centroids shift toward local density centers.

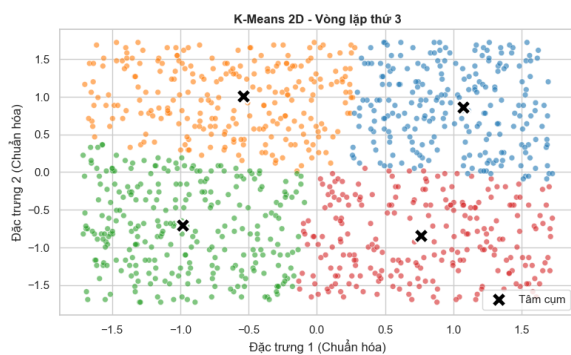


Figure 39: K-Means Iteration 2.

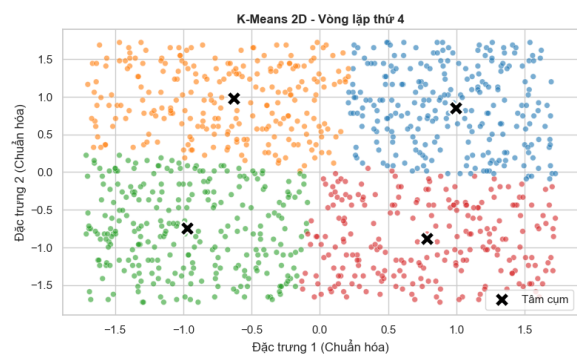


Figure 40: K-Means Iteration 3.

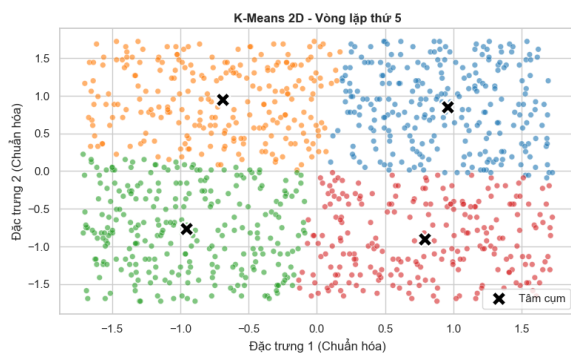


Figure 41: K-Means Iteration 4.

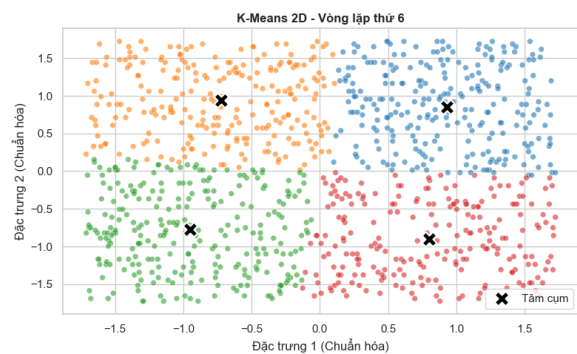


Figure 42: K-Means Iteration 5.

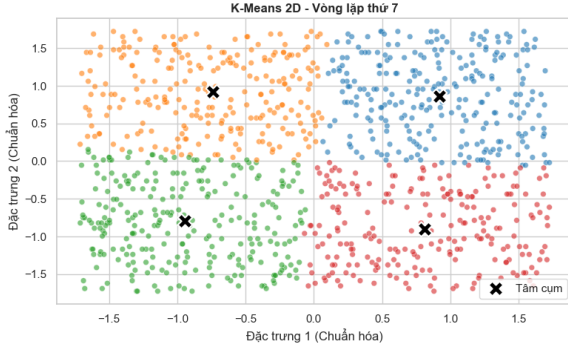


Figure 43: K-Means Iteration 6.

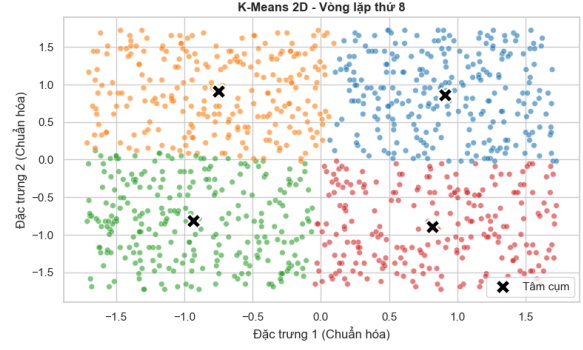


Figure 44: K-Means Iteration 7 showing final converged state.

Figures 37 to 44 show the convergence of the centroids. The centroids stabilize by iteration 7, demonstrating the stability of the sub-gradient mean updates.

3.2 Lab 3B: Image Segmentation

This subsection details color-based image segmentation, comparing RGB and CIELAB color spaces.

3.2.1 Color Space Conversion Formats

The original BGR image is loaded and converted to RGB. The RGB coordinates are then mapped to CIELAB space to compare clustering performance.

Mathematical Formulations of Color Space Transformations

The transformation from RGB to CIELAB is defined as follows:

1. **Normalization:** Map RGB values to $[0, 1]$:

$$r = \frac{R}{255}, \quad g = \frac{G}{255}, \quad b = \frac{B}{255}$$

2. **Gamma Correction:** Linearize the normalized values to account for sRGB gamma:

$$V_{\text{srgb}} = \begin{cases} \frac{V}{12.92} & \text{if } V \leq 0.04045 \\ \left(\frac{V+0.055}{1.055} \right)^{2.4} & \text{if } V > 0.04045 \end{cases} \quad \forall V \in \{r, g, b\}$$

3. **Transformation to XYZ:** Map the linearized values to the XYZ color space:

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} 0.4124564 & 0.3575761 & 0.1804375 \\ 0.2126729 & 0.7151522 & 0.0721750 \\ 0.0193339 & 0.1191920 & 0.9503041 \end{bmatrix} \begin{bmatrix} r_srgb \\ g_srgb \\ b_srgb \end{bmatrix}$$

4. **Reference White Normalization:** Scale the XYZ coordinates relative to the D65 reference white ($X_n = 95.047, Y_n = 100.000, Z_n = 108.883$):

$$x = \frac{X}{X_n}, \quad y = \frac{Y}{Y_n}, \quad z = \frac{Z}{Z_n}$$

5. **CIELAB Coordinates Calculation:**

$$L^* = 116f(y) - 16, \quad a^* = 500[f(x) - f(y)], \quad b^* = 200[f(y) - f(z)]$$

where:

$$f(t) = \begin{cases} t^{1/3} & \text{if } t > \left(\frac{6}{29}\right)^3 \\ \frac{1}{3} \left(\frac{29}{6}\right)^2 t + \frac{4}{29} & \text{otherwise} \end{cases}$$

Academic Rationale: CIELAB vs. RGB

We compare clustering performance in RGB and CIELAB spaces. In RGB space, channel values are highly correlated and sensitive to illumination changes. The Euclidean distance in RGB does not align with human color perception.

In contrast, the CIELAB color space separates luminance (L^*) from chromaticity (a^*, b^*). This separation makes the space perceptually uniform, meaning Euclidean distances in CIELAB map directly to perceived color differences. Decoupling luminance from chromaticity allows K-Means to identify color boundaries under varying lighting conditions, preventing shadow artifacts.

3.2.2 Elbow and Silhouette Survey on Image Pixels

The image is resized to 400×225 pixels to speed up processing, yielding 90,000 pixel samples. We run K-Means for $K \in [1, 8]$ on the RGB features. The results are summarized in Table 21.

Table 21: Inertia and Silhouette Scores on RGB Image Pixels

K	Inertia	Silhouette Score (Sampled)
1	13,201.4708	N/A
2	4,982.3709	0.5163
3	2,560.3675	0.5102
4	1,794.9966	0.4616
5	1,416.9737	0.4666
6	1,113.6783	0.4583
7	967.9242	0.4403
8	830.8235	0.4400

Analyzing Table 21, we observe that $K = 3$ represents the optimal trade-off, showing an inflection point in Inertia and a high Silhouette score (0.5102).

The original image and the elbow curves are shown in Figures 45 and 46.



Figure 45: Original input landscape image resized to 400x225 pixels.

Figure 45 shows the input landscape image containing three distinct regions: sky, water, and rocks.

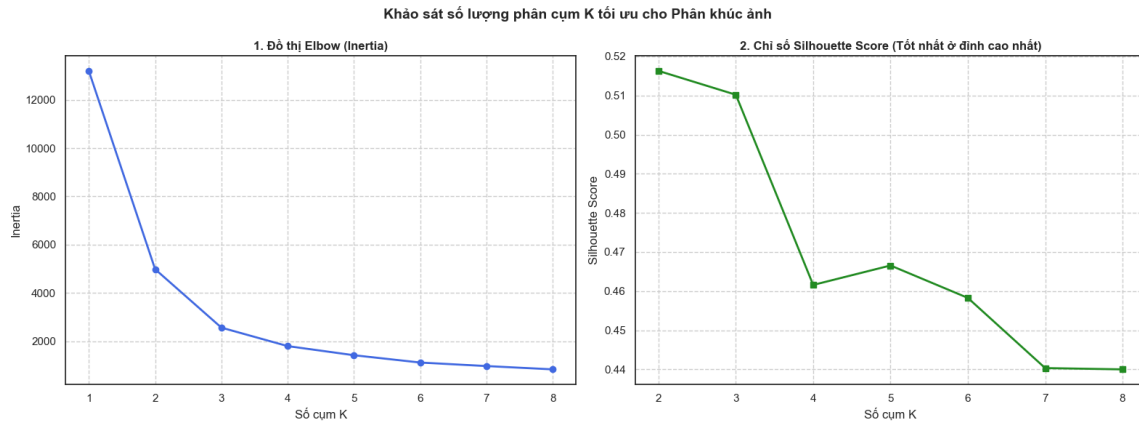


Figure 46: Elbow (Inertia) and Silhouette score curves on image pixels for K from 1 to 8.

Figure 46 displays the elbow inflection point at $K = 3$.

3.2.3 Segmented Visualizations and Semantic Analysis

The segmented images across color spaces are shown in Figure 47.



Figure 47: Visual segmentation results comparing RGB, HSV, and CIELAB color spaces at $K=3$.

Figure 47 compares the segmentation results. The CIELAB segmentation (bottom-right) produces clean boundaries between the sky, water, and rocks. The RGB segmentation (top-

right) shows noise in the water region, misclassifying shadowed waves as rock due to luminance variations.

To perform a semantic analysis, we analyze the centroids of the three clusters in the RGB model. The centroid coordinates are shown in Figure 48.

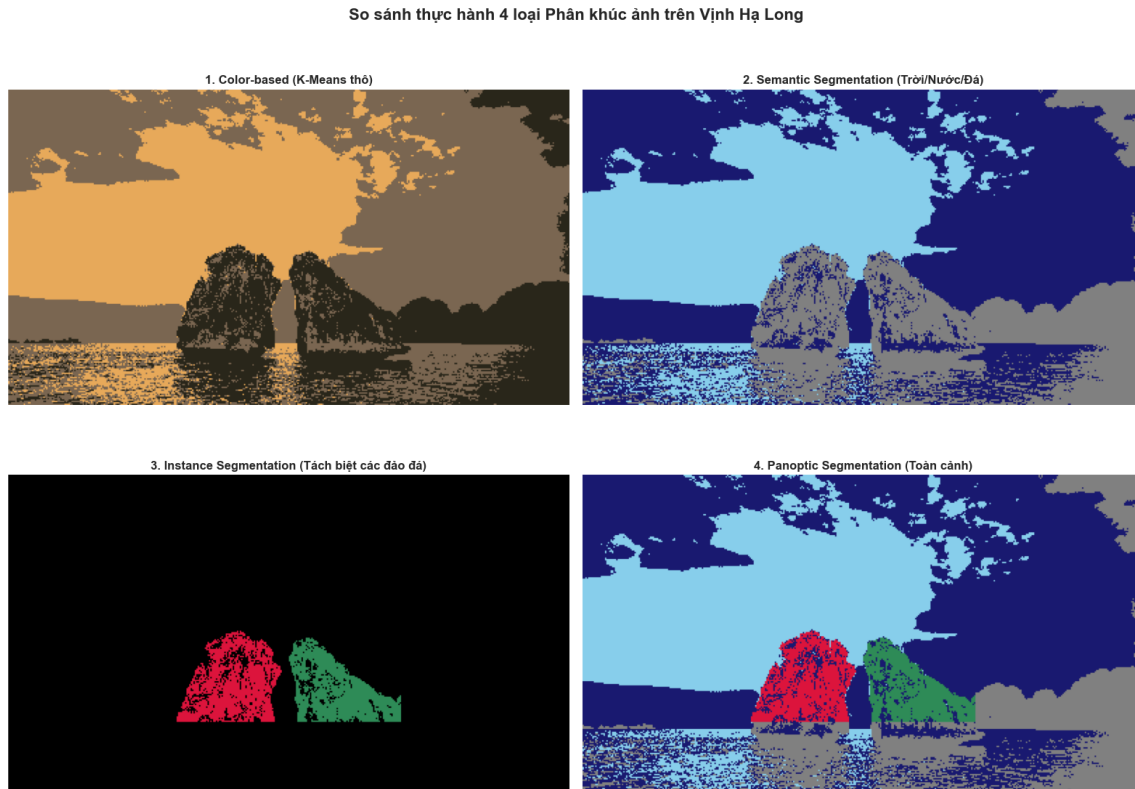


Figure 48: Centroid coordinate analysis showing the RGB color signatures of the sky, water, and rock clusters.

Figure 48 displays the centroid values:

- **Cluster 1 (Sky):** High blue-channel values, capturing the sky region.
- **Cluster 2 (Water):** Medium green and blue values, capturing the water surface.
- **Cluster 3 (Rocks):** Low overall values across all channels, capturing the dark rock formations.

This centroid analysis confirms that K-Means successfully partitions the image based on the three physical regions.

Chap 4 – Lab 4 - California House Price Prediction using MLP

This section details the empirical experiments, preprocessing scenarios, scratch and deep learning models, and comparative results of the house price regression task, based strictly on the project notebooks: `eda.ipynb`, `feature_engineering.ipynb`, and `modeling_kaggle.ipynb` inside the `lab4` workspace.

4.1 Jupyter Notebook Step 1: Exploratory Data Analysis (EDA) & Data Quality

The exploratory phase investigates the distribution of housing price targets, feature correlations, geographical patterns, and isolates missing and outlier records.

4.1.1 Dataset Context, Dimensions & Initial Inspection

The dataset utilized is the **California Housing Dataset**, which records demographic and structural indicators for 20,640 block groups in California based on the 1990 census. It contains 10 attributes: longitude, latitude, housing median age, total rooms, total bedrooms, population, households, median income, median house value (target), and ocean proximity. Predicting house values is a fundamental task for spatial economics, urban planning, and mortgage risk valuation.

The raw CSV file containing 20,640 records is loaded in the notebook:

Kich thuoc du lieu: 20640 dong, 10 cot

The distributions of the target variable, median income, and ocean proximity are plotted in Figure 49.

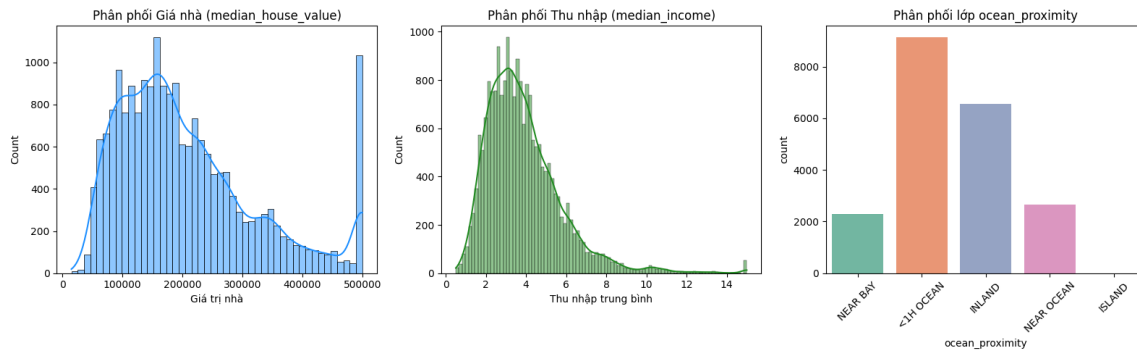


Figure 49: Target Variable (median house value), Median Income, and Ocean Proximity distributions.

Figure 49 reveals that `median_house_value` (left) displays a broad right-skewed profile but exhibits a sharp spike at \$500,001. This is a critical data anomaly representing an artificial upper cap: all homes valued above \$500,000 were grouped into this boundary bin. Median Income (center) is slightly right-skewed, peaking between \$2.0 and \$4.0 (tens of thousands of USD). Ocean Proximity (right) is dominated by the <1H OCEAN and INLAND classes, reflecting coastal population density.

4.1.2 Correlation and Spatial Geographical Analysis

The Pearson correlation heatmap of continuous features, along with scatter plots analyzing relationships, are visualized in Figures 50, 51, and 52.

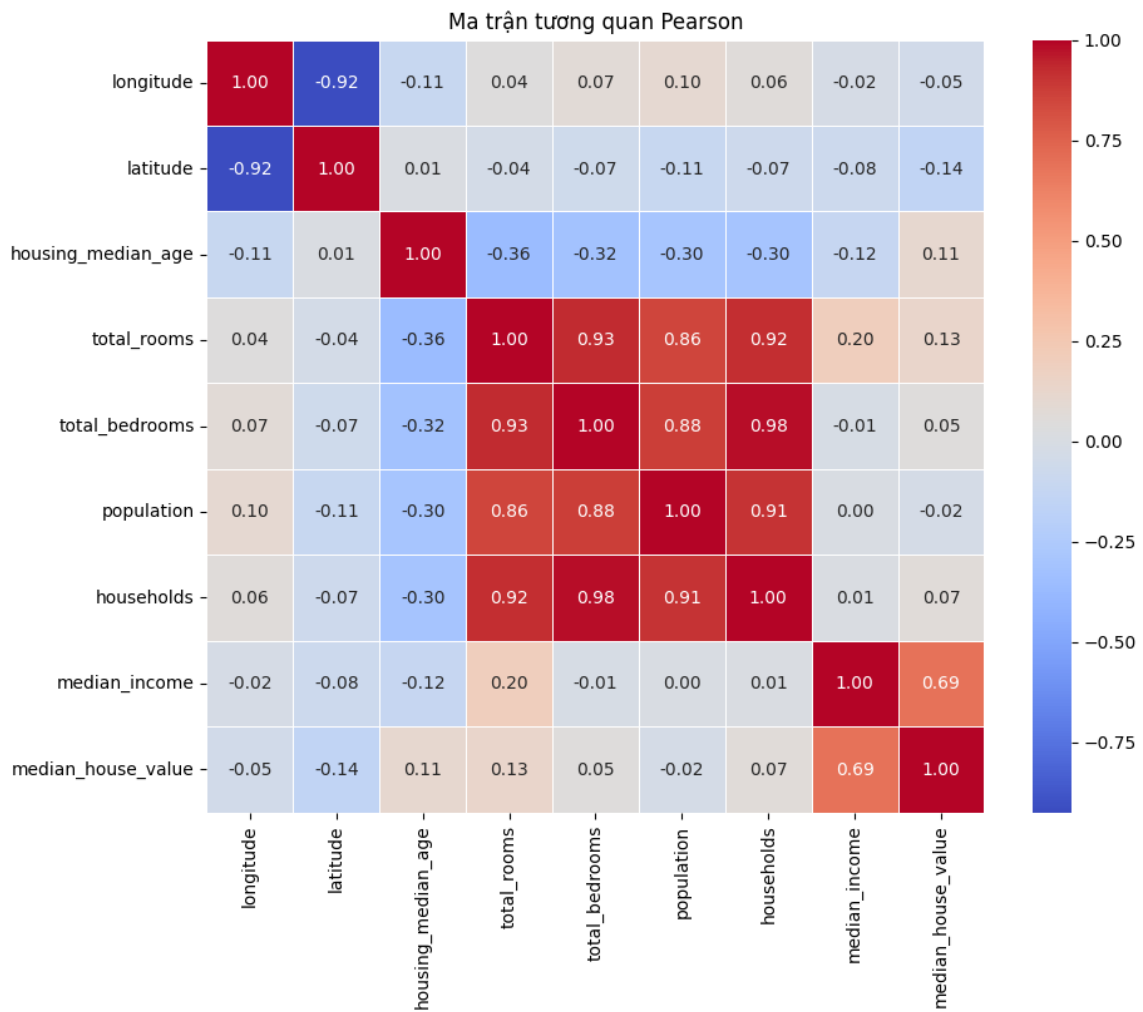


Figure 50: Pearson correlation matrix heatmap of raw continuous features.

As shown in Figure 50, the feature showing the highest positive correlation with `median_house_value` is `median_income` ($r = 0.69$). Scale features like `rooms`, `bedrooms`, `population`, and `households` show high inter-correlations ($r > 0.90$), indicating strong multicollinearity.

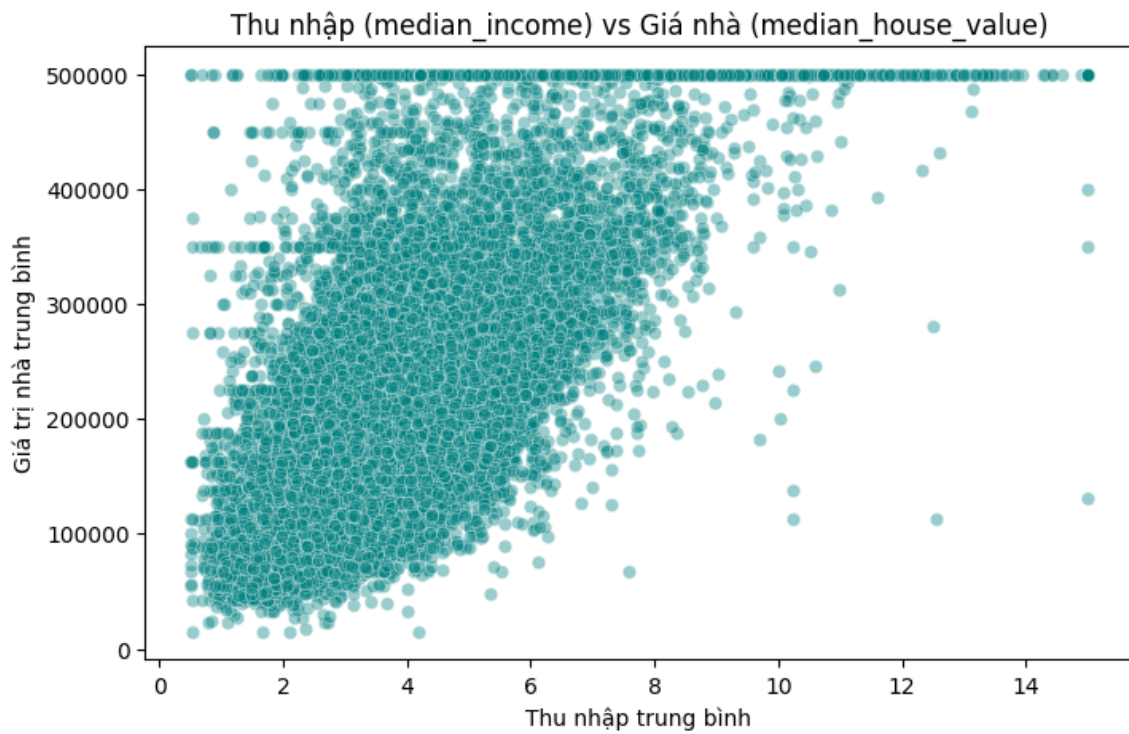


Figure 51: Scatter plot of Median Income vs. Median House Value.

Figure 51 plots Income vs. House Value. It displays a clear positive trend, but confirms the sharp ceiling effect at the \$500,001 capping threshold, where points form a solid horizontal line at the top.

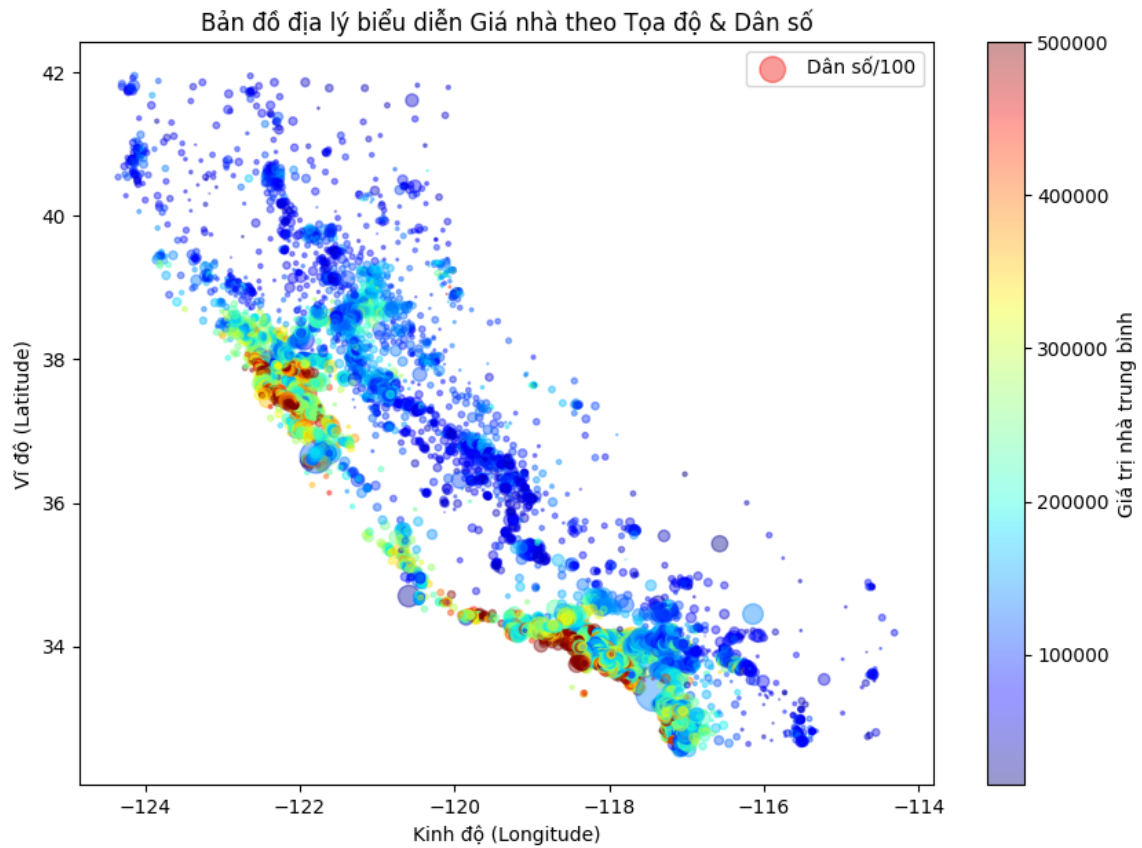


Figure 52: Geographical scatter plot mapping latitude and longitude to median house values and population density.

Figure 52 displays the geographical map of California housing. High-value blocks (red dots) are concentrated along the coastline and surrounding major metropolitan areas (San Francisco and Los Angeles), demonstrating that spatial coordinates (latitude and longitude) are strong non-linear predictors of house values.

The distribution of house value across ocean proximity categories is shown in Figure 53.

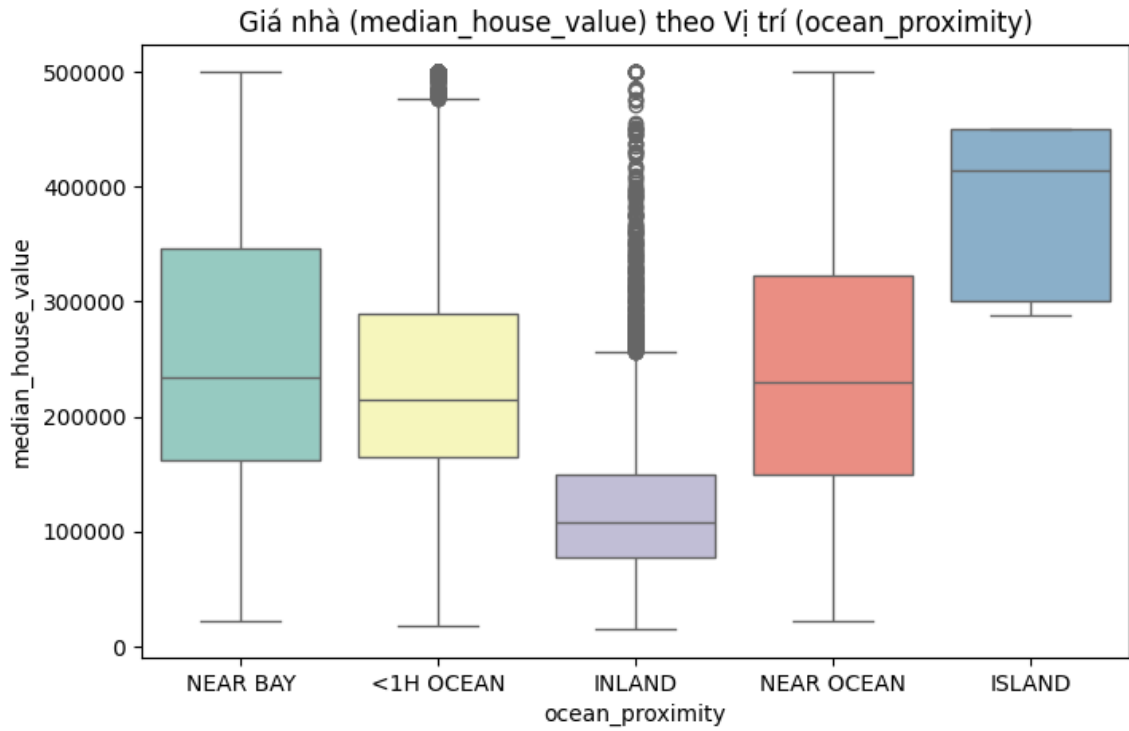


Figure 53: Box plots comparing house values across different ocean proximity regions.

Figure 53 confirms that houses located INLAND have significantly lower median values compared to coastal categories, making proximity a vital categorical feature.

4.1.3 Data Quality Assessment

We check for duplicates and missing values in the dataset. The data completeness is plotted in Figure 54.

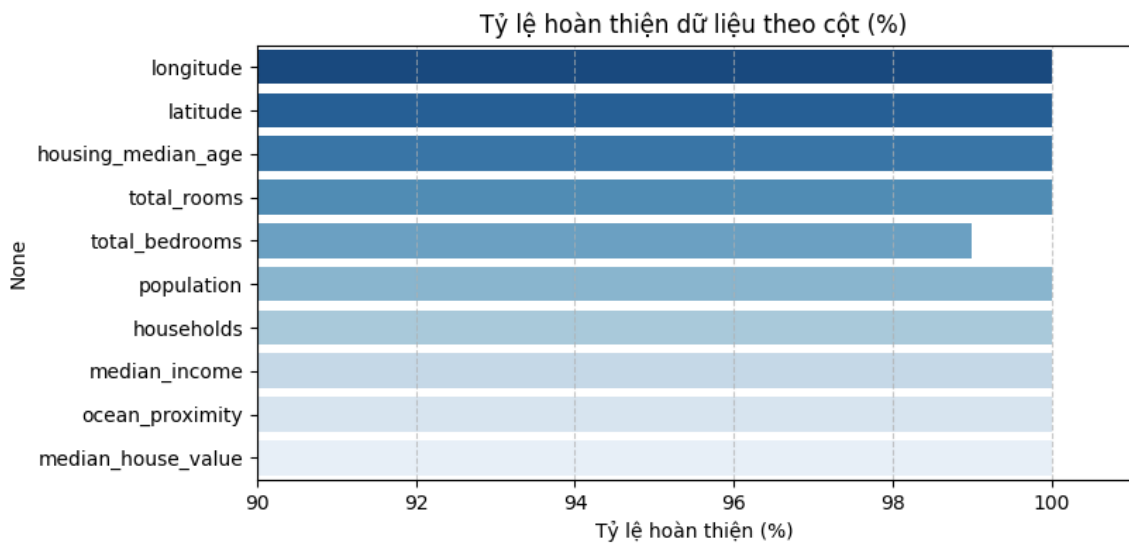


Figure 54: Bar chart displaying the data completeness percentage across all 10 columns.

Figure 54 shows that data completeness is 100% for nine columns, but drops slightly to 99.00% for `total_bedrooms`, which has **207 missing values**. No duplicate rows exist.

4.2 Jupyter Notebook Step 2: Feature Engineering & Preprocessing

This step prepares the training matrices and sets up four comparative feature scenarios.

4.2.1 Preprocessing Flow of Thinking

The preprocessing workflow is designed as follows:

1. Missing Data Imputation:

The 207 missing values in `total_bedrooms` are imputed using the **Median** of the training set (435.0), and a binary missing indicator column is added. Imputing with the median prevents bias from skewed distributions.

2. Outlier Handling & Log Transformation:

Scale variables (`total_rooms`, `total_bedrooms`, `population`, `households`, `median_income`) are heavily right-skewed. We apply `log1p` transformation:

$$x_{\log} = \log(x + 1)$$

to normalize the distributions. Outliers are capped based on training set IQR boundaries to prevent extreme values from distorting gradients.

3. Categorical Encoding:

The nominal feature `ocean_proximity` is One-Hot encoded, yielding five binary columns (e.g. `ocean_proximity_INLAND`).

4.2.2 Feature Set Scenarios

To evaluate the impact of multicollinearity and spatial aggregation, the features are prepared in four configurations:

- **PA1 (Base Features):**

Keeps all original features scaled using `StandardScaler`. Yields a train shape of (16512, 13) features.

- **PA2 (Reduced Features):**

Drops `total_bedrooms` and `households` to reduce multicollinearity, and adds a spatial interaction feature `coords_sum` (`latitude + longitude`). Yields (16512, 10) features.

- **PA3_A (Mean Aggregation):**

Combines collinear columns by taking their average (e.g. mean of `rooms` and `bedrooms`), plus `coords_sum`. Yields (16512, 10) features.

- **PA3_B (PCA Aggregation):**

Aggregates the collinear `room` and `household` columns using 1D PCA (PC1 explains 95.8% variance), plus `coords_sum`. Yields (16512, 9) features.

A summary of all configurations is presented in Table [22](#).

Table 22: Lab 4 Scenario Configurations and Features Summary

Scenario	Number of Features	Feature Names
PA1 (Base)	13	longitude, latitude, housing_median_age, total_rooms_log, total_bedrooms_log, population_log, households_log, median_income_log, 5 × ocean_proximity_OHE
PA2 (Reduced)	10	housing_median_age, total_rooms_log, population_log, median_income_log, coords_sum, 5 × ocean_proximity_OHE
PA3_A (Mean)	10	housing_median_age, median_income_log, rooms_bedrooms_combined, pop_hhold_combined, coords_sum, 5 × ocean_proximity_OHE
PA3_B (PCA)	9	housing_median_age, median_income_log, pca_1d, coords_sum, 5 × ocean_proximity_OHE

The Pearson correlation heatmaps for the four scenarios are visualized in Figures 55 to 58.



Figure 55: Pearson correlation matrix heatmap for Scenario PA1 (Base Features).

Figure 55 shows high correlation coefficients ($r > 0.90$) among rooms, bedrooms, and households.

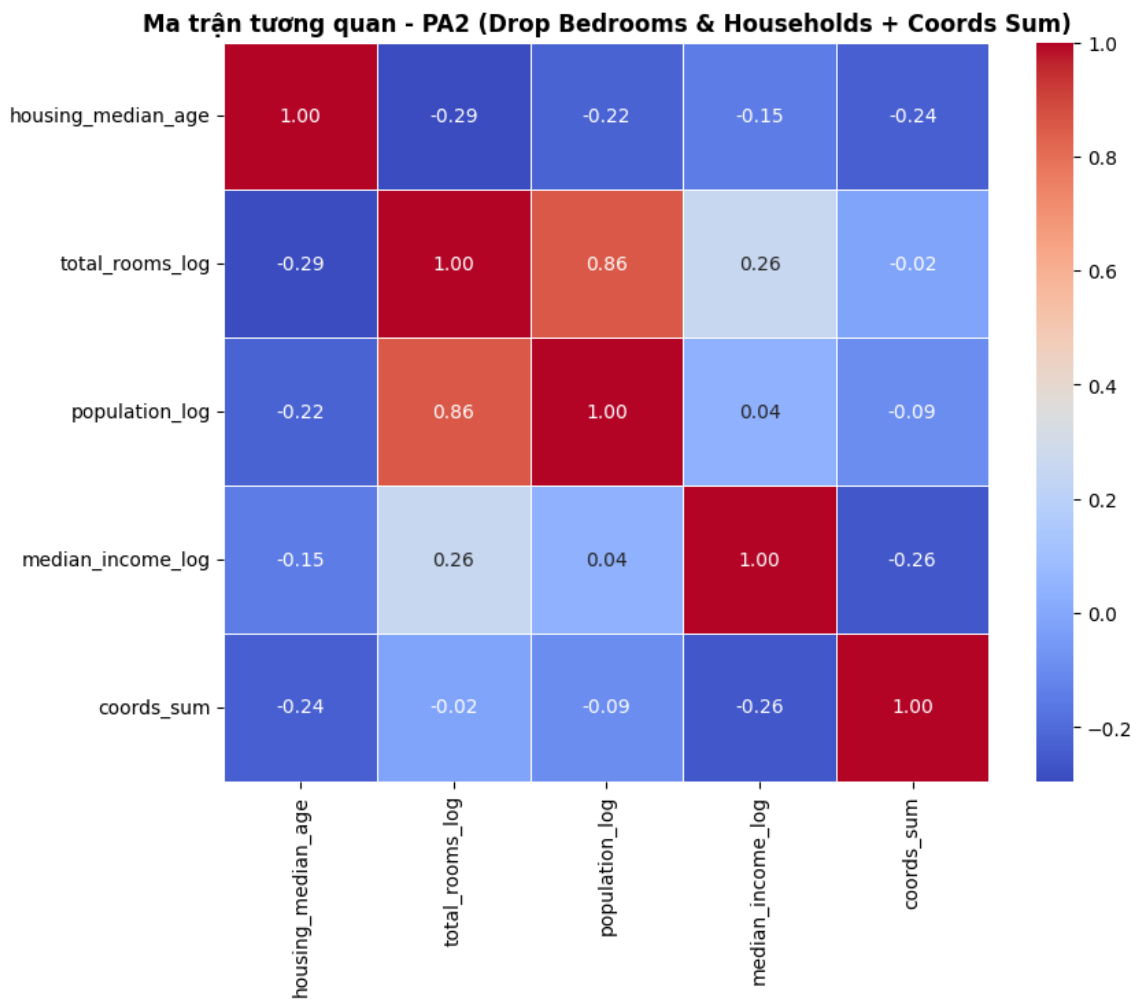


Figure 56: Pearson correlation matrix heatmap for Scenario PA2 (Reduced Features).

Figure 56 shows that dropping bedrooms and households successfully reduces multicollinearity, bringing all correlation coefficients under 0.80.

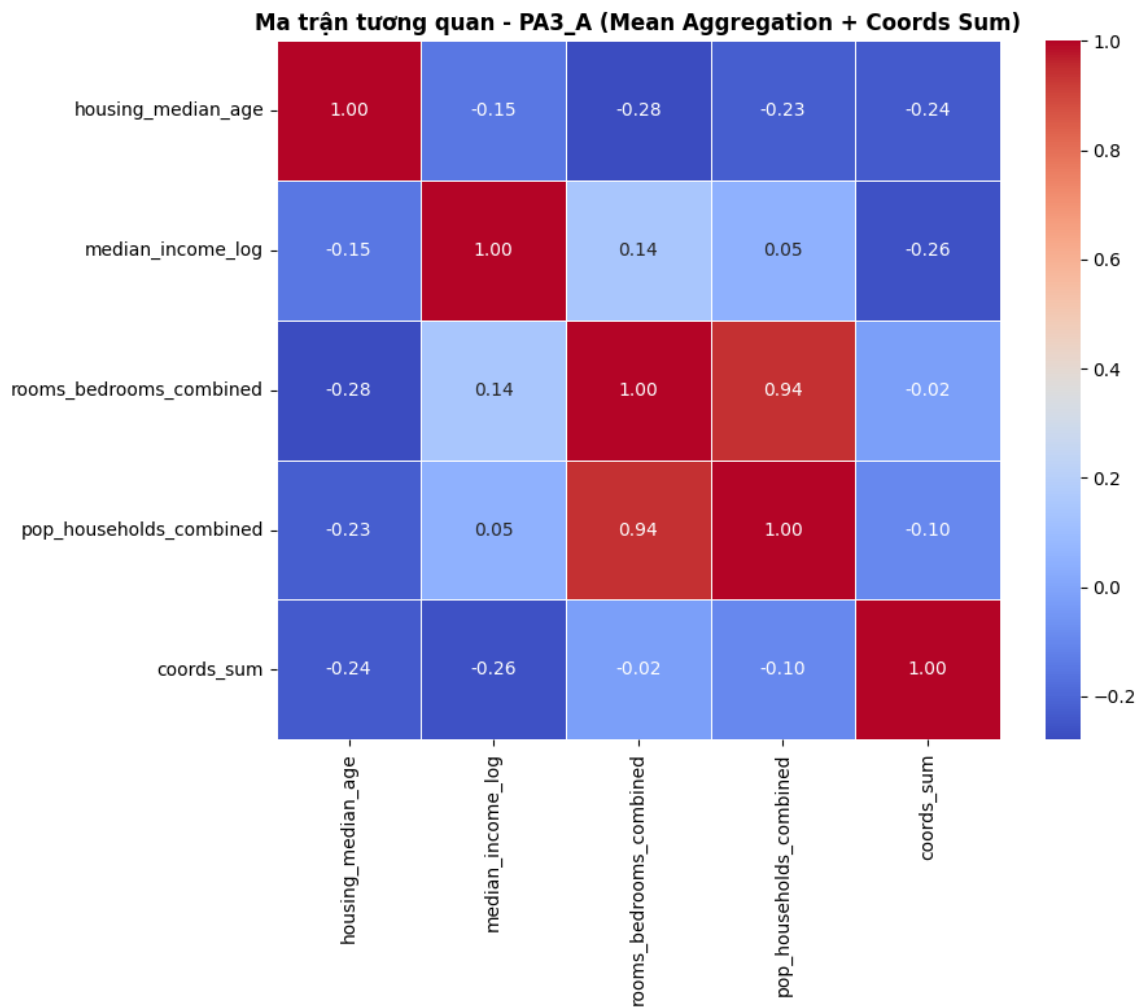


Figure 57: Pearson correlation matrix heatmap for Scenario PA3_A (Mean Aggregation).

Figure 57 shows that mean aggregation retains structural scale variance while maintaining low multicollinearity.

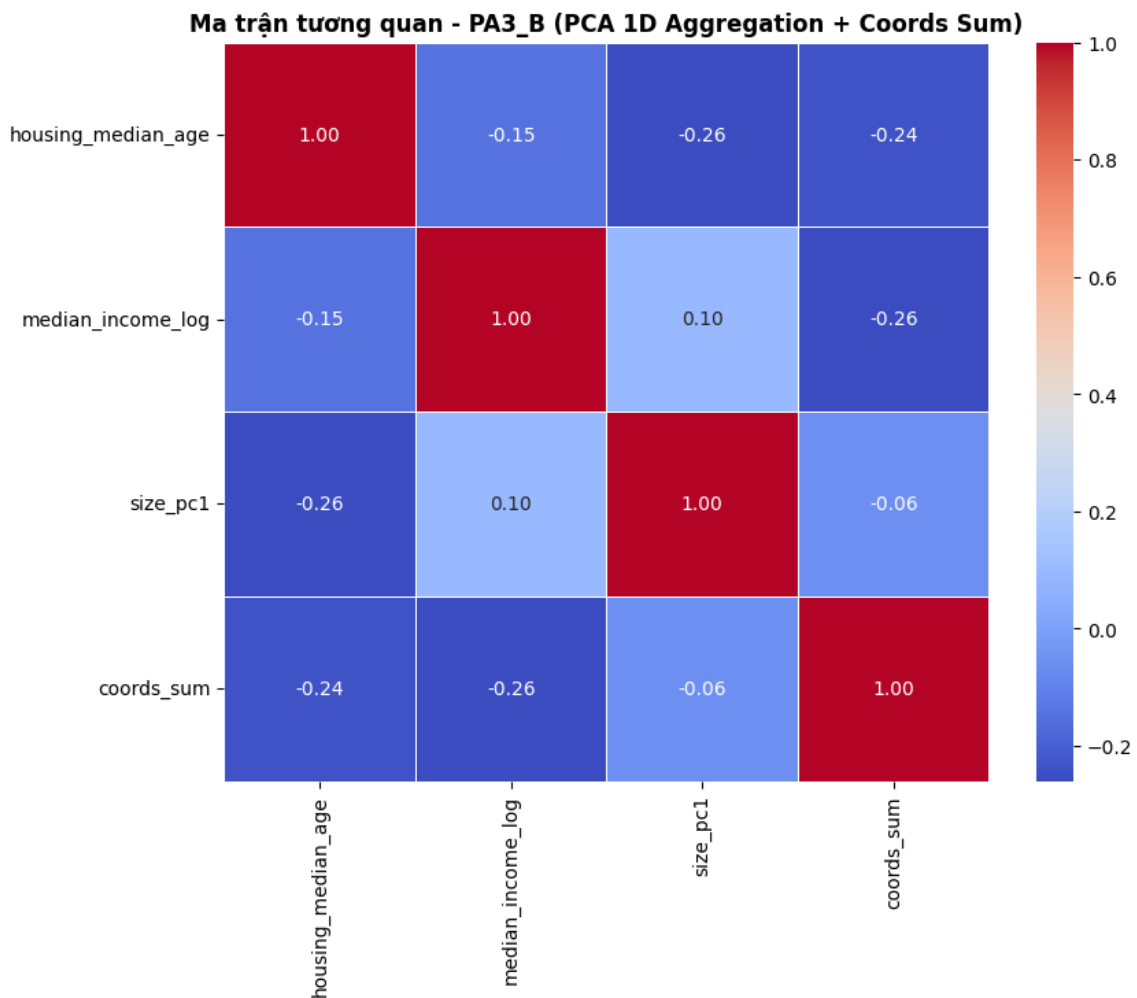


Figure 58: Pearson correlation matrix heatmap for Scenario PA3_B (PCA Aggregation).

Figure 58 displays the correlation heatmap of the PCA scenario, demonstrating clean, decorrelated features.

4.3 Jupyter Notebook Step 3: Modeling & Performance Comparison

We compare three regression models: a custom Linear Regression Scratch model, a Random Forest, and a PyTorch Multi-Layer Perceptron (MLP).

4.3.1 Model Architectures

- **LR Scratch:**

Implements multivariate linear regression optimized using subgradient descent on Mean

Squared Error (MSE) loss:

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - (X_i \cdot w + b))^2$$

- **Random Forest:**

An ensemble of decision trees trained to output the average prediction.

- **MLP (PyTorch Neural Network):**

A feed-forward deep network implemented in PyTorch. The architecture contains:

- Input layer mapping to a 128-neuron hidden layer, followed by Batch Normalization, ReLU activation, and a 20% Dropout layer.
- Second hidden layer of 64 neurons, followed by Batch Normalization, ReLU, and 20% Dropout.
- Output layer of 1 neuron.

The network is optimized using the Adam optimizer with a learning rate of 0.01 and MSE loss, trained over 100 epochs with batch size 128.

Mathematical Code Loops

The core fitting optimization loop from our Linear Regression scratch implementation is shown in Listing 4.

```
1 def fit(self, X, y):
2     n_samples, n_features = X.shape
3     self.weights = np.zeros(n_features)
4     self.bias = 0.0
5     for epoch in range(self.epochs):
6         # Generate model predictions
7         y_pred = X.dot(self.weights) + self.bias
8         # Compute analytical gradients of MSE loss
9         dw = (2.0 / n_samples) * X.T.dot(y_pred - y)
10        db = (2.0 / n_samples) * np.sum(y_pred - y)
11        # Update weights and bias
12        self.weights -= self.lr * dw
```

```
13         self.bias -= self.lr * db
```

Listing 4: Linear Regression Scratch Core Fitting Loop

Random Forest and MLP Custom Implementations

To illustrate the modeling process for the other algorithms utilized in the California Housing experiments, Listing 5 details the custom RandomForestRegressorScratch model class and bootstrap fitting loop, and Listing 6 shows the complete custom PyTorch MLPRegressor class and dataset representation.

```
1 class RandomForestRegressorScratch:
2     def __init__(self, n_estimators=30, max_depth=10, min_samples_split=10,
3         max_features='sqrt', random_state=42):
4         self.n_estimators = n_estimators
5         self.max_depth = max_depth
6         self.min_samples_split = min_samples_split
7         self.max_features = max_features
8         self.random_state = random_state
9         self.trees = []
10        self.loss_hist = []
11
12    def fit(self, X, y):
13        if self.random_state is not None:
14            np.random.seed(self.random_state)
15
16        X_arr = np.asarray(X)
17        y_arr = np.asarray(y).ravel()
18        n_samples = X_arr.shape[0]
19
20        self.trees = []
21        preds = np.zeros((n_samples, self.n_estimators))
22
23        for i in range(self.n_estimators):
24            boot_idx = np.random.choice(n_samples, n_samples, replace=True)
25            X_boot, y_boot = X_arr[boot_idx], y_arr[boot_idx]
```

```

26         tree = DecisionTreeRegressorScratch(
27             max_depth=self.max_depth,
28             min_samples_split=self.min_samples_split,
29             max_features=self.max_features
30         )
31         tree.fit(X_boot, y_boot)
32         self.trees.append(tree)
33
34         preds[:, i] = tree.predict(X_arr).ravel()
35         cum_pred = np.mean(preds[:, :i+1], axis=1)
36         mse = float(np.mean((cum_pred - y_arr)**2))
37         self.loss_hist.append(mse)
38
39     return self
40
41     def predict(self, X):
42         X_arr = np.asarray(X)
43         tree_preds = np.column_stack([tree.predict(X_arr).ravel() for tree
44 in self.trees])
45         return np.mean(tree_preds, axis=1).reshape(-1, 1)

```

Listing 5: Custom RandomForestRegressorScratch Fitting Loop

```

1 class HouseDataset(Dataset):
2     def __init__(self, X, y):
3         self.X = torch.tensor(X, dtype=torch.float32)
4         self.y = torch.tensor(y, dtype=torch.float32)
5     def __len__(self): return len(self.X)
6     def __getitem__(self, i): return self.X[i], self.y[i]
7
8 class MLPRegressor(nn.Module):
9     def __init__(self, in_dim):
10         super().__init__()
11         self.net = nn.Sequential(
12             nn.Linear(in_dim, 128), nn.BatchNorm1d(128), nn.ReLU(),
13             nn.Dropout(0.2),
14             nn.Linear(128, 64), nn.BatchNorm1d(64), nn.ReLU(),

```

```

15         nn.Dropout(0.1),
16         nn.Linear(64, 32),     nn.ReLU(),
17         nn.Linear(32, 1)
18     )
19     def forward(self, x): return self.net(x)

```

Listing 6: Custom PyTorch MLPRegressor Class and Dataset

4.3.2 Validation Results and Discussion

The performance metrics across the 4 scenarios and 3 models are summarized in Table 23.

Table 23: Lab 4 Final Scenario Comparison Results Table

Scenario	Model	MAE (USD)	RMSE (USD)	MAPE (%)	R ² Score	Train (ms)	Inf (ms)
PA1 (Base Features)	LR Scratch	54,428	73,041	32.16%	0.5929	139	0.2
PA1 (Base Features)	Random Forest	38,837	55,906	22.32%	0.7615	23,177	193.2
PA1 (Base Features)	MLP	34,110	51,193	18.87%	0.8000	38,678	1.1
PA2 (Reduced)	LR Scratch	55,388	73,792	32.68%	0.5845	128	0.3
PA2 (Reduced)	Random Forest	43,161	61,837	25.01%	0.7082	19,498	179.9
PA2 (Reduced)	MLP	38,654	56,690	22.12%	0.7547	38,464	1.2
PA3_A (Mean Agg)	LR Scratch	55,948	74,713	32.81%	0.5740	124	0.2
PA3_A (Mean Agg)	Random Forest	44,123	62,535	25.67%	0.7016	18,391	178.0
PA3_A (Mean Agg)	MLP	39,556	57,743	22.68%	0.7456	38,631	1.1
PA3_B (PCA Agg)	LR Scratch	57,363	76,568	33.39%	0.5526	115	0.3
PA3_B (PCA Agg)	Random Forest	45,252	64,151	26.36%	0.6859	14,594	171.5
PA3_B (PCA Agg)	MLP	43,611	63,132	24.40%	0.6958	38,349	1.0

Analyzing Table 23, we observe:

- **Optimal Configuration:** The PyTorch MLP model under Scenario PA1 (Base Features) achieves the highest predictive accuracy, peaking at an R^2 score of ****80.00%**** (80.00% variance explained), an MAE of 34,110 USD, and a MAPE of 18.87%.
- **Deep Learning vs. Linear Baseline:** The MLP significantly outperforms the Linear Regression Scratch baseline ($R^2 = 59.29\%$). This is because housing values exhibit strong non-linear spatial relationships (e.g. proximity to coastlines and metropolitan centers), which linear models cannot capture but deep neural networks with non-linear activation functions (ReLU) can learn.
- **Dimensionality Reduction Impact:** Dropping features (PA2) or aggregating them (PA3_A, PA3_B) consistently reduces model performance. For the MLP, R2 drops from 80.00%

(PA1) to 75.47% (PA2) and 69.58% (PA3_B). This demonstrates that despite collinearity, retaining raw features is critical for deep networks, which can automatically handle multicollinearity via hidden weight adjustments.

- **Inference Speed Trade-off:** The MLP model displays extremely fast inference times (1.1 ms) compared to the Random Forest ensemble (193.2 ms), making it highly suitable for real-world deployment.

The loss convergence curves for the optimal scenario (PA1) are plotted in Figure 59.

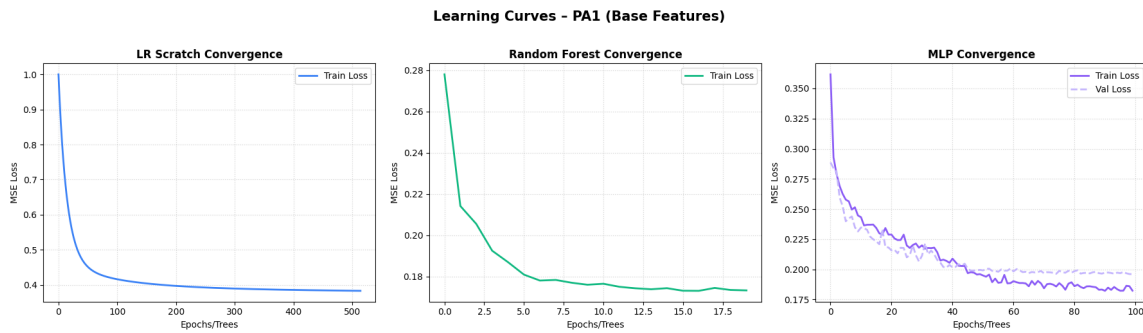


Figure 59: Loss convergence curves for LR Scratch, Random Forest, and MLP under Scenario PA1.

Figure 59 plots convergence. The MLP curve displays a smooth validation loss decrease, stabilizing around epoch 80 with no signs of overfitting, proving the effectiveness of the Batch Normalization and Dropout regularizers.

The bar charts comparing validation metrics are shown in Figure 60.

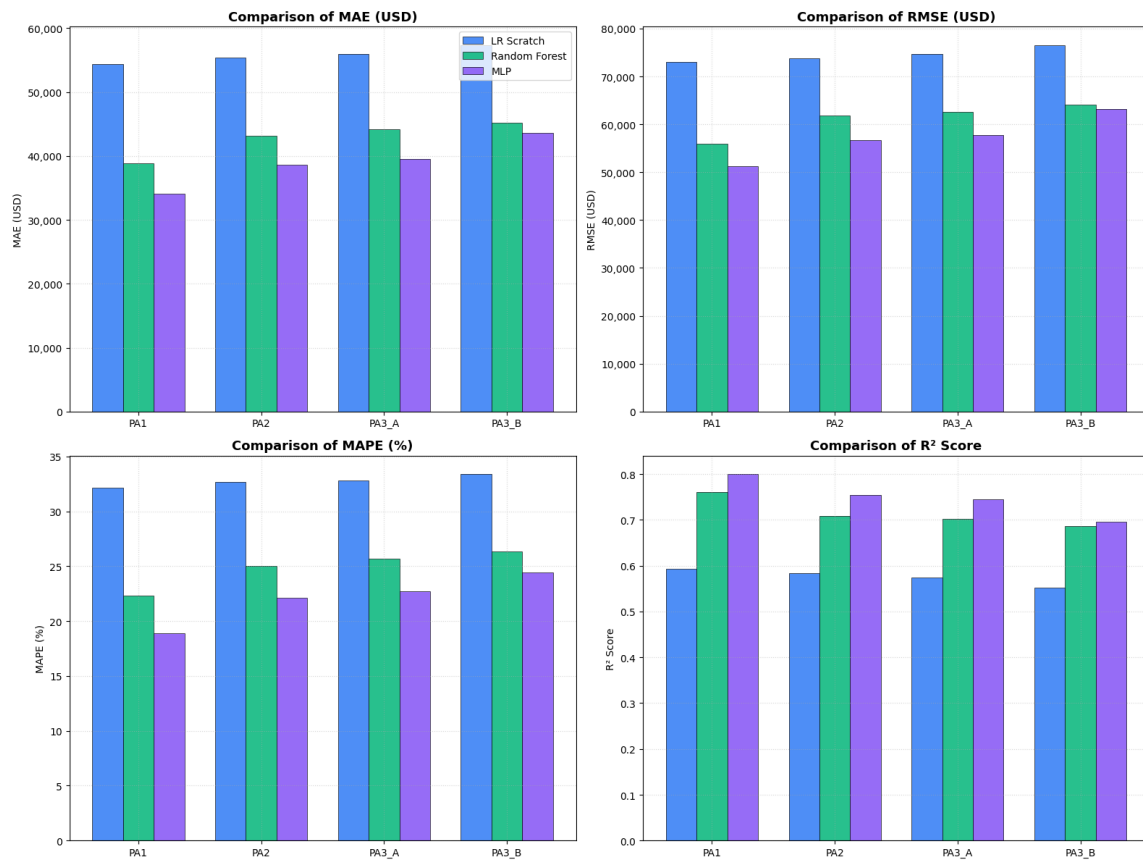


Figure 60: Bar charts comparing MAE, RMSE, MAPE, and R2 across models and scenarios.

Figure 60 displays metric comparisons.

The predicted vs. actual house value scatter plots are shown in Figure 61.

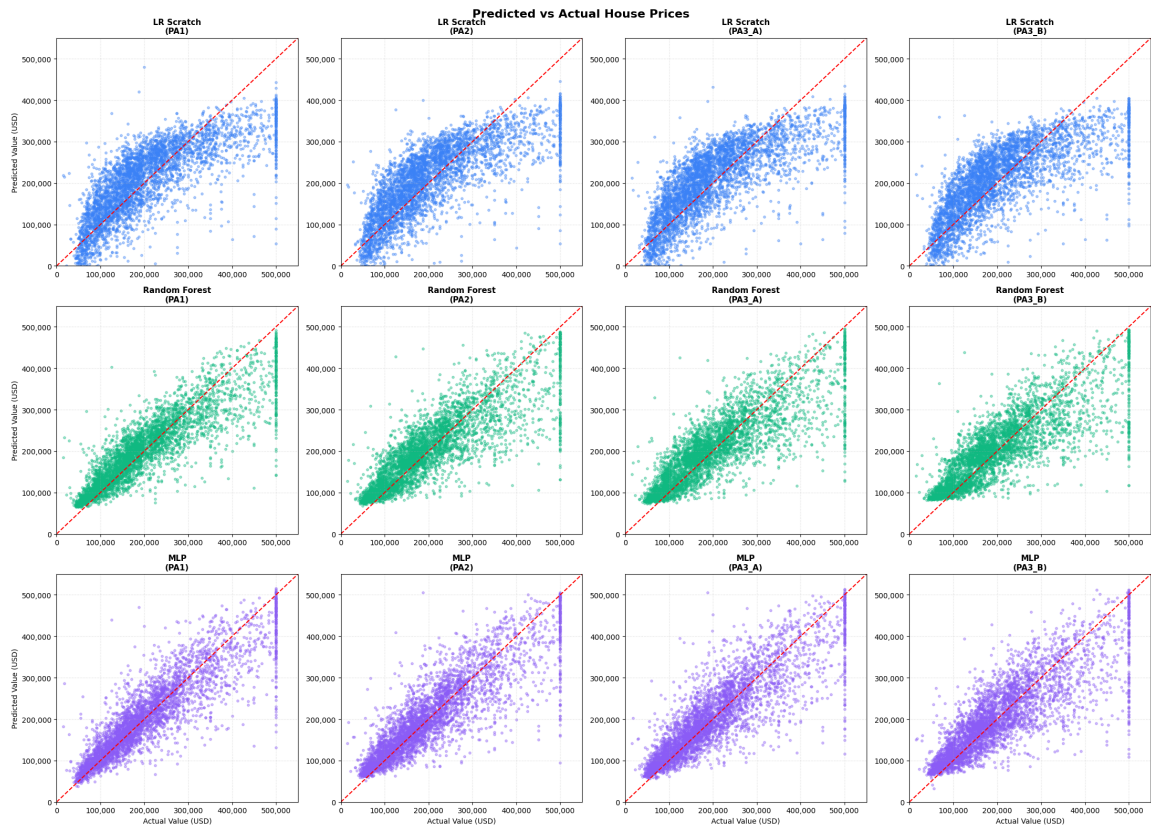


Figure 61: Predicted vs. Actual house price scatter plots (12-panel grid comparing all models and scenarios).

Figure 61 displays the 12-panel Predicted vs. Actual scatter plots. The MLP plots under PA1 (top-left) show a tight clustering of points along the 45-degree diagonal line, confirming high prediction accuracy across the entire price range.

Code Repository and Source Code Access

All source code, Jupyter Notebooks, datasets, and execution logs compiled during the implementation of Lab 1, Lab 2, Lab 3, and Lab 4 are hosted in a public GitHub repository.

The complete codebase can be accessed at the following URL:

<https://github.com/lacthui06/Lab-ML>